



中国研究生创新实践系列大赛
“华为杯”第二十届中国研究生
数学建模竞赛

学校

中南大学

参赛队号

23105330424

1.史代双

队员姓名

2.梁时清

3.华明清

中国研究生创新实践系列大赛

“华为杯”第二十届中国研究生

数学建模竞赛

题 目 出血性脑卒中临床智能诊疗建模

摘 要：

本文主要针对出血性脑卒中的若干医疗数据进行量化分析，进而发掘出血性脑卒中的发病风险，整合影像学特征、患者临床信息及临床诊疗方案，构建精准预测患者预后的模型，并据此优化临床决策，为改善出血性脑卒中患者预后作出了贡献。文章综合采用了**基于随机森林的预测模型、基于 DTW 距离的 K-Means 算法、基于 XGBoost 的回归模型、自定义特征提取**等方法来对出血性脑卒中诊疗进行建模。

针对问题一，原始数据变量类型多、纬度高、多张表之间相互关联以及存在不少缺失值、数据类别之间不平衡等问题，因此首先要**联合多张表的信息进行数据预处理和数据整合**。再根据每位病人的历史数据判断其是否在 48 小时内发生了血肿扩张，并将结果记录在表中，作为后续预测任务训练样本的标签值。随后构建了**基于随机森林的预测模型**。将前 100 例患者的个人史，疾病史，发病相关、首次影像检查结果等作为输入变量，是否发生血肿扩张作为输出变量进行模型训练，同时还完成了**基于随机森林的数据降维，提取了数据的关键特征**。训练完成的模型可以输出一个 0~1 之间的值，代表了患者发生血肿扩张的概率，实验结果表明我们的模型预测效果非常好。

针对问题二，首先绘制全体患者水肿体积随时间进展的散点图，随后选取合适拟合方式，得到拟合曲线，并计算了前 100 个患者真实值和所拟合曲线之间存在的残差，得到的残差较大，考虑到全体患者间的个体差异较大，难以用一个拟合曲线代表全体，可以尝试对患者分类（亚型），那么同一亚型的个体差异就相对小很多，因此接下来根据水肿体积随时间进展状况来对患者进行聚类。由于不同患者的随访频率和次数不同，可将问题转化为**非等长（随访次数）非均匀采样（随访频率）的时间序列聚类问题**。本文使用了**基于 DTW 距离的 K-Means 算法**来进行聚类分析，本文认为将人群分为 5 个亚型是最合适的，对应了 5 种水肿体积随时间的进展模式。接下来对发展趋势采用**分时分箱量化编码**，得到前、中、后期的相对水肿体积评级，依此再使用**趋势性卡方检验**分析不同治疗方法是否对三个时期的水肿改善有帮助，进而得到不同治疗方法对水肿体积进展模式的影响。最后使用相关性分析来说明血肿体积、水肿体积及治疗方法三者之间的关系，利用 **Shapiro-Wilk 检验**分析其是否满足正态分布，对不满足正态分布的序列使用 **Mann-Whitney U 检验**分析相关性，对满足正态分布的序列使用 **Pearson 相关性**。

针对问题三，首先根据特征与目标变量间的 **Spearman 秩相关的相关性检验模型**分析相关性。并对原始数据进行降维，舍弃低相关性的特征，然后建立了**基于 XGBoost 的回归模型**，同时给出了几个评价指标，用于评估模型的预测性能。然后，对增加了随访数据的情况，本文针对**非等长非均匀采样的短序列**设计了 **LATEST 权均值、Integrated Slope 斜**

度值和 Hua 系数三个关键特征，以提取随访数据的时序特征，再将提取到的特征以及前一问的特征重新输入到 XGBoost 模型中去，实验多个指标结果表明，使用了随访时序特征的模型效果大有提高。最后，本文分析了出血性脑卒中患者的预后（90 天 mRS）和个人史、疾病史、治疗方法及影像特征等之间的关联关系，并为临床相关决策提出了一些建议。

关键词：出血性脑卒中 随机森林 DTW 聚类 XGBoost 特征提取 相关性分析

目录

一、问题简介	5
1.1 问题背景	5
1.2 问题重述	5
二、问题分析	7
2.1 问题一分析	7
2.1.1 问题一（a）分析	7
2.1.2 问题一（b）分析	7
2.2 问题二分析	7
2.2.1 问题二（a）分析	7
2.2.2 问题二（b）分析	7
2.2.3 问题二（c）分析	8
2.2.4 问题二（d）分析	8
2.3 问题三分析	8
2.3.1 问题三（a）分析	8
2.3.2 问题三（b）分析	8
2.3.3 问题三（c）分析	8
三、模型假设与符号说明	9
3.1 模型假设	9
3.2 符号说明	9
四、问题一模型建立与求解	10
4.1 问题一（a）模型建立与求解	10
4.2 问题一（b）模型的建立与求解	12
4.2.1 数据处理	12
4.2.2 随机森林特征提取模型	14
4.2.3 基于随机森林的血肿扩张预测模型	16
4.3 表 4-答案文件中问题一的计算结果	20
五、问题二模型建立与求解	24
5.1 问题二 a：模型的建立与求解	24
5.1.1 数据预处理	24
5.1.2 sub001-sub100 患者整体水肿体积回归模型	24
5.2 问题二 b：模型的建立与求解	26
5.2.1 动态时间规整（DTW）	26
5.2.2 DTW 面临的问题	27
5.2.3 数据标注化解决 DTW 不能规整强度拉伸问题	28
5.2.4 基于 DTW 距离的 K-Means 聚类分析	29
5.2.5 基于聚类结果的回归模型	31
5.3 问题二 c：模型的建立与求解	34
5.4 问题二 d：模型的建立与求解	37
5.4.1 血肿体积、水肿体积与治疗方法之间的关系	37
5.4.2 血肿体积与水肿体积之间的关系	39
5.5 表 4-答案文件中问题二的计算结果	40
六、问题三模型建立与求解	43
6.1 问题三 a：模型的建立与求解	43

6.1.1 数据分布观察与处理.....	43
6.1.2 相关性检验	44
6.1.3 建立 XGBoost 回归模型	45
6.2 问题三 b: 模型的建立与求解.....	48
6.2.1 难点分析及解决思路.....	48
6.2.1 建模方案	49
6.3 问题三 c: 模型的建立与求解	50
6.4 表 4-答案文件中问题三的计算结果.....	52
七、总结与评价	57
7.1 模型优点	57
7.2 模型缺点	57
7.3 模型的改进与推广	57
八、参考文献	58
九、附录.....	59

一、问题简介

1.1 问题背景

出血性脑卒中是脑卒中的一种，其是由非外伤性脑实质内血管破裂引起的脑出血^[1]。出血性脑卒中是一种严重的脑血管疾病，通常是由脑内血管破裂引起的，导致脑组织出血。这与缺血性脑卒中不同，后者是由于脑血管阻塞而引起的。该病发生时非常凶险，如果患者功能转归较差，其病死率与致残率就会变得非常高。故而出血性脑卒中发生后，需引起重视，及时给予患者有效治疗。出血性脑卒中患者多患有脑动脉硬化、高血压，主要症状为意识障碍、呕吐、头痛、感觉障碍、偏瘫等，又称之为脑淤血。发生出血性脑卒中后还会引发一系列复杂的生理病理反应，较严重还会留下永久性的神经功能障碍，给患者家庭带来无法承受的家庭负担。随着科学技术的发展，利用医学影像判断病理已成为医学诊断的普遍工具，利用医学影像特征以及患者个体差异特征预测出血性脑卒中发病时间已经成为一种可能。这种方法对出血性脑卒中的临床治疗具有重要的意义。

患者在出现出血性脑卒中后会出现许多并发症，有些并发症在出血性脑卒中治疗之后仍会对患者带来持续影响，血肿范围扩大便是其中一个危险的并发症。在出血发生后的短时间内，在脑组织受损、炎症等等因素的影响下，患者的血肿范围会持续扩大，颅内压会在此时跟着增加，其增加速度因人而异，但是一般都非常快，正是因为这个原因，患者的神经功能可能会进一步恶化，患者的生命此时也难以保障。在血肿影响患者健康的同时，其周围的水肿也会让脑组织受压，进而加重神经功能的损伤。发病病人因个体差异以及治疗方式的不同对水肿体积增大的预防也不同。因此，通过患者的血肿以及其周围水肿扩张的位置及其速度预测患者预后情况成为了一个重要的课题。

医学影像对于诊断、治疗和监测疾病都非常关键，它可以提供医生和医疗专业人员有关患者病情的关键信息。利用医学影像的成像技术来或者患者内部身体信息已经越来越普遍，这些信息可以帮助医生确定疾病的诊断、制定治疗计划和监测疾病的进展。近年来，随着人工智能的快速发展，越来越多的医学疾病诊断与治疗开始与人工智能模型结合在一起。由医学影像数据驱动的人工智能模型因为其效率高、准确率高、持续学习、可处理大数据等特点开始被逐步应用在无创动态监测出血性脑卒中后脑组织损伤和演变之中。构建的出血性脑卒中智能诊疗模型可以预测血肿及其周围水肿体积的大小等等从而帮助医生对患者病理信息有一个更加精准判断，进而为患者实现精准个性化的疗效评估和预后预测。因此，通过建立智能医学诊断模型，利用医学影像数据来预测疾病的发生给医生的诊疗过程带来了极大的便利。

1.2 问题重述

根据上述背景，该题目提供了 4 个附件数据以及相关的名词解释等等，欲解决的关键问题如下：

问题 1 (a)：判断患者 48 小时内是否发生血肿扩张事件

通过发病到首次影像检查时间间隔、各检查时间点对应的血肿体积来判断患者 sub001 至 sub100 发病后 48 小时内是否发生血肿扩张事件，同时记录血肿扩张事件的发生时间。

问题 1 (b)：构建血肿扩张概率预测模型

基于前 100 名患者的个人史、疾病史、发病及治疗相关特征和患者影像检查结果构建模型来预测所有患者发生血肿扩张的概率

问题 2 (a)：构建全体患者水肿体积随时间进展曲线

根据前 100 个患者的水肿体积和其重复检查点，构建一条适用于全体患者的水肿体积

随时间变化的进展曲线，同时计算前 100 个患者真实值和拟合值曲线之间的残差

问题 2 (b)：构建不同人群的水肿体积随时间进展曲线

通过患者的水肿体积随时间进展曲线分析患者水肿体积随时间进展模式的个体差异，构建适用于不同人群的水肿体积随时间的进展曲线，同时计算前 100 个患者真实值和拟合曲线之间的残差

问题 2 (c)：分析治疗方式对水肿体积进展模式的影响

通过 160 名患者的不同的治疗方式来分析其对水肿体积进展模式的影响

问题 2 (d)：分析血肿体积、水肿体积、及治疗方式三者间的关系

通过 160 名患者的血肿体积大小、水肿体积大小及不同的治疗方式来推断三者间存在的关系

问题 3 (a)：构建患者 90 天 mRS 评分预测模型

根据前 100 名患者的个人史、疾病史、发病相关的特征及首次影像中不同部位血肿与水肿的相关信息构建可以预测患者 90 天 mRS 评分的模型

问题 3 (b)：构建含随访影像的患者 90 天 mRS 评分预测模型

根据前 100 名患者的所有临床治疗手段以及首次影像和之后几次随访影像中不同部位血肿与水肿的相关信息构建可以预测含随访影像的患者 90 天 mRS 评分的模型

问题 3 (c)：分析患者预后方法、个体差异及影响特征关联关系

分析出血性脑卒中患者的预后和个人史、疾病史、治疗方法及影像特征等之间的关联关系，通过这个关系提出相应意见应用于临床相关决策

1.3 总体思路框架

总体思路框架如图 1-1。

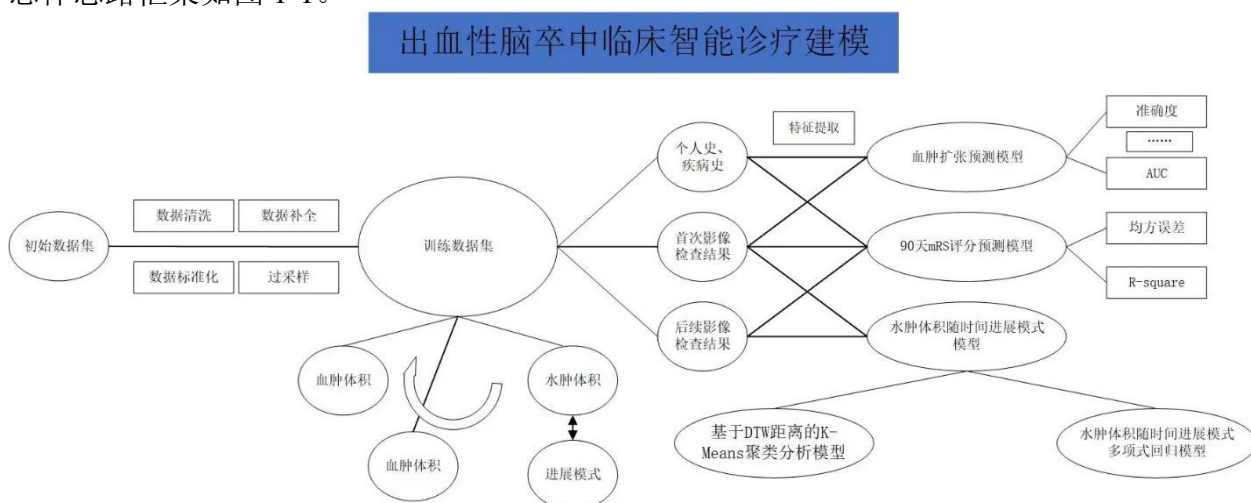


图 1-1 总体思路框架

二、问题分析

2.1 问题一分析

2.1.1 问题一（a）分析

针对问题 a，依据患者入院首次影像检查流水号、患者发生出血性脑卒中到首次影像检查时间的间隔、各时间点流水号及对应的血肿体积大小来判断患者 48 小时内是否发生血肿扩张事件。

首先我们要比对表二中的入院每一次影像检查流水号与附表一中的入院每一次检查流水号得到 sub0-sub100 患者的入院检查真实时间点及后续检查的真实时间点。然后判断后续检查真实时间点与首次检查真实时间点的时间间隔是否在规定时间内，之后再通过此时间点的血肿体积判断此时是否发生血肿扩张，结合发病到首次影像时间间隔以及首次影像至血肿扩张发生时间间隔即可得到血肿扩张时间。

2.1.2 问题一（b）分析

针对问题 b，根据表一中前 100 例患者的疾病相关信息、表二的影像检查结果以及表三首次影像检查结果建立预测模型预测所以患者发生血肿扩张的概率。

首先我们需要对表一、表二、表三的相关数据进行整理，整理之后进行数据清洗，把数据补全的患者挑选出来，以便之后对缺失的数据进行补充。对清洗完后的数据进行标准化与归一化。数据样本存在不平衡的现象，所以采用随机过采样的方法来平衡各样本类别数量。之后使用**随机森林特征提取模型**^[2]求得每一个特征的重要性分数，通过每一个特征的重要性分数选取相关性大的 **15 个特征**作为输入变量。使用这 15 个特征选取与数据缺失的患者信息最相近的患者，使用与数据缺失的患者信息最相近的患者的信息来补全数据缺失的患者信息。之后通过 15 个输入特征及 1 个目标变量来训练随机森林预测模型，通过网格搜索法来寻找最优超参数，最后将预测患者的相关信息输入模型得到最后的预测概率。

2.2 问题二分析

问题二要求对血肿周围水肿的发生及进展建模，并探索治疗干预和水肿进展的关联关系。

2.2.1 问题二（a）分析

对于 a 问题，要求绘制前 100 个患者的水肿体积随时间进展曲线并计算残差。首先对数据预处理，去除严重偏离样本中心的数据点。考虑到不同患者水肿体积个体差异明显，样本点（随访时间和次数）之间偏差较大，线性模型难以提取差异特征；高斯模型又容易出现过拟合问题，故选取**多项式模型**进行回归。最后通过比较真实数据与拟合曲线之间的数据，计算得到残差。

2.2.2 问题二（b）分析

对于 b 问题，要对前 100 个患者水肿体积随时间进展模式进行分类。进展模式大致可以分为增长、减弱、先增后减、先减后增等，可以通过聚类分析。由于不同患者的随访频率和次数不同，可将问题转化为**非等长（随访次数）非均匀采样（随访频率）的时间序列聚类问题**。可以采用**动态时间规整（Dynamic Time Warping, DTW）**分析非等长非均匀采样的时间序列的相似关系。与 a 问题类似，不同患者水肿体积个体差异明显，进行 DTW 分析钱还要进行数据的**标准化**。通过**手肘法**确定最优聚类数量，分别对聚类后的每一簇建立

拟合模型，分别计算得到残差。

2.2.3 问题二（c）分析

对于 c 问题，要分析治疗方法对水肿体积进展模式的影响。这里就要根据前一问得到的水肿体积进展模式，分别将各模式与各种治疗方法进行相关性分析，得到哪些治疗方法或者治疗组合方案有助于病人水肿体积趋向于哪种发展模式。

2.2.4 问题二（d）分析

对于 d 问题，要分析血肿体积、水肿体积及治疗方法三者之间的联系。不同于问题 c，这里分析**聚焦体积大小**而不是进展模式。由于不同患者血肿、水肿体积个体差异明显，不同患者样本数量和时间间隔差异较大，无法直接使用非等长非均匀采样时间序列分析相关性，故使用其**统计量**统计血肿体积、水肿体积的**特征**。考虑到不同治疗方法的使用前提和效果的不同，以是否使用某一治疗方式为前提，使用 **Mann-Whitney U 检验**统计使用该治疗方式和不使用该治疗方式的相关性系数，从而判断治疗方法与血肿、水肿体积的联系。

2.3 问题三分析

2.3.1 问题三（a）分析

针对问题三（a），根据表一中前 100 例患者的疾病相关信息、表二的影像检查结果以及表三首次影像检查结果建立预测模型预测所有患者的 90 天 mRS 评分。

之前第一题以及做过数据处理了，这里就不赘述。首先建立 **Spearman 秩相关相关性检验模型**对处理过后的数据进行特征提取，提取相关性高的特性进行训练。然后建立基于 **XGBoost** 的 mRS 评分预测模型，利用均方误差、均方误差标准差等评价指标求取最佳超参数。使用最佳超参数训练模型预测患者 90 天 mRS 评分。

2.3.2 问题三（b）分析

针对非等长非均匀采样的短序列设计了 **LATEST 权均值、Integrated Slope 斜度值和 Hua 系数**三个**关键特征**，以提取随访数据的时序特征，再将提取到的特征以及前一问的特征重新输入到 XGBoost 模型中去

2.3.3 问题三（c）分析

针对问题 3（c），要分析出患者的预后和个人史、疾病史、治疗方法及影像特征的关系，我们需要分别对患者个人差异及影像特征做 **Spearman 相关性检验**，通过每个因素的得分推断每个因素与患者的预后关系，之后通过每个因素与患者的预后关系来给临床相关决策提出意见。

三、模型假设与符号说明

3.1 模型假设

考虑到现实问题，为了简化问题拍出无关因素的干扰，根据患者发病情况以及治愈情况本文做出以下假设：

- 假设 1: 附件中提供的数据时真实可靠的；
- 假设 2: 患者在进行影像检查时其血肿体积不会变化
- 假设 3: 个别数据错误不会对结果产生重大影响
- 假设 4: 患者只在一家医院进行过检查和治疗
- 假设 5: 患者不存在其他会影响出血性脑卒中的疾病
- 假设 6: 只考虑发病的影响下血肿体积变化，不考虑人为意外因素

3.2 符号说明

表 3-1 符号说明

符号	符号说明
t_{later}	后续检查时间与初次检查时间间隔
$t_{primary}$	初次检查时间与发病时间间隔
V_{HD_later}	后续检查血肿体积大小
$V_{HD_primary}$	初始血肿体积大小
TP	被模型预测为正例的正例
FN	被模型预测为反例的反例
FP	被模型预测为正例的反例
TN	被模型预测为反例的正例

四、问题一模型建立与求解

4.1 问题一（a）模型建立与求解

针对问题一，经过阅读附件相关材料以及附表相关数据，想要知道患者是否在发病后 48 小时内发生血肿扩张以及其血肿扩张时间我们处理的步骤如下：

- （1） 比对表二中流水号与附表一中流水号得到真实时间点。
- （2） 判断后续检查真实时间点与首次检查真实时间点时间间隔是否在规定时间内
- （3） 通过此时间点的血肿体积判断此时是否发生血肿扩张
- （4） 结合发病到首次影像时间间隔以及首次影像至血肿扩张发生时间间隔得到血肿扩张时间

比对表二中的入院每一次影像检查流水号与附表一中的入院每一次检查流水号得到 sub0-sub100 患者的入院检查真实时间点及后续检查的真实时间点。

表 4-1 患者首次检查流水号与真实时间

患者编号	表 1 首次检查流水号	附表 1 首次检查流水号	真实时间点
sub001	20161212002136	20161212002136	2016/12/12 23:32
sub001	20160406002131	20160406002131	2016/4/6 21:21
sub001	20160413000006	20160413000006	2016/4/13 1:18
...	...	...	...
sub159	20200218000582	20200218000582	2020/2/18 15:29
sub160	20200821002584	20200821002584	2020/8/21 22:32

得到患者首次检查的真实时间点后我们要判断后续时间点与首次检查真实时间点的
时间间隔是否在 48 小时内：

$$\Delta t = t_{\text{later}} - t_{\text{primary}} \quad (4-1)$$

其中 $T_{\text{后续}}$ 代表后续检查时间， $T_{\text{初始}}$ 代表初次检查时间。

在得到 $T_{\text{后续}}$ 与 $T_{\text{初始}}$ 的时间间隔之后，判断其是否处于 48 小时之内。如果不在 48 小时之内，则判断此时没有发生血肿扩张事件，如果在 48 小时之内，则判断其水肿体积大小是否符合以下条件：

$$\begin{cases} V_{\text{HD_later}} - V_{\text{HD_primary}} \geq 6 \\ \frac{V_{\text{HD_later}} - V_{\text{HD_primary}}}{V_{\text{HD_primary}}} \geq 0.33 \end{cases} \quad (4-2)$$

其中 $V_{\text{HD_later}}$ 代表后续检查水肿体积大小， $V_{\text{HD_primary}}$ 代表首次检查水肿体积大小。

如果符合时间条件的后续检查水肿体积符合以上条件任意一个，则说明此患者 48 小时内发生了血肿扩张。之后通过以下公式得到患者血肿体积扩张事件发生时间：

$$t = t_1 + t_2 \quad (4-3)$$

其中 t_1 代表发病到首次影像时间间隔， t_2 代表首次影像至血肿扩张发生时间，结合 t_1 和 t_2 即可得到患者血肿扩张发生时间。

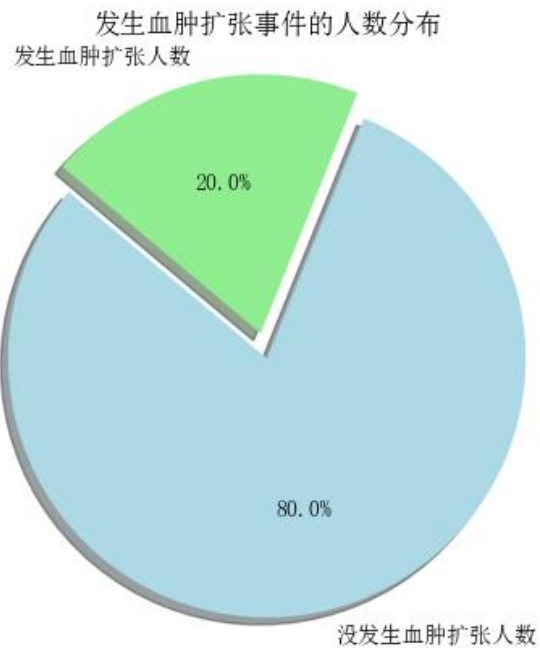


图 4-1 发生血肿扩张事件的人数分布

通过以上步骤得到的发生血肿体积总人数为 32 人，没发生血肿体积总人数为 128 人，见图 4-1，可以看出患者在发病后 48 小时内有一定几率会发生血肿扩张事件，概率为 20%。各血肿扩张人数占比见图 4-2，可以看到，患者基本都集中在发病后 5-20 小时发生血肿扩张。

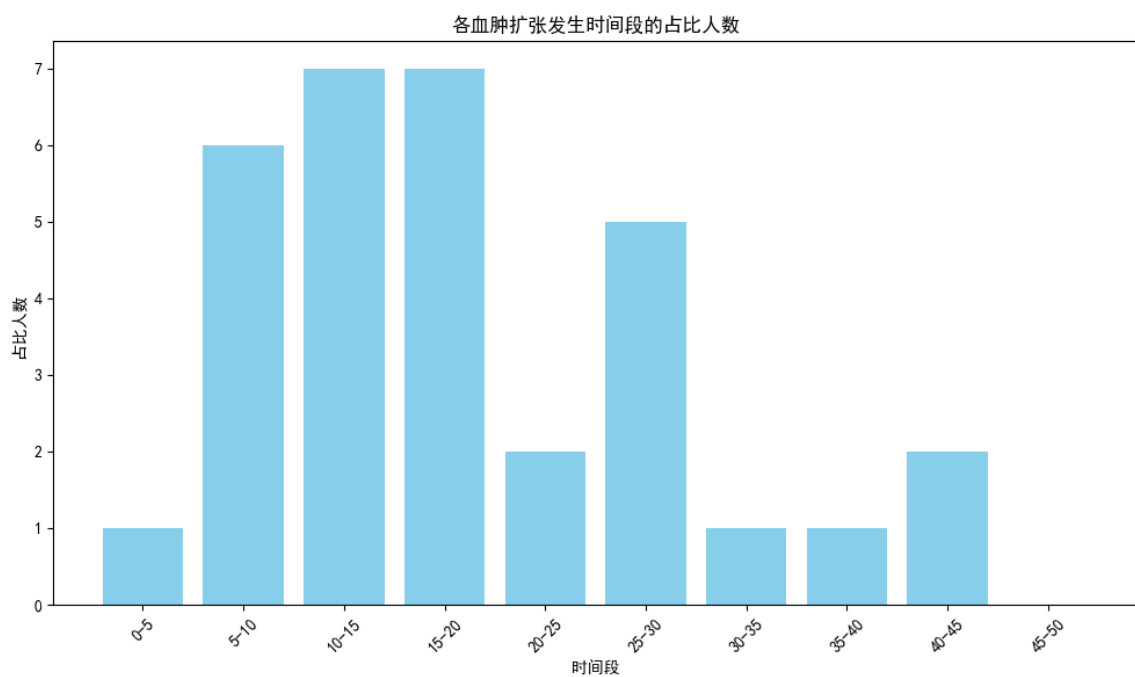


图 4-2 各血肿扩张时间人数占比

4.2 问题一（b）模型的建立与求解

针对由表一中前 100 例患者的疾病相关信息、表二的影像检查结果以及表三首次影像检查结果建立预测模型预测所有患者发生血肿扩张的概率的问题，我们从以下四个步骤进行求解：

- （1）数据处理：包括对数据的清洗、标准化以及过采样。
- （2）特征提取：利用随机森林特征提取模型对所有特征进行重要程度评分，将评分高的当做输入特性，之后进行数据补全。
- （3）建立模型：建立随机森林预测模型，将经过特性提取后的数据放入到模型进行训练，经过网格搜索法得到最佳的 $n_estimators$ 以及 max_depth 参数，即完成随机森林预测模型的建立。
- （4）模型评价：使用均方误差、均方误差标准差、R-squared、R-squared 标准差作为模型评价指标，对比 SVM、SVR、GBM，随机森林具有显著的优势。

4.2.1 数据处理

首先，观察所有患者的相关信息，对比每个患者表一、表二、附表一的首次入院流水号，查看相关数据是否有数据缺失或者数据错误。相关统计如图 4-3。之后对所有患者的所有特征进行宏观分析统计，可以看出，患者的相关特征有浮点型变量（HM_volume、HM_ACA_R_Ratio 等等），有二值变量（高血压病史、卒中病史等等），有整形变量（年龄、血压等等），有文字变量（性别），相关统计见图 4-4。

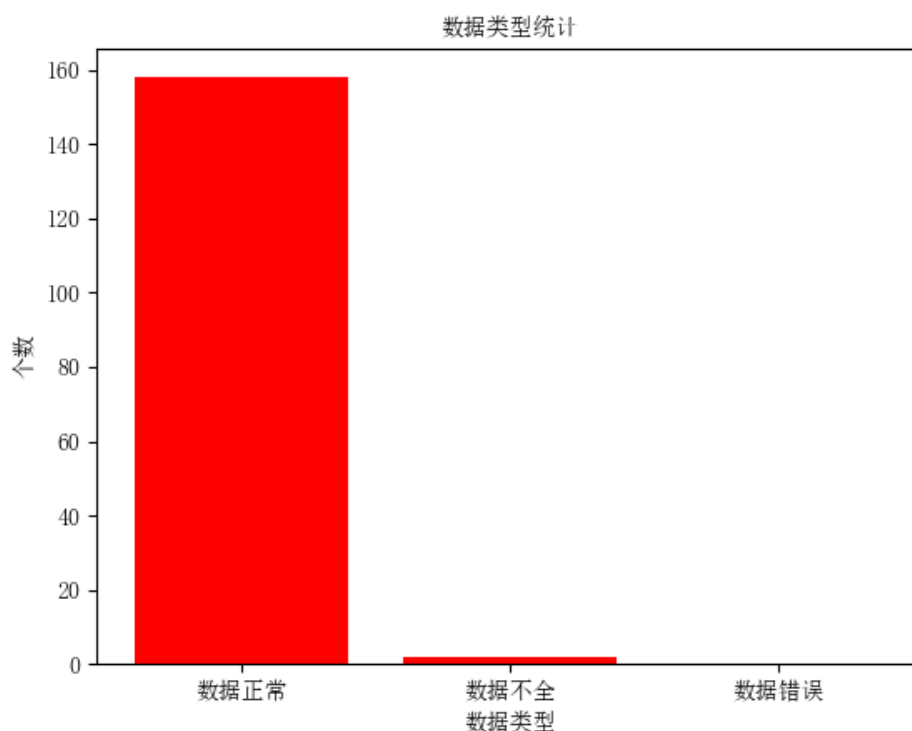


图 4-3 数据类型统计

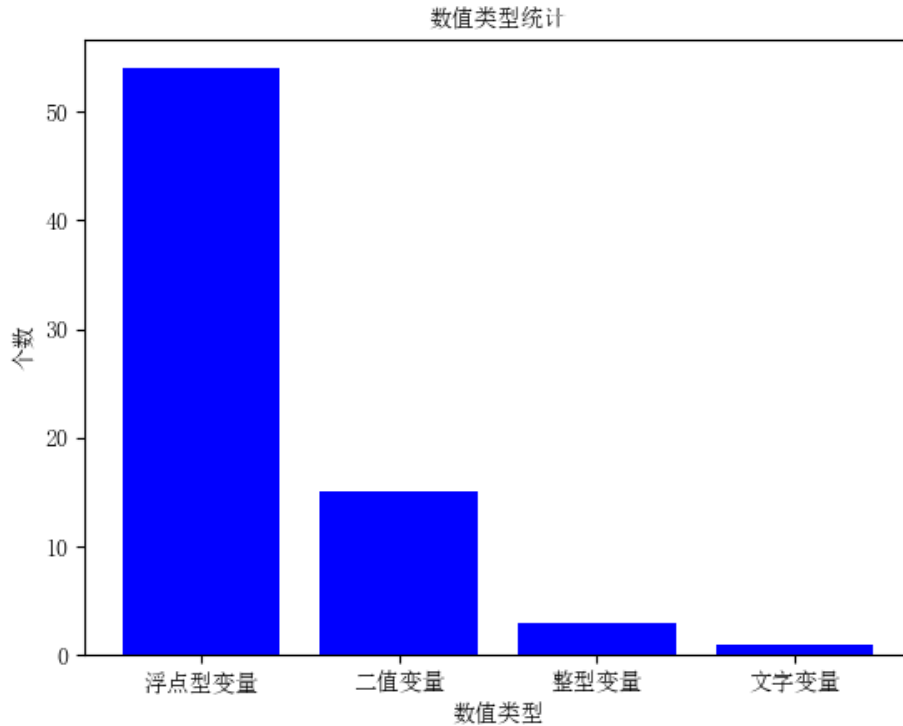


图 4-4 数值类型统计

对于数据补全的患者（sub128, sub129），由于数据样本不大，数据样本比较珍贵，故不删除数据不全的数据，选择数据补全的处理方法。数据补全在相关性分析之后即可将与缺失数据的患者相关信息最接近的患者对其进行数据补全。对于数值类型不同的处理，我们选择的方法是对数据进行标准化、归一化。采用的是最大最小值归一化方式，其计算方式如下：

$$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (4-4)$$

其中 $X_{normalized}$ 代表归一化后的值， X 代表所选患者特征值大小， X_{min} 代表所有患者特征值最小值， X_{max} 代表所有患者特征值最大值。

对于浮点型变量和整型变量，由于其大小差距很大，为避免其大小给相关性带来的误差，故对其进行归一化，二值型变量在此条件下则不同进行处理，对于文字变量（性别），为尽量降低多重共线性，我们对其进行编码处理，男性编码为（1,0）女生编码为（0,1）。同时由于样本不均衡，出血性脑卒中占比如图 4-5，我们使用重复采样随机选择少数类别的样本来增加其数量，经过过采样处理之后，样本分布比较均衡，如图 4-6 所示。在经过处理之后，我们得到的是一个 74 维的特征输入向量。

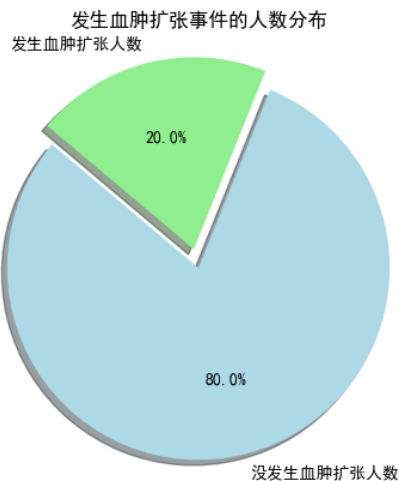


图 4-5 过采样前发生血肿扩张人数分布

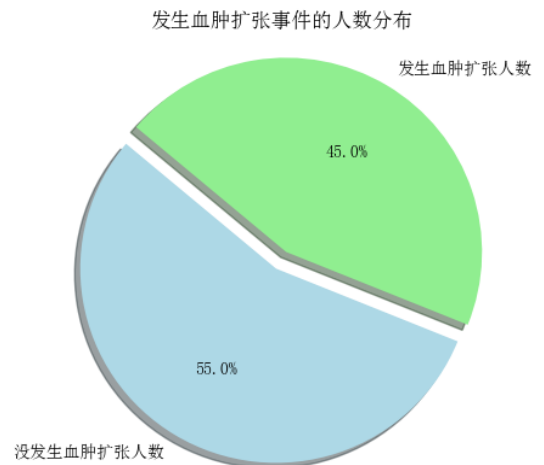


图 4-6 过采样前发生血肿扩张人数分布

4.2.2 随机森林特征提取模型

数据特征的维度为 74，处于一个比较大的水平，在查阅相关文献以及数据分析之后，我们得出结论，数据的 74 个特征之中存在很多冗余信息，同时存在多重共线性，可能会增加计算复杂度，降低模型性能。对此，我们打算对数据做相关性分析，进行特征提取。由于数据特征存在维度高、噪声大等特点，所以我们选择使用随机森林相关性分析对数据进行重要性分析以及特征提取^[3]。我们使用 Gini 重要性来衡量相关性。Gini 系数度量了一个节点上随机抽取的样本被错误分类的概率。对于每个节点，算法都会计算它的 Gini 系数，然后将这些值加权平均到每个特征上。Gini 重要性得分越高的特征被认为越重要。其计算公式为：

$$Gini(D) = 1 - \sum_{i=1}^C (p_i)^2 \quad (4-5)$$

其中 p_i 是指属于类别 i 的概率。

使用随机森林相关性分析计算每个特征重要性得分，得分从大到小排列。综合模型泛化性、模型复杂度等因素，选取其中重要性分数最大的 15 个特征作为模型的输入变量。15 个特征及其得分（归一化之后）如图 4-7 所示。

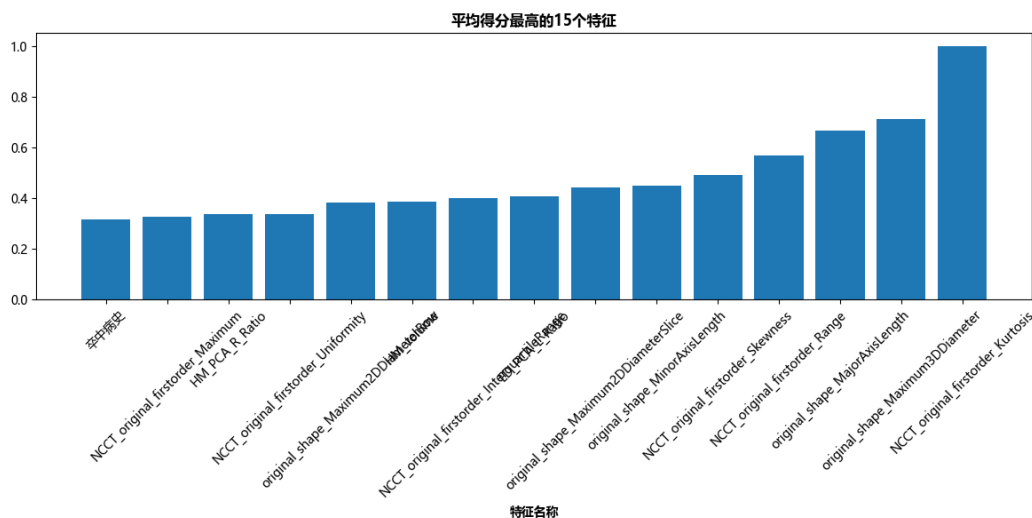


图 4-7 平均得分最高的 15 个特征

得到 15 个最相关特征之后我们需要对 sub128 和 sub129 患者的数据进行补全，从 15 个平均得分最高的 15 个特征中选取 sub128 和 sub129 患者具有的卒中病史、HM_PCA_R_Ratio、HM_volume 三个特征。使用卒中病史、HM_PCA_R_Ratio、HM_volume 三个特征分别对 sub128 患者和 sub129 患者和其他患者进行特征比对，距离比对公式为：

$$e = \sum_{\substack{i=1,2,3 \\ 0 \leq n < 160 (n \neq j)}} (T_{ij} - T_{in})^2 \quad (4-6)$$

其中 e 代表误差大小， i 代表特征编号， j 代表患者编号， T_{ij} 代表数据缺失患者特征值， T_{in} 代表其他病人特征值。

经过计算之后，我们得到了每名患者与 sub128 患者以及 sub129 患者的距离，距离越小就说明这名患者与数据缺失患者的信息相似度就越高。得到的距离误差如图 4-8 和图 4-9 所示，从图中我们可以看出 sub128 患者与 2 号患者的信息最相近，所以将 2 号患者的相关信息对 128 号患者缺失信息进行补全，sub129 患者与 97 号患者的信息最相近，所以将 97 号患者的相关信息对 129 号患者缺失信息进行补全。

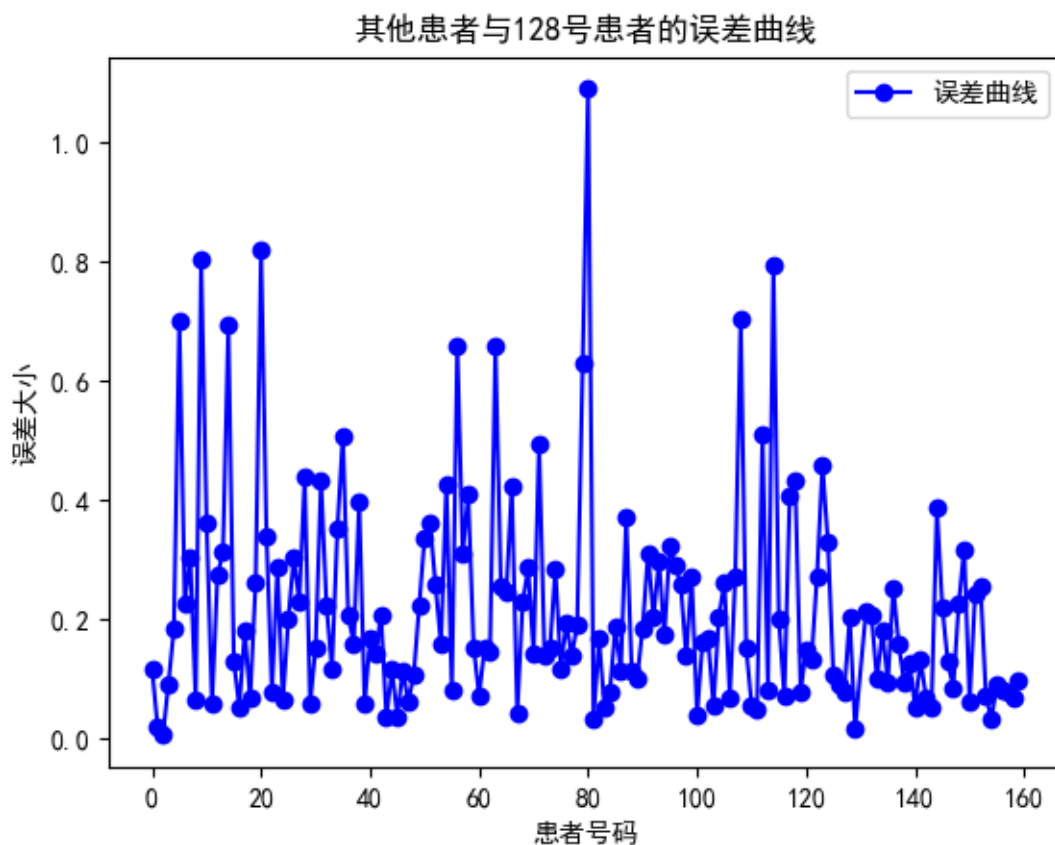


图 4-8 其他患者与 128 号患者的误差曲线

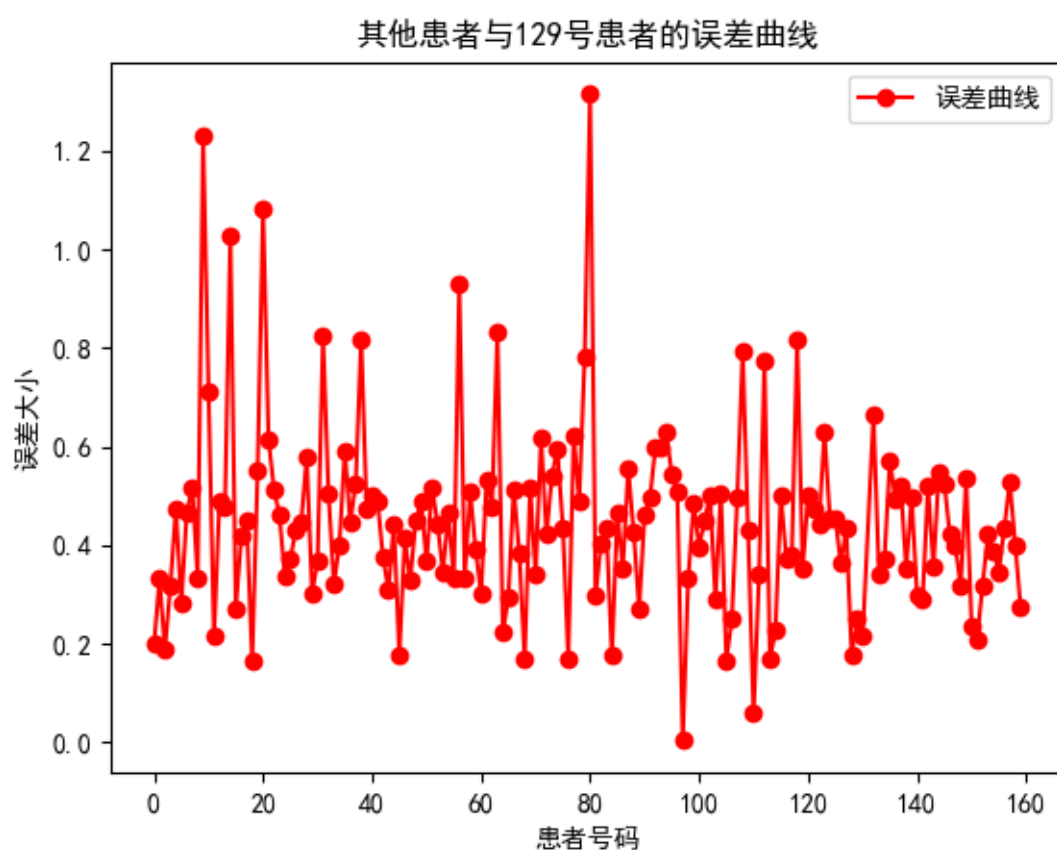


图 4-9 其他患者与 129 号患者的误差曲线

4.2.3 基于随机森林的血肿扩张预测模型

随机森林是多棵决策树的集成，决策树结构如下：

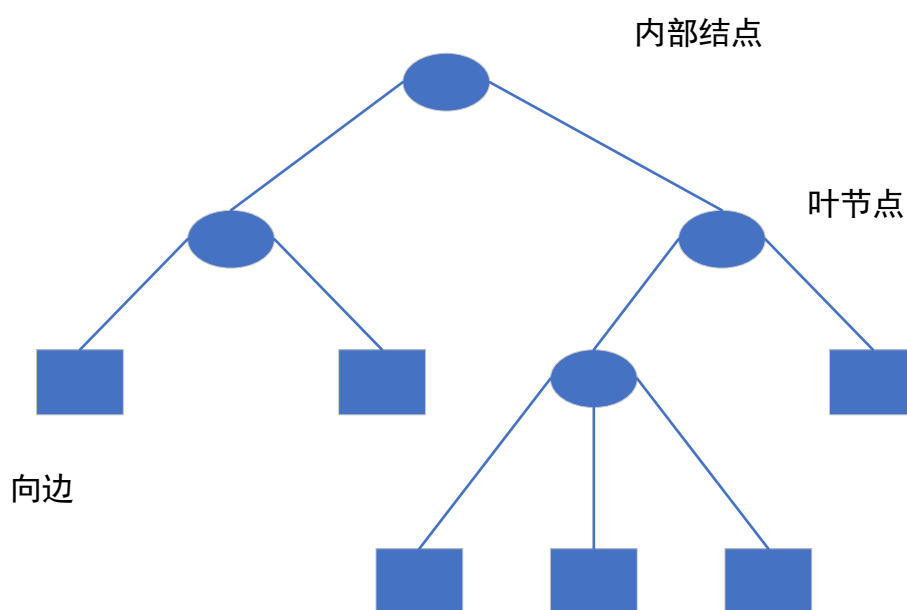


图 4-10 决策树结构

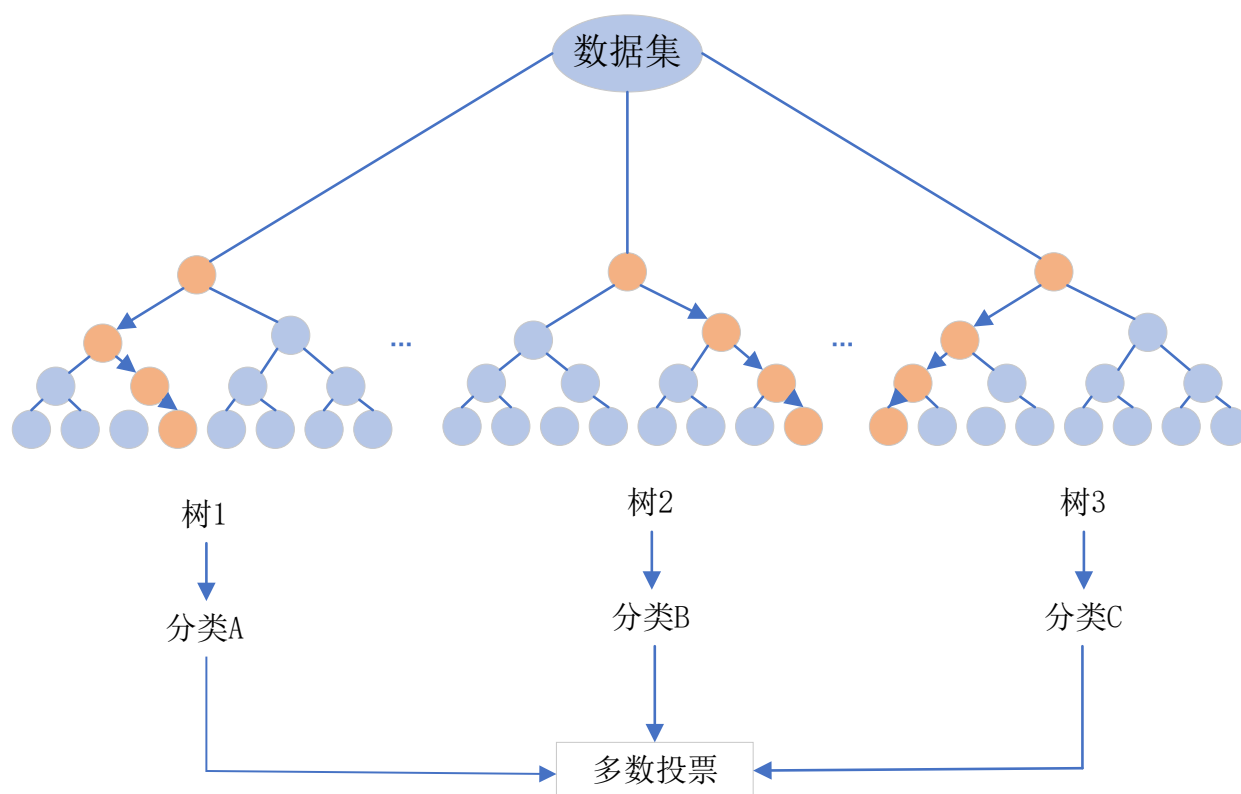


图 4-11 随机森林结构

众多决策树构成了随机森林，每棵决策树都会有一个投票结果，最终投票结果最多的类别，就是最终的模型预测结果。建立随机森林预测模型，首先我们使用装袋法在出血性脑卒中患者数据行列上进行随机抽样，之后建造多个决策树弱分类器，每一个决策树分类器用随机方法构成。学习集是从原训练集中通过有放回抽样得到的自助样本。参与构建该决策树的变量也是随机抽出，参与变量数通常大大小于可用变量数(即第一点上的在列上而随机抽样) 单个决策树在产生学习集和确定参与变量后，使用 CART 算法计算，不剪枝 最后分类结果取决于各个决策树分类器简单多数选举。

对出血性脑卒中患者数据建立随机森林预测模型进行训练，训练步骤如下：

- (1) 读取数据：读取出血性脑卒中患者的个人信息特征数据以及是否发生血肿扩张与否。
- (2) 拆分数数据集：训练样本为 sub001 至 sub100 患者的信息，划分训练集和测试集。
- (3) 训练模型：使用训练集来训练随机森林模型，每个子集上训练一颗决策树。
- (4) 超参数调优：通过交叉验证选择超参数 `estimators` 和 `depth` 的最佳值。
- (5) 评估模型：使用测试集来评估模型的性能，通过基本预测模型评价指标来评价模型的好坏以获得最优超参数。
- (6) 模型应用：将训练好的模型用于实际数据的预测，输入 sub001 至 sub160 的特征信息，使用模型来预测患者是否发生血肿扩张。

首先我们先对处理完后的数据进行读取，其中可用样本为 sub001 至 sub100 患者的信息，我们把数据集按 7:3 的比例划分训练集和测试集，之后使用训练集来训练随机森林模型，每个子集上训练一颗决策树。对于 `n_estimators` 参数以及 `max_depth` 两个参数我们采取网格搜索法进行调优，`n_estimators` 从 10 到 100 按 5 的步长进行搜索，`max_depth` 从 1 到 10 按 1 的步长进行搜索，我们采用准确率、精确率、召回率、F1 分数、ROC 曲线、AUC 分数来评价模型。

(1) 准确度(Accuracy)

准确度表示模型正确预测的样本数量占总样本数量的比例。其公式如下：

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (4-7)$$

其中各参数概念如下表所示：

表 4-2 TP、FN、FP、TN 相关概念

	预测正例	预测反例
真实正例	TP	FN
真实反例	FP	TN

(2) 精确度 (Precision) :

精确度表示模型正确预测为正例的样本数量占所有预测为正例的样本数量的比例。其公式如下：

$$Precision = \frac{TP}{TP + FP} \quad (4-8)$$

(3) 召回率 (Recall) :

召回率表示模型正确预测为正例的样本数量占所有实际正例的样本数量的比例。其公式如下：

$$Recall = \frac{TP}{(TP + FN)} \quad (4-9)$$

(5) F1 分数

F1 分数综合考虑了精确度和召回率，是一个综合性能指标。其公式如下：

$$F_1_score = \frac{2 * (\text{精确度} * \text{召回率})}{(\text{精确度} + \text{召回率})} \quad (4-10)$$

(6) AUC

AUC 是 ROC 曲线下的面积，ROC 曲线描述了模型在不同阈值下的召回率和假正率之间的权衡。AUC 表示模型能够将正例排在负例前面的概率，范围在 0.5 到 1 之间。AUC 等于 0.5 表示模型性能等于随机猜测，等于 1 表示完美性能。

(7) ROC 曲线

ROC 曲线是以假正率 (FPR, False Positive Rate) 为横坐标，召回率为纵坐标的曲线。ROC 曲线描述了在不同阈值下，模型的性能如何变化。

随机森林预测模型训练过程中，模型的评价如下图所示：

准确度随n_estimators与max_depth的变化趋势

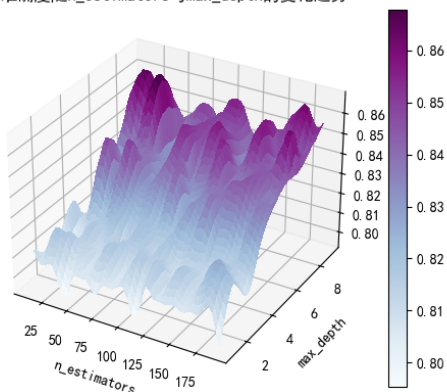


图 4-12 准确度变化趋势

精确度随n_estimators与max_depth的变化趋势

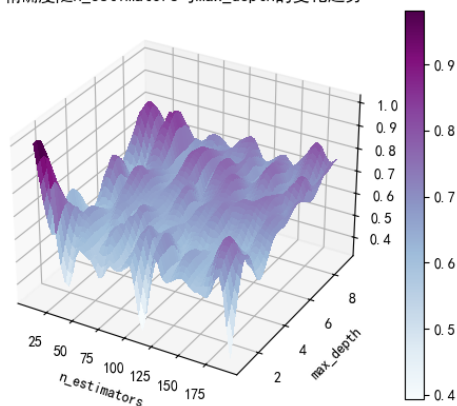


图 4-13 精确度变化趋势

召回率随n_estimators与max_depth的变化趋势

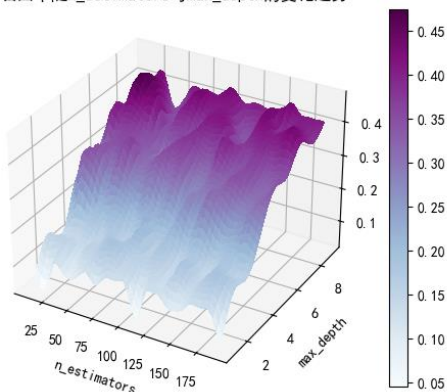


图 4-14 召回率变化趋势

F1分数随n_estimators与max_depth的变化趋势

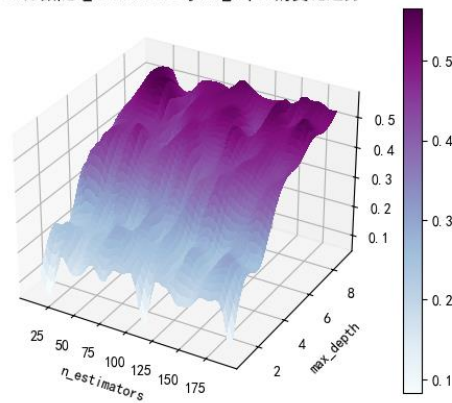


图 4-15 F1 分数变化趋势

AUC分数随n_estimators与max_depth的变化趋势

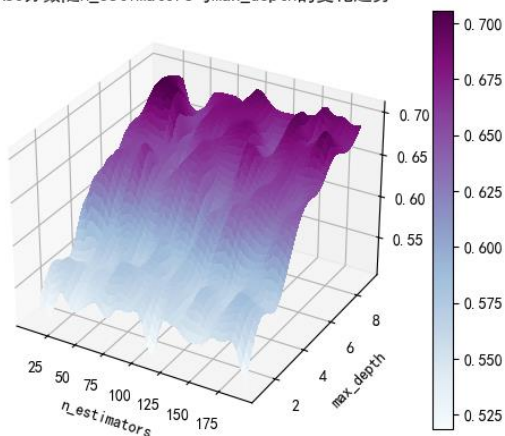


图 4-16 AUC 变化趋势

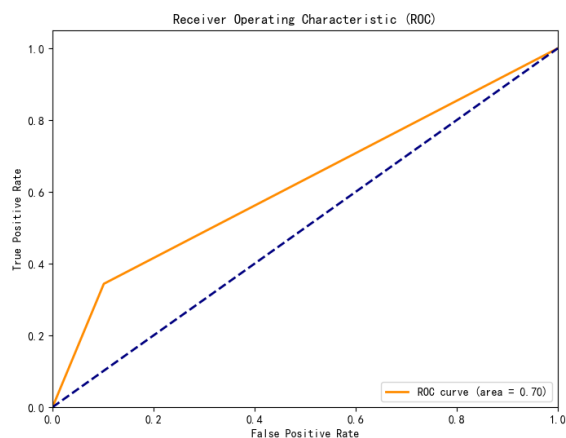


图 4-17 ROC 曲线

训练完之后得到的最佳参数 $n_estimators=30$, $max_depth=8$, 得到的准确度为 0.8625, 精确度为 0.7778, 召回率为 0.4375, F1 分数为 0.56, AUC 为 0.7031。在保证准确度的情况下, 其他指标达到预期, 说明模型具有一定的预测能力。利用此模型输入患者 sub001 至 sub160 的特征得到的结果输入到附表 1 中。

4.3 表 4-答案文件中问题一的计算结果

		问题 1: 血肿扩张		
	首次影像检查流水号	是否发生血肿扩张	血肿扩张时间	血肿扩张预测概率
		1 是, 0 否	单位: 小时	
sub001	20161212002136	0		0.1220109
sub002	20160406002131	0		0.30747995
sub003	20160413000006	1	9.5225	0.4535491
sub004	20161215001667	0		0.04978754
sub005	20161222000978	1	26.4675	0.52868998
sub006	20161110001074	0		0.10381119
sub007	20161208000139	0		0.0991669
sub008	20161219000091	0		0.1695109
sub009	20161031001987	1	40.056111	0.59193501
sub010	20161012002008	0		0.15266935
sub011	20160209000219	0		0.13307093
sub012	20161031001142	0		0.08060181
sub013	20161124000397	0		0.12742454
sub014	20160513001799	0		0.08825217
sub015	20161013001234	0		0.26643406
sub016	20161130000004	0		0.1695109
sub017	20160510002436	1	14.870278	0.40747995
sub018	20160602001707	0		0.16485909
sub019	20160117000135	0		0.07312254
sub020	20160723000013	0		0.40571429
sub021	20160317001244	0		0.21693406
sub022	20160803001239	0		0.0648158
sub023	20160321000142	0		0.13757398
sub024	20170802000637	0		0.07134901
sub025	20171226002293	0		0.10726848
sub026	20171008000512	0		0.08060181
sub027	20170206000071	0		0.05886622
sub028	20171013002097	0		0.1440625
sub029	20170607000010	0		0.03192951
sub030	20171025000480	0		0.22619143
sub031	20170307002130	0		0.14454505
sub032	20171009000137	0		0.38591941
sub033	20170115000362	1	30.813056	0.96666667
sub034	20170119000729	0		0.0715109
sub035	20171014001244	0		0.02549647

sub036	20170204001714	1	39.501111	0.21971026
sub037	20170426000005	0		0.0935625
sub038	20170518002194	1	15.809167	0.1645805
sub039	20170425002487	1	29.176111	0.7252557
sub040	20170902000876	0		0.10248161
sub041	20171002000282	0		0.09311428
sub042	20170420000636	0		0.14600133
sub043	20170325000428	0		0.14893884
sub044	20170528000084	0		0.10265341
sub045	20170324001892	0		0.0787347
sub046	20170511000016	0		0.24332592
sub047	20171019001652	0		0.16094617
sub048	20170402000556	1	12.860833	0.27376335
sub049	20171005000770	0		0.0986321
sub050	20171105000372	0		0.0826145
sub051	20170422000935	0		0.11862634
sub052	20170608001310	0		0.05192951
sub053	20170511001392	0		0.0595109
sub054	20170612002216	1	16.225	0.54193501
sub055	20170316001977	0		0.16635361
sub056	20170120000152	0		0.1996879
sub057	20170825001844	1	14.615833	0.85
sub058	20170125000984	0		0.09999258
sub059	20170912002314	0		0.16628571
sub060	20180109000613	1	23.726111	0.15363515
sub061	20180226000725	1	6.5416667	0.5577529
sub062	20181221002264	0		0.04992043
sub063	20181020001229	0		0.0966321
sub064	20180801000501	0		0.00860175
sub065	20180131001727	0		0.03392951
sub066	20181208000909	0		0.07134901
sub067	20181207001317	0		0.09173868
sub068	20180412001426	0		0.0715109
sub069	20180619001505	0		0.15630975
sub070	20180427000292	1	9.6533333	0.28056676
sub071	20181103001264	0		0.29339071
sub072	20181007000826	0		0.17131363
sub073	20180911001645	0		0.10772301
sub074	20180719000020	0		0.11028542
sub075	20180428001767	0		0.10313515
sub076	20180619002401	1	15.993611	0.30079262
sub077	20180503002304	1	14.119444	0.43795614
sub078	20180929000040	0		0.39339071

sub079	20180929000037	1	27.847222	0.27605468
sub080	20180130001917	1	20.573333	0.36088147
sub081	20180120000249	1	27.421667	0.5527041
sub082	20180221000793	0		0.0715109
sub083	20181004000706	0		0.1069304
sub084	20180716000006	0		0.22425679
sub085	20181127002511	0		0.0935625
sub086	20180108000002	0		0.08567141
sub087	20180216000198	0		0.31728305
sub088	20180521000314	0		0.05598111
sub089	20180314002318	0		0.16056291
sub090	20180910002366	0		0.14645588
sub091	20181019001130	0		0.08567141
sub092	20181116001089	1	11.924722	0.4481039
sub093	20181214000208	0		0.09369539
sub094	20180412001795	0		0.14251069
sub095	20180316001329	1	7.4316667	0.52183911
sub096	20180802001789	0		0.08091781
sub097	20181010000767	0		0.14044836
sub098	20180612002507	1	42.756667	0.55739422
sub099	20180620002296	1	17.672778	0.53795614
sub100	20180314000010	0		0.05697
sub101	20180311000432	0		0.30248161
sub102	20180708000024	0		0.18339868
sub103	20181015001677	0		0.18066667
sub104	20190105000694	0		0.1695109
sub105	20190108002459	0		0.15829018
sub106	20190519000853	0		0.24284862
sub107	20190526000209	0		0.0915625
sub108	20190701002502	0		0.10326804
sub109	20190716000013	0		0.31890756
sub110	20190717001385	0		0.07324717
sub111	20190727000556	0		0.17736358
sub112	20190803000014	0		0.13757398
sub113	20190901000442	0		0.33915152
sub114	20190903000373	0		0.17719178
sub115	20190904002299	0		0.2442164
sub116	20190906001283	0		0.08325377
sub117	20190917002094	0		0.07567141
sub118	20190923002580	0		0.0595109
sub119	20191003000352	0		0.12450614
sub120	20191004001025	0		0.18098508
sub121	20191027000468	0		0.3071085

sub122	20191028002738	0		0.13961465
sub123	20191024001280	0		0.0201648
sub124	20191030001526	0		0.27452221
sub125	20191111002510	0		0.16141371
sub126	20191126002590	0		0.0735236
sub127	20191127002176	0		0.38081328
sub128	20191208000592	0		0.0826145
sub129	20191224000008	0		0.0795625
sub130	20191228000237	0		0.15630975
sub131	20160413000006	1	15.393611	0.47173092
sub132	20161215001667	0		0.55739422
sub133	20200112000228	0		0.1912619
sub134	20200101000392	0		0.1440625
sub135	20201130003288	0		0.36671889
sub136	20201203002778	1	19.375	0.17425679
sub137	20201217002368	0		0.1826145
sub138	20200403000012	0		0.26919927
sub139	20200814000015	1	13.839444	0.28043002
sub140	20200412000331	0		0.15822301
sub141	20200711001264	0		0.08060181
sub142	20200411000014	1	10.289167	0.0895109
sub143	20200214000572	0		0.24688739
sub144	20200124000041	0		0.20203891
sub145	20200613001086	0		0.04284862
sub146	20200531000253	0		0.15829018
sub147	20201220000155	1	8.8630556	0.0989951
sub148	20200609001016	0		0.0826145
sub149	20200409001101	0		0.26141371
sub150	20200118000372	0		0.06302042
sub151	20201023001238	1	8.2302778	0.43795614
sub152	20201109000009	0		0.2069304
sub153	20201212001420	0		0.25178219
sub154	20201129000299	0		0.17133911
sub155	20200301000025	0		0.24796661
sub156	20200306000927	0		0.16978754
sub157	20201009003102	1	1.3666667	0.24238109
sub158	20200410001952	0		0.0807347
sub159	20200218000582	1	26.533333	0.14489605
sub160	20200821002584	1	17.483333	0.10265341

五、问题二模型建立与求解

5.1 问题二 a：模型的建立与求解

根据“表 2-患者影像信息水肿及水肿的体积及位置”前 100 个患者(sub001 至 sub100)的水肿体积(ED_volume)和重复检查时间点,构建一条全体患者水肿体积随时间进展曲线(x 轴:发病至影像检查时间,y 轴:水肿体积, $y=f(x)$),计算前 100 个患者(sub001 至 sub100)真实值和所拟合曲线之间存在的残差。

5.1.1 数据预处理

a 问题涉及到的数据包括病人的 ID(sub001-sub100)、发病到首次影像检查时间间隔、首次影像检查流水号、后续随访流水号、每次影像检查得到的水肿体积(ED_volume)等。

首先处理 x 轴时间: x 轴指发病至影像检查的时间,首次影像检查时间由“表 1-患者列表及临床信息”直接得到,由“附表 1-检索表格-流水号 vs 时间”用后续随访流水号匹配得到后续随访的时间点,计算后续随访的时间点与入院首次检查时间点的时间差,再加上首次影像检查时间即得到发病至随访的时间,时间单位均是小时(h)。对于 y 轴水肿体积,从“表 2-患者影像信息水肿及水肿的体积及位置”中直接读取,注意水肿体积 y 要与发病至影像检查的时间 x 相匹配。

从 sub001-sub100 病人中共得到 435 组(x,y)。

5.1.2 sub001-sub100 患者整体水肿体积回归模型

得到的 435 组(x,y)如图 5-1 所示。

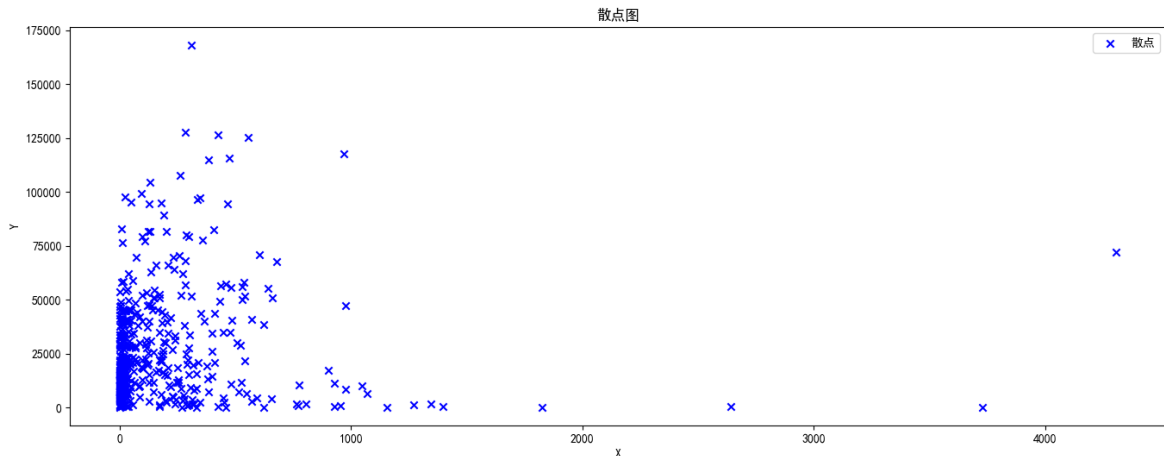


图 5-1 sub001-sub100 患者水肿体积散点图

分析患者水肿体积散点分布情况,发现数据集中在散点图的左下角,且在散点图的右方和上方存在严重偏离样本中心的异常数据。删除 x 坐标超过最大值 50%以上、y 坐标超过最大值 70%以上的数据点。

选取多项式模型进行回归,结果如图 5-2、5-3、5-4。

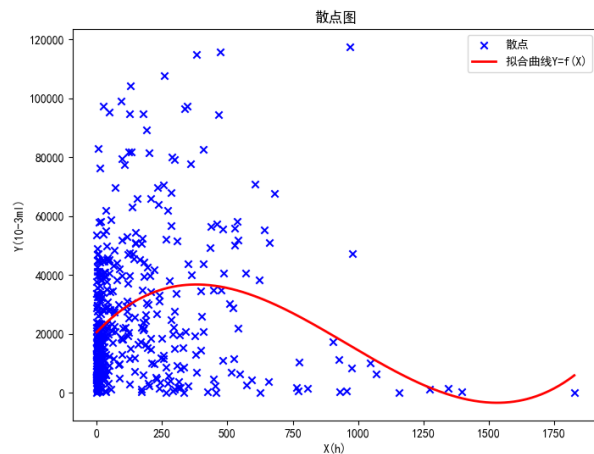


图 5-2 三次项拟合

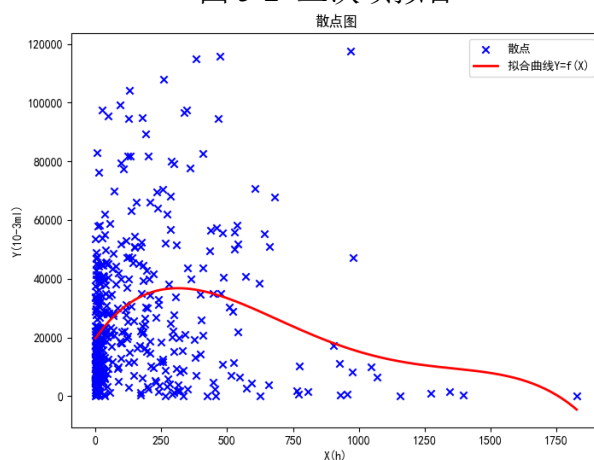


图 5-3 四次项拟合

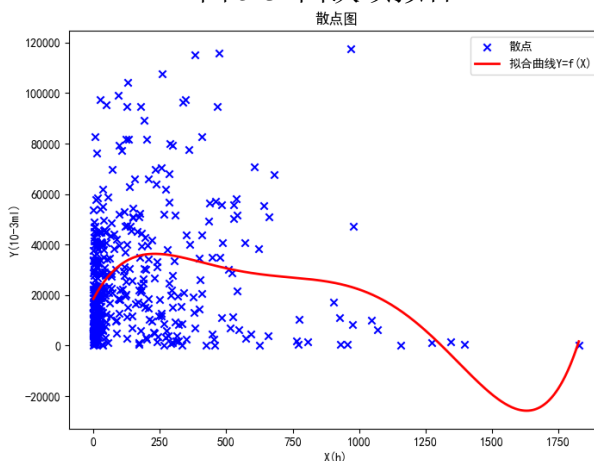


图 5-4 五次项拟合

三次项的拟合函数为：

$$\hat{y}_3 = -5.2807 \times 10^{-5} x^3 - 0.1517 x^2 + 92.6796 x + 20568 \quad (5-1)$$

四次项的拟合函数为：

$$\hat{y}_4 = -5.2071 \times 10^{-8} x^4 + 2.1335 \times 10^{-4} x^3 - 0.2927 x^2 + 126.9227 x + 19687 \quad (5-2)$$

五次项的拟合函数为：

$$\hat{y}_5 = 1.7760 \times 10^{-10} x^5 - 7.4710 \times 10^{-7} x^4 + 1.1145 \times 10^{-3} x^3 - 0.7383 x^2 + 197.1857 x + 18454 \quad (5-3)$$

其中，x 轴单位为小时（h），y 轴单位为体积（ 10^{-3}ml ）

使用均方误差 (MSE)和均方根误差 (RMSE)判断拟合效果，其计算公式如下：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5-4)$$

$$RMSE = \sqrt{MSE} \quad (5-5)$$

其中，n 是观测数据点的数量， y_i 是实际观测值， $\hat{y}_i$ 是模型预测的值。MSE 越小，表示模型对数据的拟合越好。分别计算三次项、四次项、五次项拟合函数的均方误差 (MSE) 和均方根误差 (RMSE)，计算结果表所示：

表 5-1 不同治疗方式对水肿体积的影响

多项式拟合	RMSE
三次项	28480
四次项	26547
五次项	32108

从表格中均方误差 (MSE)和均方根误差 (RMSE)可以看出，四次项的拟合效果最好。故全体患者水肿体积随时间进展曲线为：

$$y = f(x) = -5.2071 \times 10^{-8} x^4 + 2.1335 \times 10^{-4} x^3 - 0.2927 x^2 + 126.9227 x + 19687 \quad (5-6)$$

（x：发病至影像检查时间，y：水肿体积， $y=f(x)$ ）

计算前 100 个患者（sub001 至 sub100）真实值和所拟合曲线之间存在的残差，残差的计算公式如下：

$$e = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - \hat{y}_i) \quad (5-7)$$

将计算后前 100 个患者的残差填入“表 4-答案文件”。

5.2 问题二 b：模型的建立与求解

探索患者水肿体积随时间进展模式的个体差异，构建不同人群（分亚组：3-5 个）的水肿体积随时间进展曲线，并计算前 100 个患者（sub001 至 sub100）真实值和曲线间的残差。由于不同患者的随访频率和次数不同，可将问题转化为非等长（随访次数）非均匀采样（随访频率）的时间序列聚类问题。可以采用动态时间规整（Dynamic Time Warping, DTW）分析非等长非均匀采样的时间序列的相似关系。

b 问所用到的数据与 a 问不同。a 问只需要用到 435 组(x,y)，不需要考虑这 435 组(x,y)属于哪个患者 ID（sub），也不需要考虑同一患者不同检查的水肿体积的时间连续关系。b 问用到的 435 组(x,y)数据有各自的患者 ID 标签，同一患者 ID 的(x,y)要按照时间排列，考虑时间顺序。

5.2.1 动态时间规整（DTW）

DTW 是一种动态规划算法。假设有两个时间序列 C 和 Q，长度分别为 m 和 n。

$$C = c_1, c_2, \dots, c_m \quad (5-8)$$

$$Q = q_1, q_2, \dots, q_n \quad (5-9)$$

定义一个距离度量函数 $d(i, j)$ ，表示序列 Q 中第 i 个元素和序列 R 中第 j 个元素之间的距离。通常使用欧几里德距离： $d(i, j) = \sqrt{(c_i - q_j)^2}$ ；计算距离矩阵 D ，其中 $D[i][j]$ 表示 $d(i, j)$ ，即序列 Q 中第 i 个元素和序列 R 中第 j 个元素之间的距离；创建一个动态规划矩阵（DTW），其大小与距离矩阵 D 相同。初始化 DTW 的第一行和第一列，通常设置为足够大的值；动态规划计算路径，从 DTW 矩阵的左上角（DTW[0][0]）开始，通过以下规则计算 DTW 矩阵的每个元素（DTW[i][j]）：

$$DTW[i][j] = d(i, j) + \min(DTW[i-1][j], DTW[i][j-1], DTW[i-1][j-1]) \quad (5-10)$$

其中， $\min$ 表示取三个值中的最小值。这个过程将计算出一个最小路径，其路径上的元素之和表示最小的时间规整距离；最终，DTW 距离就是最小路径上所有元素之和。最终，可以从 DTW 矩阵的右下角开始，根据最小值的路径回溯到左上角。这就是最佳的时间对齐方式。其动态时间规整效果如图 5-5 所示。

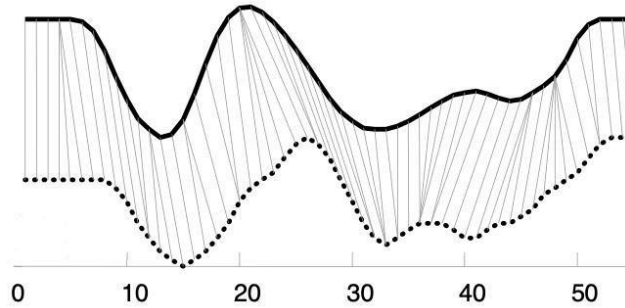


图 5-5 动态时间规整

5.2.2 DTW 面临的问题

DTW 距离放宽了全局形态特征相似性在尺度上的限制，可以处理不等长的时间序列^[5]。它在一定程度上克服了尺度位移的问题，但依然无法发现具有强度拉伸或位移的相似形态特征^[6]。尺度位移和强度拉伸或位移的情况如图 5-6 和图 5-7 所示。

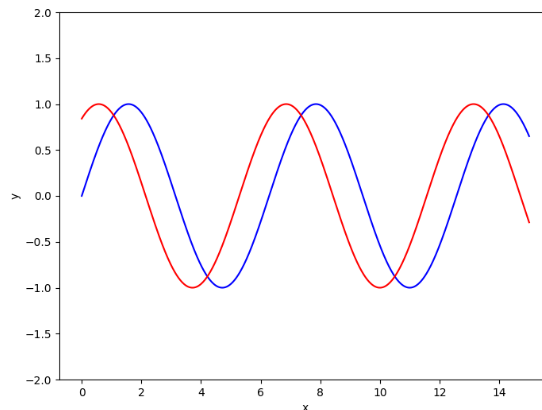


图 5-6 尺度位移

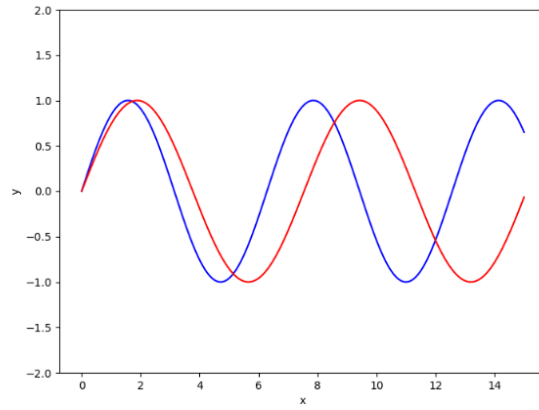


图 5-7 强度拉伸或位移

考虑到由于个体差异导致不同病人水肿体积随时间进展模式相同但水肿体积不同，如图 5-8，可以看出 sub6 和 sub11 两位病人的水肿体积随时间均是减小的，即曲线的走势相同，有相同的进展模式。但 sub6 和 sub11 两位病人的水肿体积的初始值不同，导致图像发生了强度拉伸和强度位移。在这种情况下，DTW 不能很好的规整二者的进展模式。

这里是由于某些情况水肿体积较大导致 DTW 不能很好的规整进展模式，同样的，由于某些情况时间较大也会影响 DTW 的规整结果。

考虑到由于体积和时间的初始值或者最大值会对 DTW 的规整，可以对每位病人分别进行数据的标准化。

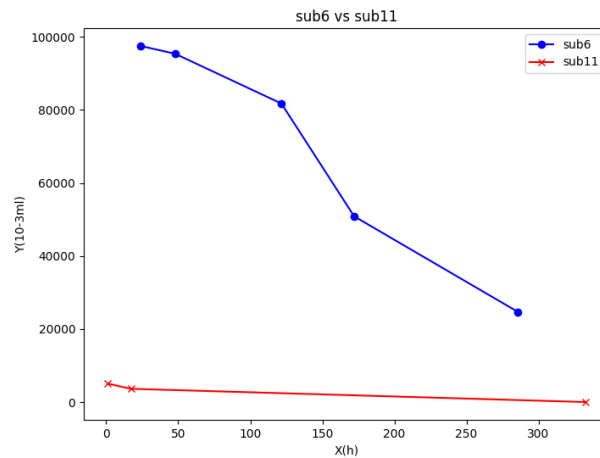


图 5-8 sub6 和 sub11 的水肿体积随时间进展模式

5.2.3 数据标注化解解决 DTW 不能规整强度拉伸问题

对每位病人分别进行数据的 Z-Score 标准化。Z-Score 标准化公式计算如下：

$$X_{norm} = \frac{X - \mu}{\sigma} \quad (5-11)$$

其中， X_{norm} 是标准化后的值， X 是原始值， μ 是数据集的均值， σ 是数据集的标准差。对于每一位病人的 $x(h)$ ， $y(10^{-3}ml)$ 分别标准化，记录 100 个患者 (sub001 至 sub100) 各自的 μ_x 、 σ_x 、 μ_y 、 σ_y 。

同样对于 sub6 和 sub11 两位病人的水肿体积随时间变化情况，将 x 、 y 标准化后的水

肿体积随时间进展模式如图 5-9 所示。

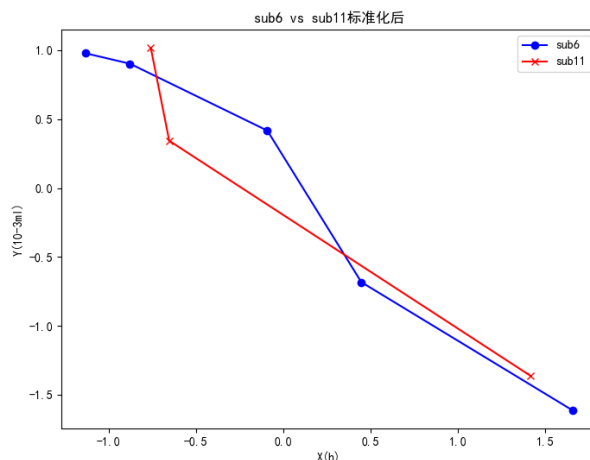


图 5-9 sub6 和 sub11 标准化后的水肿体积随时间进展模式

对比图 5-8 和图 5-9，可以看出标准化可以很好的消除因水肿绝对体积差异对水肿体积随时间进展模式的影响，从而凸显曲线的变化特征。

5.2.4 基于 DTW 距离的 K-Means 聚类分析

K-Means 算法的核心思想是将数据点分成 **K** 个簇，其中每个簇的中心是该簇中所有数据点的平均值（质心）。传统 **K-Means** 聚类分析采用欧氏距离作为样本之间的距离度量方法，但对于时间序列的时间偏移和形状差异的问题，欧氏距离并不适用。对于这个为非等长（随访次数）非均匀采样（随访频率）的时间序列聚类问题，采用 **DTW** 距离处理时间序列之间的时间偏移和相似性^[4]。

通过手肘法确定聚类的 **K** 值。手肘法计算每个簇下的簇内平方和。通常情况下，随着簇数量的增加，每个簇内的数据点更容易靠近其簇质心，簇内平方和会逐渐减小。当簇数量增加到一定程度时，簇内平方和的下降幅度会减小。手肘法的目标是找到簇内平方和开始迅速减小的"肘部"，代表表示数据的自然分割点，即最佳的 **K** 值。计算得到的簇内平方和如图 5-11。

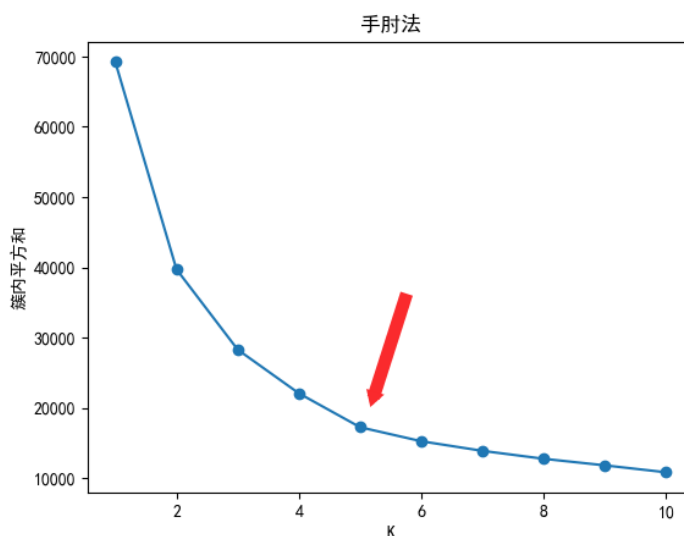


图 5-11 手肘法与簇内平方和

从图像中可以看出，当 $K=5$ 时，对应"肘部"，故选取 $K=5$ 为数据的最佳聚类点。得到的聚类结果如图 5-12、5-13、5-14、5-15、5-16。

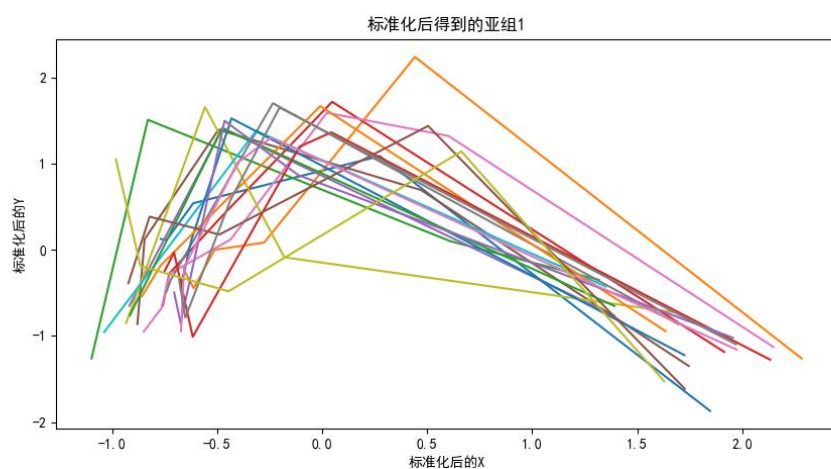


图 5-12 标注化后聚类得到的亚组 1

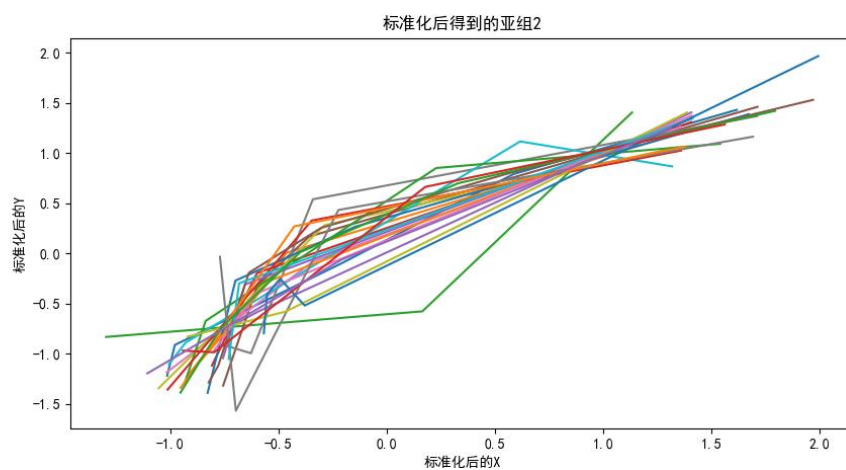


图 5-13 标注化后聚类得到的亚组 2

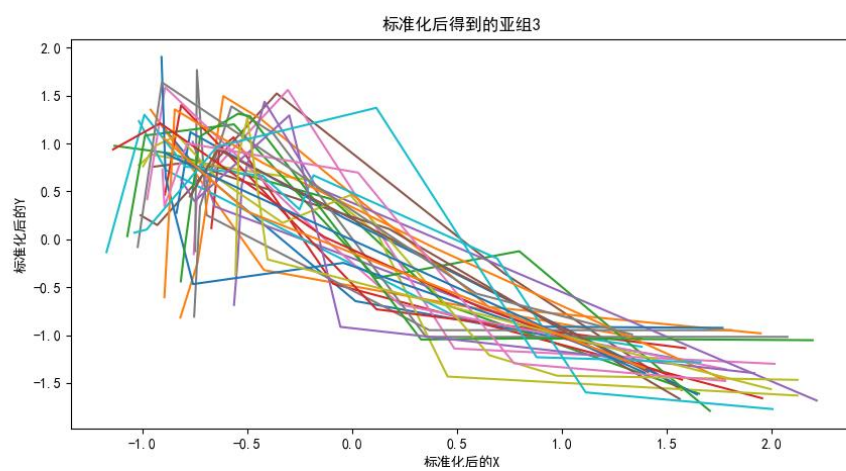


图 5-14 标注化后聚类得到的亚组 3

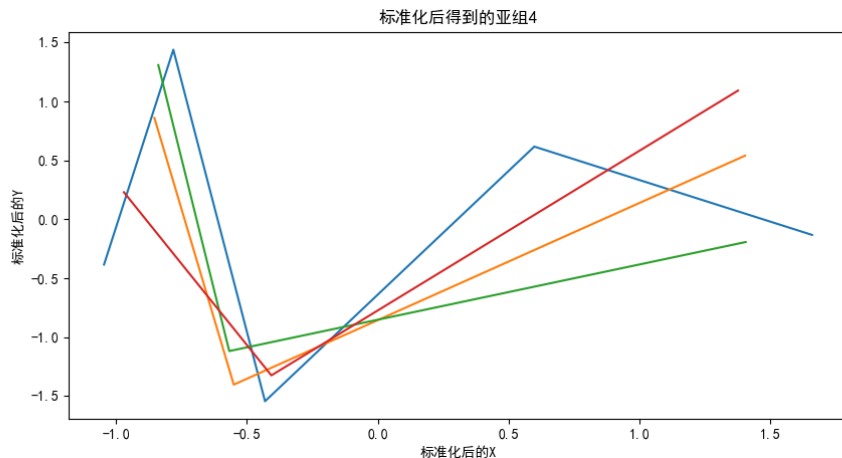


图 5-15 标注化后聚类得到的亚组 4

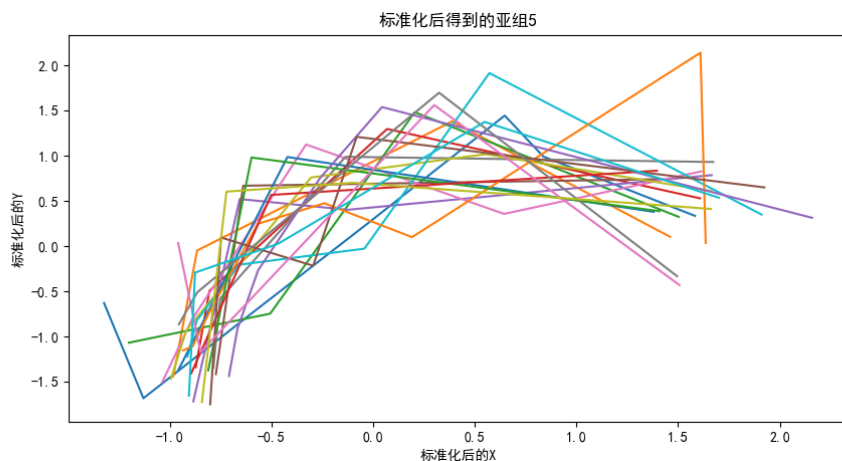


图 5-16 标注化后聚类得到的亚组 5

分析聚类结果，可以得出：亚组 1 表示入院后水肿体积先增大后减小，但最后一次随访检查到的水肿体积相较于入院时有所好转；亚组 2 表示入院后水肿体积持续增大，表明入院后病情在持续恶化；亚组 3 表示入院后水肿体积逐渐减小，表明入院后病情得到控制，病状持续减轻；亚组 4 表示入院后水肿体积先减小后增大，表明入院后病情好转了一段时间后突然恶化；亚组 5 也表示入院后水肿体积先增大后减小，相较于亚组 1，最后一次随访检查到的水肿体积相较于入院时依然发生恶化，但也过了拐点，病情也在逐渐好转。

综上所述，基于 DTW 距离的 K-Means 聚类分析可以较好的区分患者水肿体积随时间进展模式的个体差异，分出的 5 个亚组均有较高的可解释性，取得了较好的结果。

5.2.5 基于聚类结果的回归模型

分析 5.2.4 得到的 5 个亚组，采用多项式回归建立 5 个亚组各自的回归模型。取 5 个亚组标准化后的散点 (x,y) ，采用多项式回归，得到回归曲线如图 5-17、5-18、5-19、5-20、5-21。

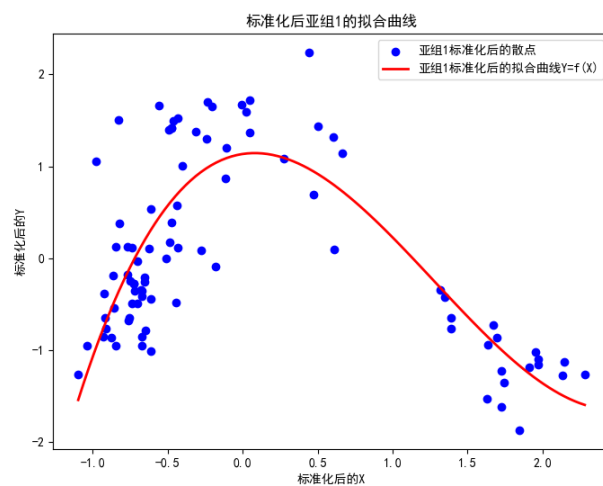


图 5-17 标准后亚组 1 的拟合曲线

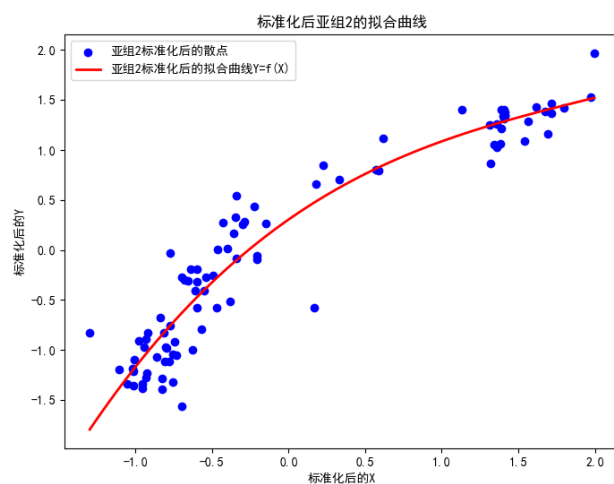


图 5-18 标准后亚组 2 的拟合曲线

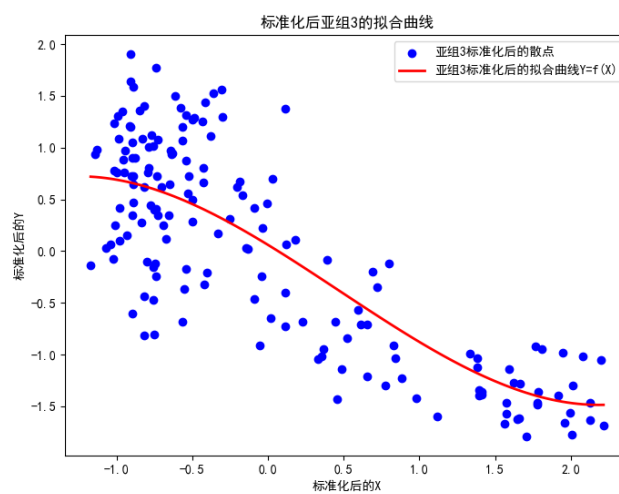


图 5-19 标准后亚组 3 的拟合曲线

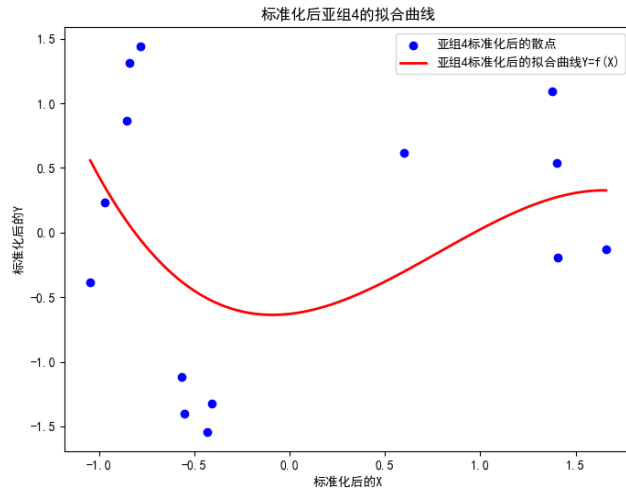


图 5-20 标准后亚组 4 的拟合曲线

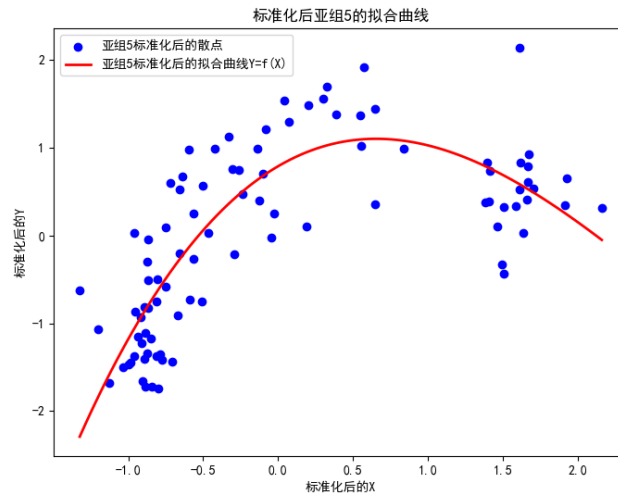


图 5-21 标准后亚组 5 的拟合曲线

亚组 1-5 的多项式回归拟合方程为：

亚组 1：

$$\hat{y}_1 = 0.4048x^3 - 1.5541x^2 + 0.2431x + 1.1345 \quad (5-12)$$

亚组 2：

$$\hat{y}_2 = 0.0579x^3 - 0.3463x^2 + 1.0700x + 0.3051 \quad (5-13)$$

$\hat{y}_2 = 0.0579x^3 - 0.3463x^2 + 1.0700x + 0.3051$ 亚组 3：引入岭回归，L2 正则化项的 $\alpha=8$ 。

$$\hat{y}_3 = -0.3669x^3 + 0.8575x^2 + 0.1632x - 0.6297 \quad (5-14)$$

亚组 4：引入 Lasso 回归，L1 正则化项的 $\alpha=0.048$ 。

$$\hat{y}_4 = -0.3668x^3 + 0.8574x^2 + 0.1631x - 0.6297 \quad (5-15)$$

亚组 5：

$$\hat{y}_5 = 0.1033x^3 - 0.8668x^2 + 1.0007x + 0.7920 \quad (5-16)$$

注意，这里得到的标准化后拟合曲线，如果想计算残差，还要用 5.2.3 提到的 100 个患者（sub001 至 sub100）各自的 μ_x 、 σ_x 、 μ_y 、 σ_y 进行反向标准化：

标准化后模型预测的值

$$\hat{y}_i = \frac{y_p - \mu_y}{\sigma_y} \quad (5-17)$$

即反向标准化后的预测值

$$y_p = \sigma_y \hat{y}_i + \mu_y = f(\hat{X}_i) = f\left(\frac{x_i - \mu_x}{\sigma_x}\right) \quad (5-18)$$

$$e = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - y_p) \quad (5-19)$$

其中， n 是一个患者的检查次数， $\hat{y}_i$ 是亚组 i 标准化后的预测值， x_i 、 y_i 是患者标准化后的某次检查的时间和水肿体积。

计算后前 100 个患者在不同亚组中的残差并填入“表 4-答案文件”。

5.3 问题二 c：模型的建立与求解

在得到五种水肿体积进展模式后，我们将样本按照进展模式分为 5 组，如图所示。

如果将其看为五种不同类，对应根据病人状况确定其发展模式会是一个分类问题。可以考虑各种治疗方法与各发展模式类的相关性。分类问题常常使用 One-Hot 编码，例如，对于上述五类发展模式使用 One-Hot 编码的结果如表 5-2 所示。

表 5-2 五类发展模式的 One-Hot 编码

水肿发展模式	One-Hot 编码
模式一	[1,0,0,0,0]
模式二	[0,1,0,0,0]
模式三	[0,0,1,0,0]
模式四	[0,0,0,1,0]
模式五	[0,0,0,0,1]

然而这样的编码实际上并不合理，这五类发展模式并不是完全不一致的，某些类之间存在一定的相似性。如发展模式一和模式五，从曲线图上可以直观看出它们之间有很强的相似性，而发展模式二和发展模式三，这是趋势差别很大的两类曲线，然而这些类之间的编码距离都是 2，即所有类间距离都是一致的，这显然与我们前一问的结果不相符。其中距离需要一个度量，这里选取的距离度量应满足如下性质：

非负性： $S(i, j) \geq 0$;

同一性： $S(i, j) = 0$ 当且仅当 $i = j$

对称性： $S(i, j) = S(j, i)$

直递性： $S(i, k) \leq S(i, j) + S(j, k)$

这里选取欧式距离来度量模式间的距离，其计算公式为：

$$S(i, j) = \|\mathbf{x}_i - \mathbf{x}_j\| = \sqrt{\sum_{u=1}^n |x_{iu} - x_{ju}|^2} \quad (5-20)$$

式中， $\mathbf{x}_i, \mathbf{x}_j$ 分别表示第 i 类和第 j 类的类向量， x_{iu} 是向量 $\mathbf{x}_i$ 的第 u 个分量。

那么，采样这样的编码难以有效发现治疗方法与发展模式之间的相关性。因此，为了能很好地衡量不同类之间相似性或者差距，我们重新对发展模式进行量化和编码。具体步骤如下：

首先将水肿体积发展过程分为 3 个阶段：前期、中期和后期。再在每个阶段根据水肿体积的相对大小进行评级（分箱操作），设置 0、1、2、3 四个等级，分别代表水肿程度小、中、大、严重。因此每个病人都会得到三个评级（可能有随访数据不够，导致该时期无评级，只要剔除缺失数据就行，不影响后续做相关性分析）。对五类发展模式重新表述三个时期的等级评分，如表 5-3 所示。

表 5-3 水肿发展模式对应的评级

水肿发展模式	前期状况评级	中期状况评级	后期状况评级
模式一	1	3	0
模式二	1	2	3
模式三	3	1	0
模式四	2	1	3
模式五	0	3	2

采用这种编码方式后重新计算一下类间距离： $S(1, 5)=5$ ， $S(1, 2)=10$ ， $S(2, 3)=14$ ，相似类的类间距离小，不相似类的类间距离大，显然这与我们的直觉是相符的。需要说明的是，实际样本可能会出现[1,3,1]的评级状况，虽然它不符合上述任意一项类编码，但我们可以将其归于模式一，因为它与模式一的距离只有 1，我们认为这是类内距离，即它和模式一是一类。

接下分别考察不同治疗方法对前、中、后期状况评级的影响。这里我们使用趋势性卡方检验，又称 Cochran-Armitage 趋势检验（Cochran-Armitage Trend Test），它是一种用于分析两个分类变量之间是否存在有序趋势的统计检验方法。通常情况下，其中一个分类变量是二元的，表示两种不同的情况，而另一个分类变量是有序的，表示多个等级或水平。这个检验的主要目的是确定两个变量之间是否存在线性趋势，即一个变量的增加（或减少）是否与另一个变量的概率变化呈现有序的关联。它通常用于分析两个变量之间的剂量-响应关系，以检测某种变化是否随着另一变量的增加或减少而呈现出趋势。

Cochran-Armitage 趋势检验的基本思想是通过计算卡方统计量来检验两个变量之间的有序关联性。如果卡方统计量的值显著，那么可以得出两个变量之间存在有序趋势的结论。

通常，该检验包括以下步骤：

提出假设：明确零假设（两个变量之间没有有序趋势）和备择假设（两个变量之间存在有序趋势）。

计算卡方统计量：使用观察到的数据计算卡方统计量，该统计量度量了观察到的有序趋势与期望有序趋势之间的差异。

计算自由度：确定卡方统计量的自由度。

计算 p-值：根据卡方统计量和自由度计算 p-值。

判断显著性：比较 p-值与显著性水平（通常为 0.05）。如果 p-值小于显著性水平，则拒绝零假设，认为存在有序趋势。否则，接受零假设，认为没有有序趋势。

Cochran-Armitage 趋势检验通常用于医学研究、流行病学和生物统计学等领域，以探索剂量-响应关系和有序分类变量之间的关联性。这个检验提供了一种强大的工具，可以帮助研究人员确定变量之间的趋势性关系。结果如表 5-4、5-5、5-6。

表 5-4 各治疗方法对前期发展的影响

治疗方法	统计量	P 值	结论
脑室引流	2.2304	0.5260	对改善无显著影响
止血治疗	6.8682	0.0762	对前期改善有显著影响
降颅压治疗	3.9947	0.0493	对前期改善有显著影响
降压治疗	4.6983	0.1953	对改善无显著影响
镇静、镇痛治疗	1.9073	0.5919	对改善无显著影响
止吐护胃	0.4803	0.9232	对改善无显著影响
营养神经	4.2969	0.2311	对改善无显著影响

表 5-5 各治疗方法对中期发展的影响

治疗方法	统计量	P 值	结论
脑室引流	4.4326	0.1090	对中期改善有显著影响
止血治疗	0.9356	0.6264	对改善无显著影响
降颅压治疗	3.7112	0.1564	对改善无显著影响
降压治疗	2.6767	0.2623	对改善无显著影响
镇静、镇痛治疗	0.2594	0.8783	对改善无显著影响
止吐护胃	0.1174	0.9429	对改善无显著影响
营养神经	1.3978	0.4971	对改善无显著影响

表 5-6 各治疗方法对后期发展的影响

治疗方法	统计量	P 值	结论
脑室引流	2.2485	0.3249	对改善无显著影响
止血治疗	5.6055	0.0606	对后期改善有显著影响
降颅压治疗	1.0702	0.5856	对改善无显著影响
降压治疗	4.8270	0.0895	对后期改善有显著影响
镇静、镇痛治疗	0.9039	0.6364	对改善无显著影响
止吐护胃	0.4842	0.7850	对改善无显著影响
营养神经	3.0841	0.2139	对改善无显著影响

首先分析五类发展模式，第一类发展模式水肿体积的改善主要发生在中期和后期；第二类发展模式全局都没有改善；第三类发展模式水肿体积在前、中后期都有所改善；第三类发展模式水肿体积在前、中后期都有所改善；。显然，总结上表可以得出脑室引流有助于对水肿体积中期的改善，做过脑室引流的病人的水肿体积发展模式更有可能是模式一或者模式三、四，止血治疗对前期和后期的水肿影响比较大，可能导致的发展模式是模式三。

降颅压治疗的效果主要体现在中期，因此可能导致的发展模式是模式一；降压治疗主要对后期水肿发展改善有影响，可能的发展模式是模式五；镇静、镇痛治疗，止吐护胃以及营养神经等治疗方式对前中后期影响都不显著，因此认为这三种治疗方法对发展模式没有影响。此外，还应该注意，各种治疗方法之间并不是完全独立的，一些治疗方法往往会同时用到，作为组合治疗的手段。图 5-22 治疗方法间的热力图，从图中可以看出，降颅压治疗和止血治疗之间有着较强的相关性，止吐护胃和镇静、镇痛治疗之间也有较强的相关性，根据我们对数据的统计，这些治疗组合出现频率很高。

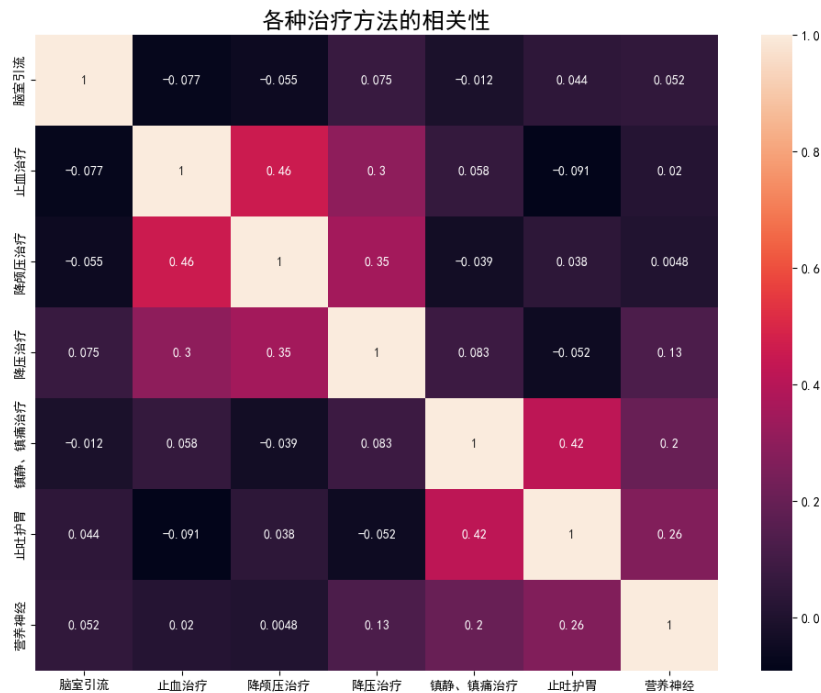


图 5-22 疗方法间的热力图

5.4 问题二 d：模型的建立与求解

分析血肿体积、水肿体积及治疗方法三者之间的关系。两两考虑二者的关系，分别探讨血肿体积与治疗方法、水肿体积与治疗方法、血肿体积与水肿体积之间的关系。根据“表 1-患者列表及临床信息”与“表 2-患者影像信息血肿及水肿的体积及位置”，由于 sub101-sub130 这 30 名患者只有入院首次影像检查的信息，蕴含的数据信息不足，故舍弃。因此选取 sub1-sub100 和 sub131-sub160 共 130 名患者的信息。

5.4.1 血肿体积、水肿体积与治疗方法之间的关系

目前共有脑室引流；止血治疗；降颅压治疗；降压治疗；镇静、镇痛治疗；止吐护胃；营养神经共 7 种治疗方式。根据是否使用该治疗方式为界，将数据集划分为两部分。130 名患者中：

- 使用脑室引流的病人有 9 人，不使用有 121 人；
- 使用止血治疗的病人有 99 人，不使用有 31 人；
- 使用降颅压治疗的病人有 99 人，不使用有 31 人；
- 使用降压治疗的病人有 117 人，不使用有 13 人；
- 使用镇静、镇痛治疗的病人有 95 人，不使用有 35 人；
- 使用止吐护胃的病人有 122 人，不使用有 8 人；
- 使用营养神经的病人有 122 人，不使用有 8 人；

由于每人的检查次数不同，寻求合理的统计量来衡量样本特征。对每位患者分析其多次检查的血肿、水肿体积的均值和最大值作为特征，根据是否使用某个治疗方式进行相关性分析。

首先对血肿、水肿即根据是否使用这 7 种治疗方式将样本分 7 次划分为共 24 个子样本，使用 Shapiro-Wilk 检验分析其是否满足正态分布。

对于血肿体积，发现只有使用脑室引流的 9 个病人、不使用降压治疗的 13 个病人满足正态分布。

对于水肿体积，发现只有使用脑室引流的 9 个病人、不使用止吐护胃的 8 个病人满足正态分布。

对于这些满足正态分布子样本，出现的次数少且样本小，无法反应总体样本服从正态分布。因此进行相关性分析时，使用无要求样本服从正态分布的秩相关系数来进行分析。本文采用 Mann-Whitney U 检验分析血肿、水肿体积与 7 种治疗方式的相关性。

Mann-Whitney U 检验假设：

H0: 两组独立样本的分布相同，即总体的中位数相等。

H1: 两组独立样本的分布不同，即总体的中位数不等。

将两组独立样本合并成一个样本，并为每个观察值标记其来源组别；对合并后的样本按照观察值的大小进行排序，同时保留其来源组别信息；对每个组别的观察值进行秩次转换，用排名替代原始数值。计算每个组别的秩的和，即 U 统计量。使用 U 统计量计算 P 值。对比临界值或 P 值与显著性水平（0.05）进行假设检验的判断。如果计算得到的 P 值小于显著性水平，通常拒绝零假设，认为两组独立样本之间存在显著差异。否则，接受零假设，认为两组样本之间没有显著差异。

计算得到的血肿体积的均值和最大值与 7 种治疗方式的 Mann-Whitney U 检验分析如表 5-7、5-8。

表 5-7 不同治疗方式对血肿体积均值的影响

治疗方法	P	U	影响情况
脑室引流	0.0001	112	存在显著影响
止血治疗	0.2423	1320	无显著影响
降颅压治疗	0.0001	803	存在显著影响
降压治疗	0.1015	549	无显著影响
镇静、镇痛治疗	0.8460	1625	无显著影响
止吐护胃	0.8829	504	无显著影响
营养神经	0.6651	442	无显著影响

表 5-8 不同治疗方式对血肿体积最大值的影响

治疗方法	P	U	影响情况
脑室引流	0.0001	120	存在显著影响
止血治疗	0.2782	1335.5	无显著影响
降颅压治疗	0.0001	812.5	存在显著影响
降压治疗	0.0598	517.5	无显著影响
镇静、镇痛治疗	0.4917	1531	无显著影响
止吐护胃	0.7529	455	无显著影响
营养神经	0.5643	428	无显著影响

计算得到的水肿体积的均值和最大值与 7 种治疗方式的 Mann-Whitney U 检验分析如表 5-9、5-10。

表 5-9 不同治疗方式对水肿体积均值的影响

治疗方法	P	U	影响情况
脑室引流	0.0436	324	存在显著影响
止血治疗	0.4347	1391	无显著影响
降颅压治疗	0.0002	860	存在显著影响
降压治疗	0.0046	395	存在显著影响
镇静、镇痛治疗	0.5566	1550	无显著影响
止吐护胃	0.7213	450	无显著影响
营养神经	0.0687	300	无显著影响

表 5-10 不同治疗方式对水肿体积最大值的影响

治疗方法	P	U	影响情况
脑室引流	0.0399	320	存在显著影响
止血治疗	0.8164	1491.5	无显著影响
降颅压治疗	0.0007	916.5	存在显著影响
降压治疗	0.0151	447	存在显著影响
镇静、镇痛治疗	0.3061	1467	无显著影响
止吐护胃	0.6076	434	无显著影响
营养神经	0.0527	287	无显著影响

根据表 5-7、5-8、5-9、5-10，我们可以得出：

脑室引流、降颅压治疗这两种治疗方式对血肿体积存在显著影响；

脑室引流、降颅压治疗、降压治疗这三种治疗方式对水肿体积存在显著影响；

其他治疗方式对血肿、水肿体积无显著影响。

5.4.2 血肿体积与水肿体积之间的关系

首先分析血肿体积和水肿体积之间的相关性，提取每位病人血肿体积和水肿体积的多次检查结果的平均值，绘制这两个变量的联合分布图，如图 1-1 所示。

同时计算二者之间的相关性系数值，其中 Pearson 相关系数的值为 0.3617545006446199，说明二者之间存在弱的线性相关性。Spearman 相关系数的值为 0.5643529202911415，说明二者的关系是正相关的。

首先分析血肿体积和水肿体积之间的相关性，提取每位病人血肿体积和水肿体积的多次检查结果的平均值，绘制这两个变量的联合分布图，如图 5-23 所示。

同时计算二者之间的相关性系数值，其中 Pearson 相关系数的值为 0.3617545006446199，说明二者之间存在弱的线性相关性。Spearman 相关系数的值为 0.5643529202911415，说明二者的关系是正相关的。

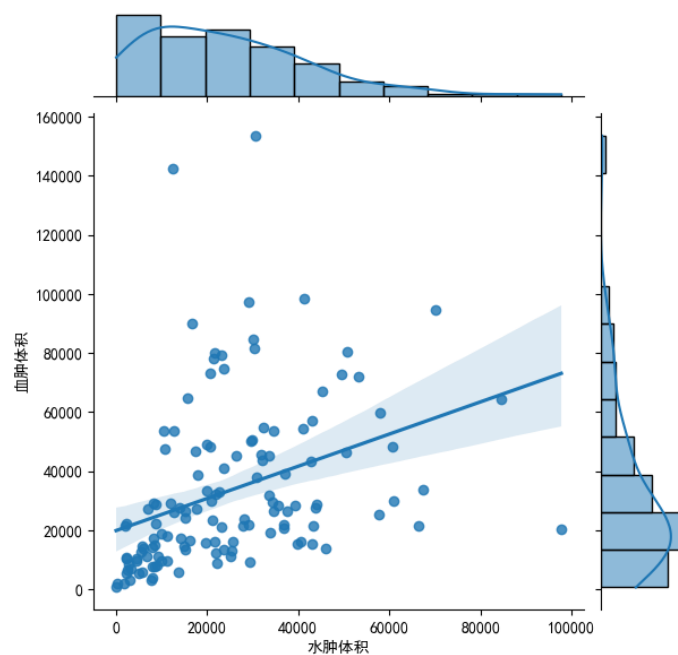


图 5-23 血肿体积和水肿体积的联合分布图

5.5 表 4-答案文件中问题二的计算结果

		问题 2：水肿体积进展曲线		
	首次影像检查流水号	残差（全体）	残差（亚组）	所属亚组
sub001	20161212002136	278542.3815	-1883.032312	2
sub002	20160406002131	6624.266163	232.2616199	1
sub003	20160413000006	58603.41005	-140.5892732	2
sub004	20161215001667	54614.6144	-529.7038263	3
sub005	20161222000978	44022.01546	-3493.826222	3
sub006	20161110001074	224687.7815	-13435.34192	3
sub007	20161208000139	66751.28228	-5118.794161	5
sub008	20161219000091	40825.85659	-1811.129529	5
sub009	20161031001987	24772.37565	-2200.277936	2
sub010	20161012002008	31260.96956	5656.797392	3
sub011	20160209000219	69589.52879	-266.1396616	3
sub012	20161031001142	67754.07016	23797.94002	1
sub013	20161124000397	90862.75774	-2203.473405	3
sub014	20160513001799	15544.04374	-35.75954992	2
sub015	20161013001234	117994.3241	-4980.473715	5
sub016	20161130000004	87191.32034	-145.1995876	2
sub017	20160510002436	19824.08012	103.7480954	5
sub018	20160602001707	70245.74342	1259.12515	2
sub019	20160117000135	49972.21905	5027.261639	3

sub020	20160723000013	63222.44909	28.34001286	3
sub021	20160317001244	38711.46246	213.2932346	5
sub022	20160803001239	55830.23723	-12.47385684	5
sub023	20160321000142	89515.73094	7539.298446	3
sub024	20170802000637	62492.52929	-2536.648896	3
sub025	20171226002293	17215.02515	-311.6647597	2
sub026	20171008000512	42930.40223	1362.884791	2
sub027	20170206000071	58632.15749	-17361.65557	1
sub028	20171013002097	76328.40381	10271.49462	1
sub029	20170607000010	76653.72308	-112.79269	3
sub030	20171025000480	194807.2466	-3391.139075	5
sub031	20170307002130	41073.59325	-592.4257497	2
sub032	20171009000137	70049.12319	-1400.473394	1
sub033	20170115000362	83145.63749	3107.616017	3
sub034	20170119000729	113117.8475	2654.166103	5
sub035	20171014001244	84254.78031	-3246.746011	3
sub036	20170204001714	64045.11331	79.12905363	4
sub037	20170426000005	75137.6679	-9585.531645	1
sub038	20170518002194	34662.8838	-762.4695191	5
sub039	20170425002487	240780.9385	24136.25926	1
sub040	20170902000876	117716.8984	3440.398533	5
sub041	20171002000282	68729.38996	-162.1062357	1
sub042	20170420000636	64993.78523	-1356.920854	3
sub043	20170325000428	20944.86007	-612.1758317	1
sub044	20170528000084	2711826.608	36188.35289	3
sub045	20170324001892	28383.60908	-1766.190749	2
sub046	20170511000016	42542.66692	-4031.114259	1
sub047	20171019001652	54116.21782	945.4371469	1
sub048	20170402000556	75339.24455	1269.996128	2
sub049	20171005000770	100585.0969	-398.8880091	3
sub050	20171105000372	41446.22657	-46.08717579	2
sub051	20170422000935	55222.23129	-616.7392801	3
sub052	20170608001310	117571.6434	24901.67851	3
sub053	20170511001392	31896.27964	-5573.355496	1
sub054	20170612002216	32678.3646	-184.8622398	2
sub055	20170316001977	32916.10372	-536.8513118	2
sub056	20170120000152	153192.185	1341.207334	3
sub057	20170825001844	52104.35205	-0.874126135	2
sub058	20170125000984	22244.59751	-161.7713105	5
sub059	20170912002314	74667.41885	-439.7477773	1
sub060	20180109000613	109304.2222	-2312.491726	3
sub061	20180226000725	212080.2623	4653.300137	2
sub062	20181221002264	31037.46995	-19.78410701	2

sub063	20181020001229	226303.5347	1836.948832	5
sub064	20180801000501	117752.447	-642.8545041	3
sub065	20180131001727	75636.46674	299.6285173	2
sub066	20181208000909	57937.57337	-66.78636443	3
sub067	20181207001317	53916.62789	-63.5556769	2
sub068	20180412001426	364634.9126	17055.64423	3
sub069	20180619001505	83017.4256	-7979.775348	3
sub070	20180427000292	43992.41651	-2.44116445	2
sub071	20181103001264	118799.7339	26645.52813	1
sub072	20181007000826	31024.18149	-43.30063132	5
sub073	20180911001645	57598.63998	-42.58618057	5
sub074	20180719000020	209216.5841	11080.7274	5
sub075	20180428001767	30974.80207	7601.417261	1
sub076	20180619002401	83951.38398	-1388.403964	1
sub077	20180503002304	45329.31307	3239.543534	5
sub078	20180929000040	73075.96154	-39.15645822	4
sub079	20180929000037	101541.9032	1723.88121	3
sub080	20180130001917	47305.43315	-437.0384659	3
sub081	20180120000249	5888499.395	5766.463821	2
sub082	20180221000793	24026.00899	-118.448319	4
sub083	20181004000706	34102.73486	-258.283136	2
sub084	20180716000006	36101.71183	659.3711063	2
sub085	20181127002511	157804.0196	-4500.868823	3
sub086	20180108000002	99790.14197	-174.6250041	5
sub087	20180216000198	102275.1282	1393.521447	1
sub088	20180521000314	14561.61565	580.7309307	4
sub089	20180314002318	56173.53613	-2435.464941	5
sub090	20180910002366	112622.6428	-19237.4754	3
sub091	20181019001130	69874.48002	197.6698939	5
sub092	20181116001089	67986.22021	-484.017776	2
sub093	20181214000208	95985.38961	1602.067713	1
sub094	20180412001795	59430.88321	5422.302109	3
sub095	20180316001329	363916.5825	-254.6324207	5
sub096	20180802001789	107855.7844	-2569.285846	3
sub097	20181010000767	57206.95535	-718.4862457	3
sub098	20180612002507	42916.99458	-8011.199657	1
sub099	20180620002296	19096.51905	-615.6387915	2
sub100	20180314000010	61802.65004	-325.5857449	3

六、问题三模型建立与求解

6.1 问题三 a：模型的建立与求解

对于问题 3（a）的求解，针对由表一中前 100 例患者的疾病相关信息、表二的影像检查结果以及表三首次影像检查结果建立预测模型来预测 sub001-sub160 患者的 90 天 mRS 评分。我们从以下四个步骤进行求解：

- （1）数据分布观察与处理：对数据的分布进行观察
- （2）相关性检验：利用 spearman 秩相关提取相关性强的特征。
- （3）建立模型：建立 XGBoost 回归模型，将经过特性提取后的数据放入到模型进行训练，经过网格搜索法求 $n_estimators$ 及 $learning_rate$
- （4）模型评价：使用均方误差、均方误差标准差、R-squared、R-squared 标准差作为模型评价指标进行模型评价得到最佳的 $n_estimators$ 以及 $learning_rate$

6.1.1 数据分布观察与处理

问题 3（a）的输入特征与问题 1（b）的输入特征一样，所以我们直接使用问题 1（b）处理过后的数据，观察数据分布情况，可以看出数据存在不平衡的问题。患者的 mRS 评分大部分的集中在等级 1-3 之间。其他等级的人数较少，可能会引起模型欠拟合 问题。

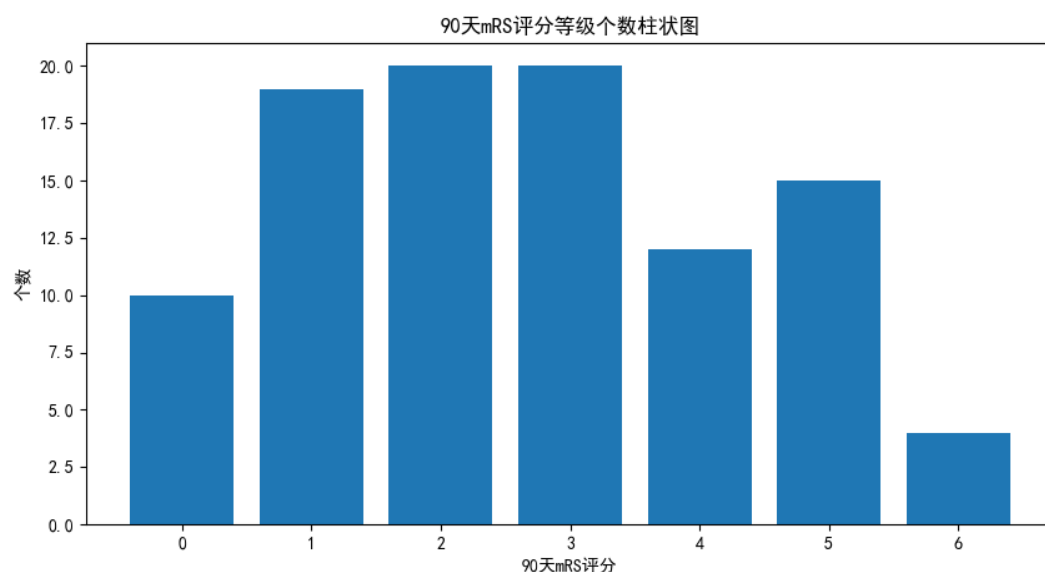


图 6-1 90 天 mRS 评分等级个数（平衡前）

我们通过简单随机过采样对数据进行平衡化，使得每个等级的人数都差不多以让模型具有鲁棒性。平衡化后数据如图 6-2 所示。数据清洗、数据补全、数据标准化等操作在问题 1（b）中已经完成，此处不赘述。得到的按 7:3 的比例划分为训练集和测试集，以待后续模型训练。

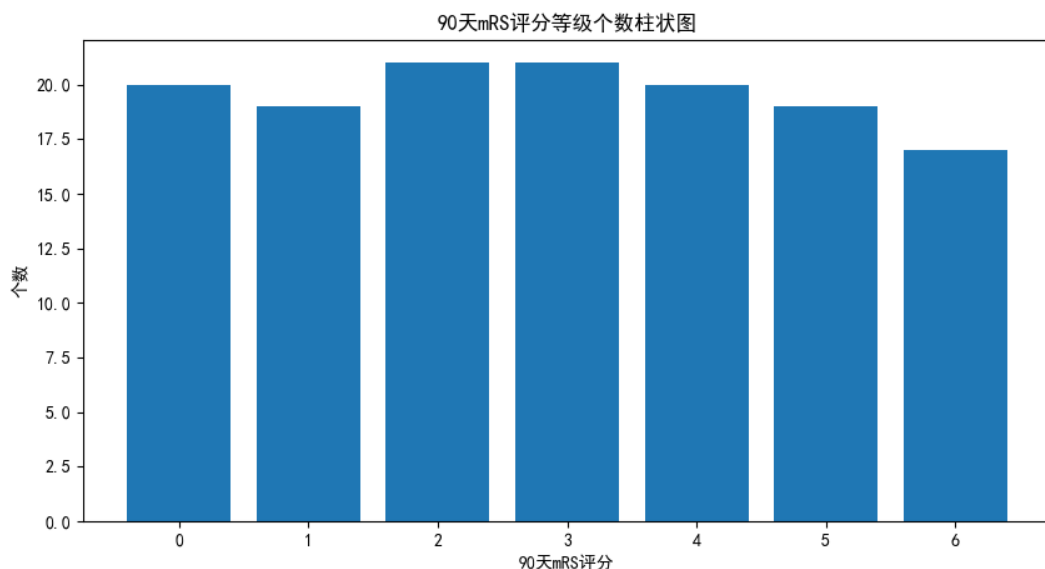


图 6-2 90 天 mRS 评分等级个数（平衡后）

6.1.2 相关性检验

输入数据的维度较高，有 74 列特征，所以考虑对数据的每一维特征进行相关性检验，求取相关性强的特征作为输入变量，可以帮助我们理解变量之间的相互影响，以及是否存在统计上显著的关联。相关性检验的常用方法有皮尔逊相关系数、spearman 秩相关系数等等。下表是相关性检验的常用方法以及相关性检验的常用方法的适用范围：

表 6-1 相关性检验的常用方法及其适用范围

相关性检验方法	适用范围
皮尔逊相关系数	用于衡量两个连续变量之间的线性关系。通常假设变量服从正态分布。
斯皮尔曼秩相关系数	用于衡量两个变量之间的单调关系，不要求变量服从正态分布，适用于有序数据或非线性关系。
肯德尔秩相关系数	用于衡量两个变量之间的有序关系，适用于有序数据或非线性关系。
点二列相关性	用于衡量两个分类变量之间的相关性，通常使用卡方检验。
回归分析	用于探究一个或多个自变量与因变量之间的关系。线性回归可以衡量连续变量之间的线性关系，逻辑回归用于二分类问题的相关性分析。

因为斯皮尔曼秩相关系数不要求变量服从正态分布，对于求解非线性关系具有很好的效果，所以我们使用斯皮尔曼秩相关系数对 74 列特征进行相关性分析。其计算公式为：

$$\rho=1-\frac{6*\sum d^2}{n*(n^2-1)} \quad (6-1)$$

其中 ρ 取值范围为 -1 到 1, -1 表示完全负相关, 1 表示完全正相关, 0 表示无相关性。

ρ 值大于 0.05 的表示特征与结果相关性较高, ρ 值小于 0.05 的表示特征与结果相关性较低。使用斯皮尔曼秩相关系数对 74 列特征进行相关性分析之后得到 55 列相关信息以及 19 列冗余信息。之后将相关性较高的特征输入模型训练。

6.1.3 建立 XGBoost 回归模型

XGBoost(eXtreme Gradient Boosting)是一种基于决策树集成的机器学习算法^[7]。它可以处理多种类型的数据, 包括数值型和类别型数据, 常用于分类和回归问题^[8]。

XGBoost 作为一种决策树集成算法将多棵决策树结合在一起, 并对决策树间关系进行优化, 以提高预测精度。每棵决策树是一个弱分类器, 将多个决策树弱分类器进行组合, 可以得到一个强分类器。XGBoost 使用的是一种基于梯度提升的迭代模型, 每轮迭代时, XGBoost 计算训练数据中模型的损失函数和加入新决策树以降低损失。在 XGBoost 中引入正则化项以避免所建模型复杂度过高, 以改善其泛化性能。XGBoost 也能计算出特征的重要程度, 从而有助于确定对于模型性能最有影响力的那些特征。

XGBoost 算法的模型建立流程如图 6-3:

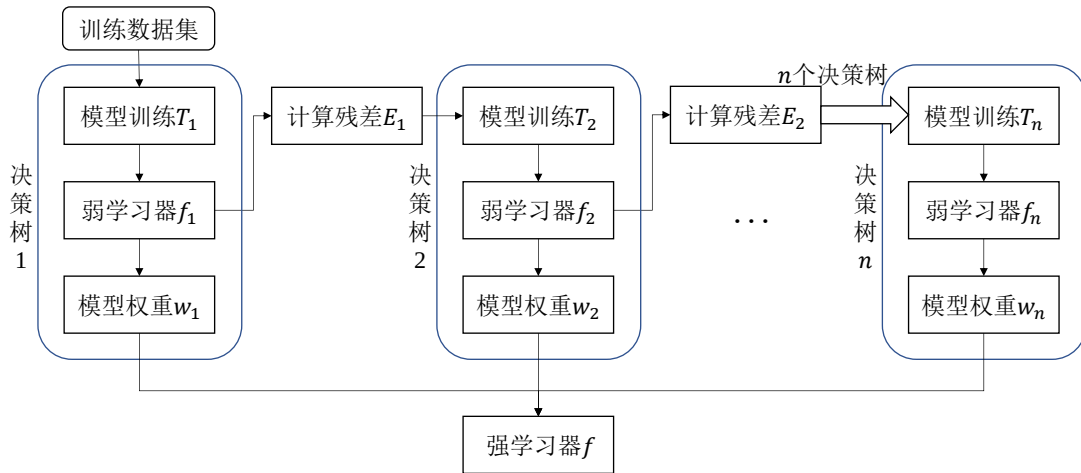


图 6-3 XGBoost 算法模型建立流程

XGBoost 算法的模型的推导公式为:

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i) = \hat{y}_i^{(t-1)} + f_i(x_i) \quad (6-2)$$

式中, $\hat{y}_i^{(t)}$ 表示第 t 次迭代后样品 i 的预测结果, $\hat{y}_i^{(t-1)}$ 表示前 $t-1$ 棵树的预测结果, $f_i(x_i)$ 表示第 t 棵树的结果。

目标函数如下:

$$Obj^{(t)} = \sum_{i=1}^t l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) + constant \quad (6-3)$$

$$Obj^{(t)} = \sum_{i=1}^t [l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t) + constant \quad (6-4)$$

式中，定义 $g_i = \partial \hat{y}_i^{(t-1)} l(y_i, \hat{y}_i^{(t-1)})$ 为 $\hat{y}_i^{(t-1)}$ 的一阶导数， $h_i = \partial^2 \hat{y}_i^{(t-1)} l(y_i, \hat{y}_i^{(t-1)})$ 为 $\hat{y}_i^{(t-1)}$ 的二阶导数。

树的复杂度定义为：

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2 \quad (6-5)$$

式中， T 为叶子节点数量， w 为叶子节点权重。考虑到 $l(y_i, \hat{y}_i^{(t-1)})$ 为定值，与常数项一起忽略，进而将目标函数简化为：

$$\begin{aligned} Obj^{(t)} &\approx \sum_{i=1}^t [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t) \\ &= \sum_{j=1}^T [(\sum_{i \in I_j} g_i) w_j(x_i) + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2] + \gamma T \end{aligned} \quad (6-6)$$

令 $G_j = \sum_{i \in I_j} g_i, H_j = \sum_{i \in I_j} h_i$ 。将得到的目标函数转换成一元二次方程问题。得到最优解 w_j^*

和目标函数最优解 L^* ，结果如下：

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (6-7)$$

$$L^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (6-8)$$

以上推导过程可以得出一棵最佳决策树以及决策树每个节点对应的最优得分，根据最优得分建立 XGBoost 算法的模型。

建立 XGBoost 模型首先需要初始化 XGBoost 回归模型的迭代次数、树的最大深度，利用网格搜索法利用评价指标来求取 XGBoost 回归模型的最佳超参数。评相关价指标有均方误差、均方误差标准差、R-squared、R-squared 标准差。

(1) 均方误差

均方误差是一种用于衡量模型预测值与实际观测值之间差异的指标。其计算公式如下：

$$MSE = \frac{\sum (y_i - \hat{y}_i)^2}{n} \quad (6-9)$$

y_i 为第 i 个观测值的实际值， $\hat{y}_i$ 为第 i 个观测值的预测值， n 为观测值的总数

(2) 均方误差标准差

均方误差标准差是均方误差的平方根，用于衡量模型的预测误差的标准差。其计算公式如下：

$$RMSE = \sqrt{MSE} \quad (6-10)$$

(3) R-squared（决定系数）

R-squared 是用于衡量一个模型对观测值变异性的解释程度，取值范围在 0 到 1 之间。值越接近 1 表示模型拟合得越好。其计算公式如下：

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (6-11)$$

其中 y_i ：第 i 个观测值的实际值， $\hat{y}_i$ ：第 i 个观测值的预测值， $\bar{y}$ ：所有观测值的平均值

(4) R-squared 标准差

R-squared 标准差是 R-squared 的标准差，这个参数用于衡量模型稳定性。如果 R-squared 标准差较小，则表示模型很稳定，在不同数据集上表现都很好。

四个参数在训练过程中随超参数的变化如图 6-4、图 6-5、图 6-6、图 6-7 所示；结果训练之后得到的最佳超参数如下表所示：

表 6-2 XGBoost 模型最佳超参数

Boost	n_estimators	Learning_rate
基于树的模型	25	0.17

基于 $n_estimators=25$ 、 $Learning_rate=0.17$ 的 XGBoost 模型预测患者 90 天 mRS 评分，最后得到的均方误差为 1.2322，均方误差标准差为 1.11，R-square 均值为 0.5637，R-square 均值标准差为 0.6605，预测正确的概率为 74%。模型的预测能力达到预期。

这里再定义一个指标——邻近命中率，其含义等同于正确率，但在统计正确次数时，预测到正确值的相邻数也会被视为预测准确一次。

均方误差随弱分类器数量与学习率的变化趋势

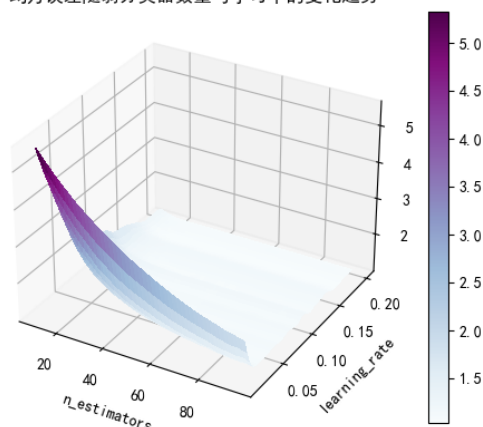


图 6-4 均方误差变化趋势

均方误差标准差随弱分类器数量与学习率的变化趋势

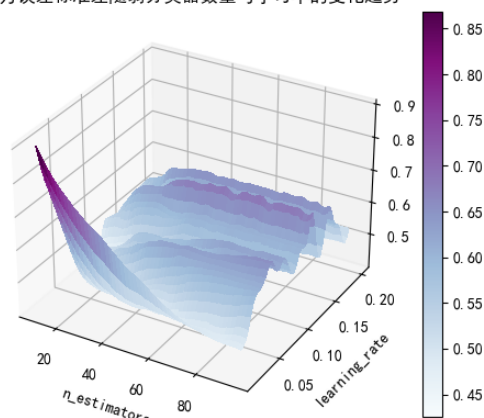


图 6-5 均方误差标准差变化趋势

R-squared均值随弱分类器数量与学习率的变化趋势

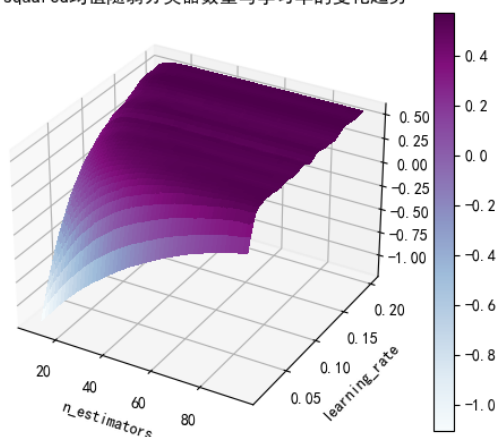


图 6-6 R-squared 均值变化趋势

R-squared标准差随弱分类器数量与学习率的变化趋势

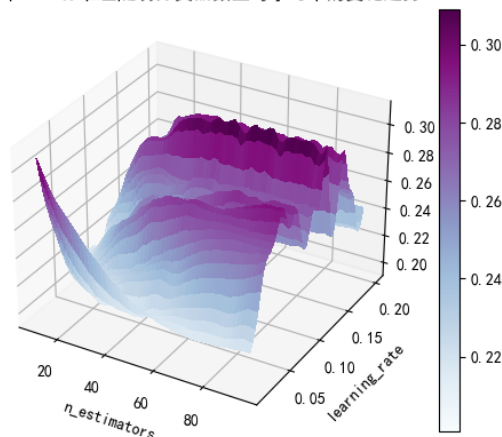


图 6-7 R-squared 均值标准差变化趋势

6.2 问题三 b: 模型的建立与求解

6.2.1 难点分析及解决思路

该问与前一问预测任务基本一致，但是可以纳入的信息更多了，拥有首次和各次随访检查的结果将有助于更加准确地预测患者的 mRS 评分。

然而，实际上每位患者可以为我们提供的额外信息并不一样多，有些患者有 8 次的随访记录和数，而有些患者可能仅有两次随访记录的数据。并且，他们每次随访的时间跨度并不一致。如果将患者影像检查结果作为一个时间序列来看待，那么很明显，每位患者的影像检查历史记录就是一个不规则采样时间序列 (irregularly sampled time series, ISTS)。这种不规则采样在医疗检测数据中很常见，例如两次身体化验的时间间隔是由病人的身体状况决定的，当病人的身体状况非常好时，可能会间隔较长的时才去进行一次体检。我们处理这个问题的难点列举如下：

- 1) 每个样本的时间序列不等长。影像检查结果具有时间顺序性，但每个病人影像检查的次数都不一致。

- 2) 每一个时间序列都不等间隔，即每一次检查距离上一次检查的时间跨度不是一个固定值（不定期检查）。
- 3) 采样点数少。首次影像距最后一次检查虽然可能时间跨度大，但中间过程检查次数稀少，平均每个病人检查次数只有 4 次。

传统机器学习模型对输入数据的维度有着严格的要求，一般不允许输入不同维度数的样本。一些深度学习模型支持输入变长序列，如 RNN、LSTM、GRU、seq2seq 等，但它们往往用于处理等间隔的时间序列，如果直接用在此处将会导致时间间隔这个信息被忽略掉。而在本问题中时间间隔又很关键。传统的时间序列分析方法一般用于处理均匀采样的时间序列，面对不等间隔时间序列，可能会将其视为等间隔序列的数据缺失，并采取插值等方法来将其补全，然而本案例中每个序列点数少，插值难以保证结果的可靠性。目前已有许多深度学习模型可以用于处理这类问题，它们不对序列插值，直接对 ISTS 建模。例如，文献^[9]提出了时间感知 LSTM (T-LSTM) 模型，连续记录之间的时间间隔包含在网络体系结构中。将时间间隔也作为变量输入到网络之中。这些模型都取得了不错的效果，但是这些模型结构更庞大，网络参数多，训练用到的数据集都是具有上万个样本的（如 MIMIC-III）。在本问题中我们只有不到 200 个样本，难以应用这些深度学习模型。可见，这些问题同时存在，对建模过程产生了极大的阻碍。

6.2.1 建模方案

问题三 a 和问题三 b 思路对比图如图 6-8。左侧是问题三 a 的流程框架图，右侧为问题三 b 的流程框架图。

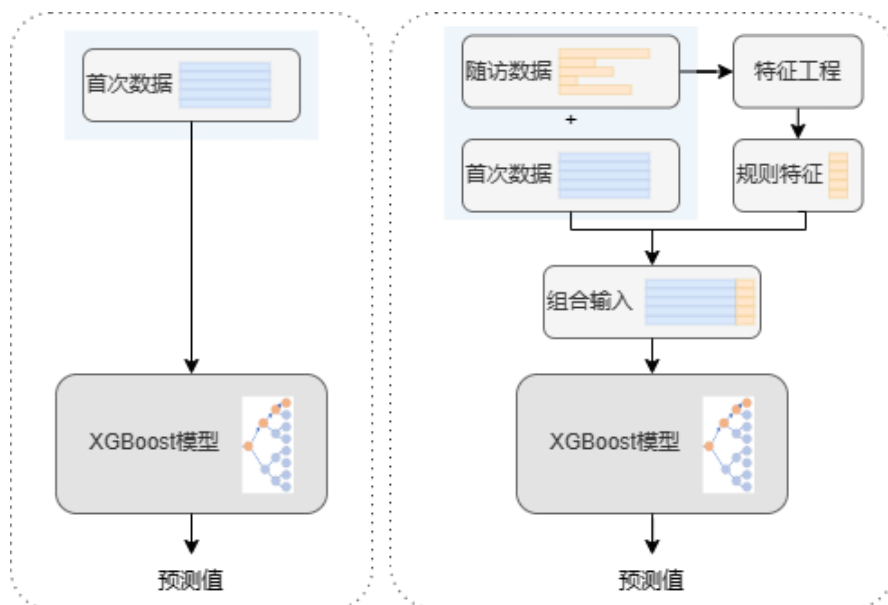


图 6-8 问题三 a 和问题三 b 流程框架对比图

既然无法直接将这种数据用于深度学习技术，那么我们可以考虑人工设计这种非均匀采样、不等长的短序列的时序特征。通常时间序列具有多个特征，每个特征刻画了时间序列的一个方面。从对时间序列不同层次上的认知可将时间序列特征分为 3 种：形态特征、结构特征以及模型特征^[10]。但现有的特征都不适用于我们的对象。因此，针对非均匀采样、不等长的短序列，我们设计了如下特征：

LATEST 权均值：

$$\bar{x} = \frac{\sum_{i=0}^N t_i x_i}{\sum_{i=0}^N t_i} \quad (6-12)$$

式中， t 表示第 i 次随访距离第一次发病的时长（当 $i=0$ 时表示首次检查结果）， x 表示第 i 次随访各影像学指标的结果。

Integrated Slope 斜度值，定义如下：

$$KI = \frac{\sum_{i=1}^N (x_i - x_0)}{N \sum_{i=1}^N (t_i - t_0)} \quad (6-13)$$

式中，各字母含义同前。

Hua 系数：

$$H = \frac{x_N}{\max \{x_i \mid i = 0, 1, 2, \dots, N-1\}} \quad (6-14)$$

式中，各字母含义同前

对每个时间序列样本的每一项指标，都计算这几个特征，并用这些特征取代原来的不定长序列，这样所有样本的影像检查变量特征维度都是 3，而与检查次数无关，将这些特征补充到前一题的输入样本中，通过 XGBoost 模型训练和预测。根据多次实验（补充几个实验作为证据），我们发现以上几个特征能很好地表征患者脑血肿和水肿的发展趋势。预测结果对比如下所示：

表 6-3 评价指标

模型输入	正确率	邻近命中率	均方误差	R ²
个人史、疾病史、发病相关及首次影像结果	74%	85%	1.1572	0.4684
个人史、疾病史、发病相关及首次影像结果+随访结果	75%	90%	0.8779	0.6324

相比前一问的结果，加入了随访检查信息的模型预测的正确率虽然提升不大，但是近邻命中率提高了 5%，均方误差降低了 31.81%，R² 提高了 35%，也更接近于 1。可见，我们的模型改善效果显著。

6.3 问题三 c：模型的建立与求解

针对问题（c）分析出血性脑卒中患者的预后和个人史、疾病史、治疗方法及影像特征之间的关联关系，我们使用 spearman 相关性分析计算出出血性脑卒中患者的 90 天 mRS 评分与个人史、疾病史、治疗方法的相关性得分图，如下图 6-9 所示。

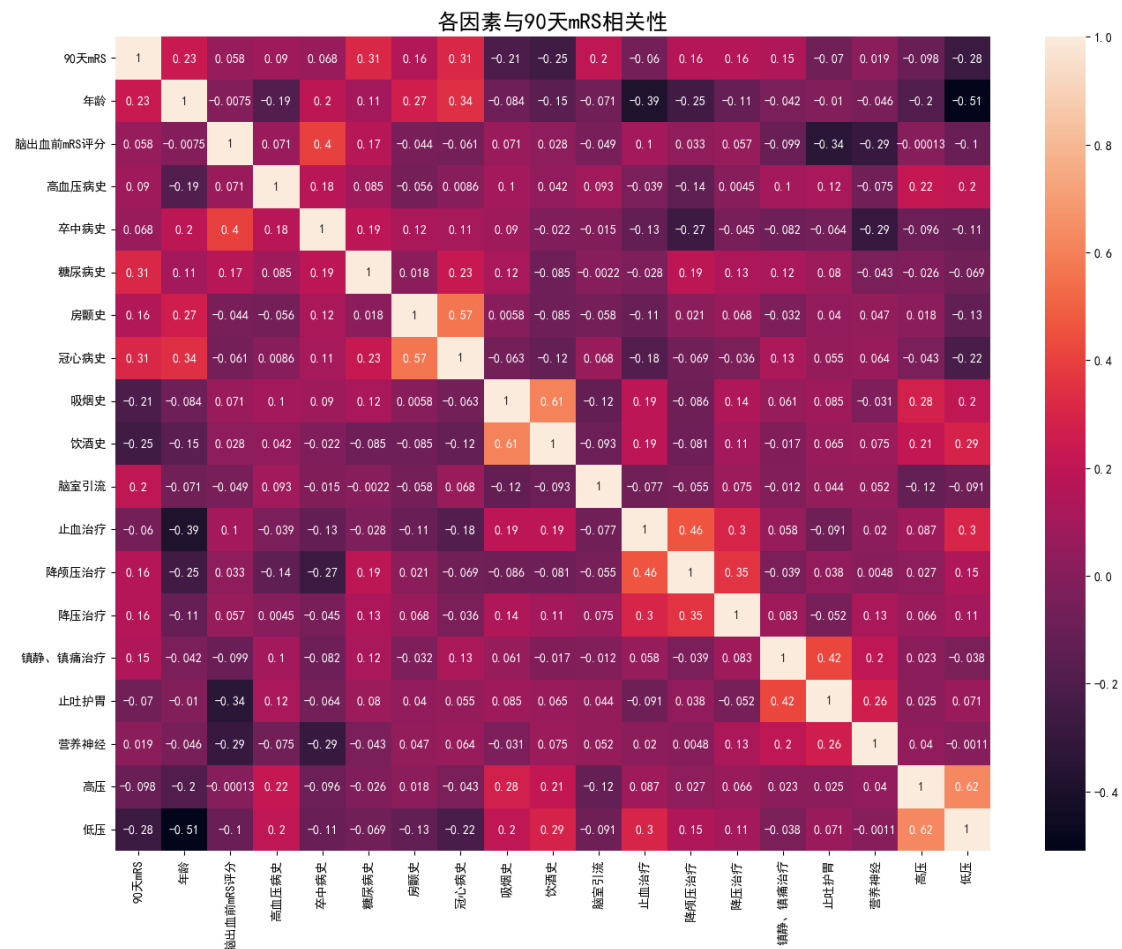


图 6-9 个体差异与 90 天 mRS 相关性分析

通过观察相关性分析图，我们可以知道患者个人信息中与 90 天 mRS 评分最相关的因素分别为糖尿病史、冠心病史、年龄等等。所以医生在治疗相关疾病时，要更多地关注这几个因素，给予糖尿病史、冠心病史、年龄更高的权重。而在治疗方式中以脑室引流的相关性最高，但是脑室引流属于手术范畴，所以一般不推荐使用此方法来提高患者预后水平，降颅压治疗、降压治疗、镇静、镇痛治疗相关性也比较高，而且其治疗成本不高，所以对于一般病人，采取降颅压治疗、降压治疗、镇静、镇痛治疗等治疗方法可以改善患者的预后情况，对于比较严重的病人，就要采取手术与修养相结合的方式，使用脑室引流手术，同时辅以降颅压治疗、降压治疗、镇静、镇痛治疗治疗。

患者预后评分与影像特征的相关性分析图如图 6-10 所示，从图中我们可以知道患者预后水平评分与血肿体积的相关性最大，说明血肿体积是患者 90 天 mRS 评分的重要因素。患者预后水平评分与水肿体积的相关性也很大，说明水肿对患者 90 天 mRS 评分的影响也很大。我们观察血肿位置与患者 90 天 mRS 评分的相关性，可以看出 HM_ACA_L_Ratio、HM_MCA_L_Ratio、HM_ACA_R_Ratio、HM_PCA_L_Ratio 的相关性都比较强，所以医生观察患者血肿影像特征时，需要重点关注大脑前动脉左侧、大脑中动脉左侧、大脑前动脉右侧、大脑后动脉左侧这四个位置信息。我们再观察水肿位置与患者 90 天 mRS 评分的相关性，可以看到 ED_ACA_L_Ratio 和 ED_PCA_L_Ratio 两个因素的相关性比较高，所以医生观察患者水肿影像特征时，需要重点关注大脑前动脉左侧、大脑后动脉左侧这两个位置信息。观察信号强度特征与患者 90 天 mRS 评分的相关性，我们可以看出 original_shape_LeastAxislengthd 的相关程度最高，其他信号强度特征相关性也很高，说明影像的信号强度特征非常影响影像质量，医生需要选择一个合适的信号强度以获取更多的

信息。观察形状特征与患者 90 天 mRS 评分的相关性，我们可以知道 NCCT_original_firstorder_Entropy 的相关性评分最高（负相关），所以在诊断过程中可以通过 NCCT_original_firstorder_Entropy 初步判断患者的正常水平。以上信息只实现了初步诊断，真正在临床中需要综合血肿体积、水肿体积、血肿位置、水肿位置、信号强度特征、形状特征等信息才能得到一个比较合理的决策。

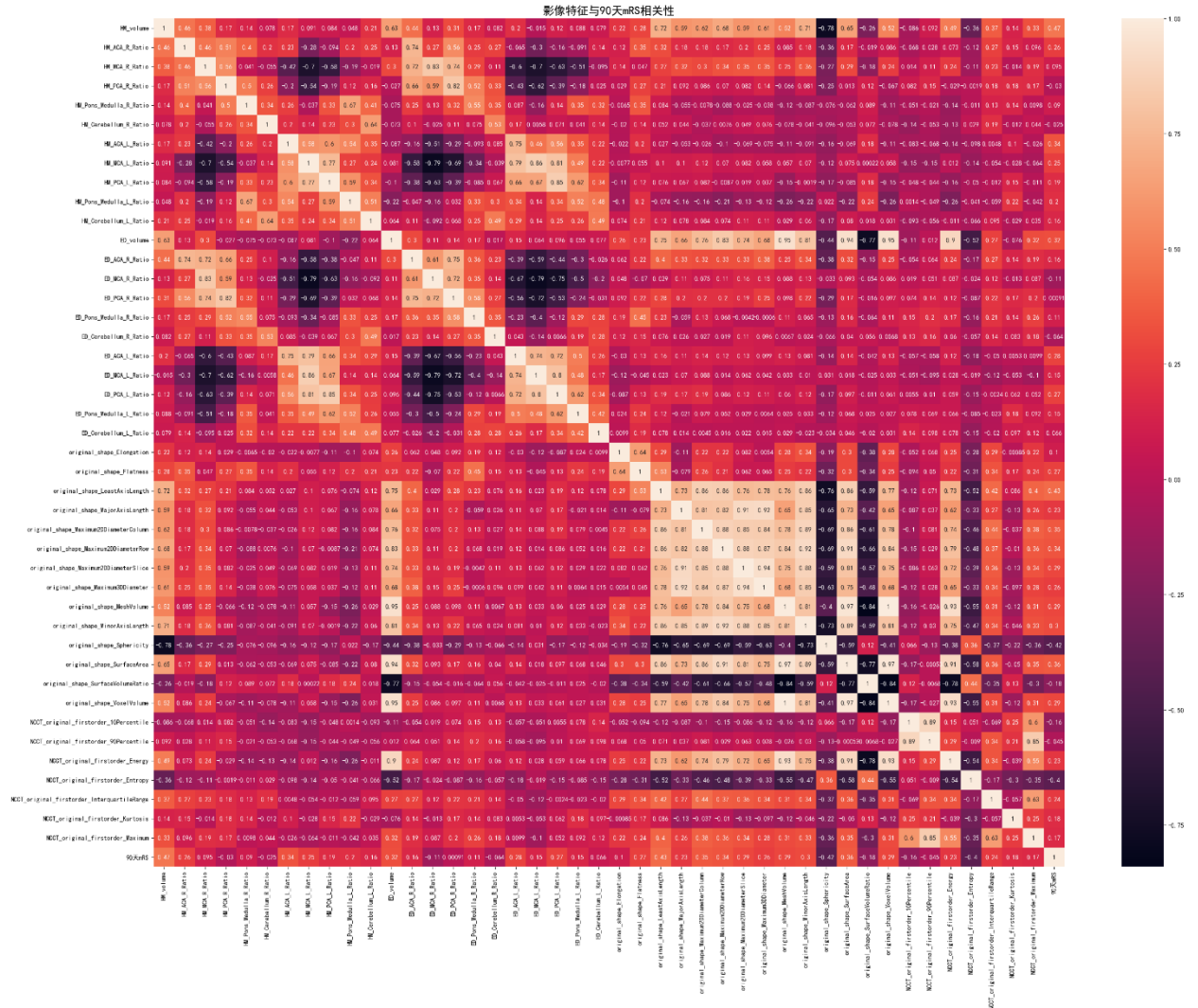


图 6-10 影像特征与 90 天 mRS 相关性分析

6.4 表 4-答案文件中问题三的计算结果

	首次影像检查流水号	问题 3：预后预测建模 预测 mRS（基于首次影像）	预测 mRS
sub001	20161212002136	4	4
sub002	20160406002131	0	0
sub003	20160413000006	5	5

sub004	20161215001667	4	4
sub005	20161222000978	3	3
sub006	20161110001074	5	5
sub007	20161208000139	2	2
sub008	20161219000091	4	4
sub009	20161031001987	3	3
sub010	20161012002008	3	3
sub011	20160209000219	1	1
sub012	20161031001142	0	0
sub013	20161124000397	1	1
sub014	20160513001799	2	2
sub015	20161013001234	2	2
sub016	20161130000004	5	5
sub017	20160510002436	3	3
sub018	20160602001707	0	0
sub019	20160117000135	1	1
sub020	20160723000013	2	2
sub021	20160317001244	3	3
sub022	20160803001239	2	2
sub023	20160321000142	1	1
sub024	20170802000637	1	1
sub025	20171226002293	2	2
sub026	20171008000512	3	3
sub027	20170206000071	6	6
sub028	20171013002097	4	4
sub029	20170607000010	4	4
sub030	20171025000480	5	5
sub031	20170307002130	4	4
sub032	20171009000137	0	0
sub033	20170115000362	5	5
sub034	20170119000729	4	4
sub035	20171014001244	5	5
sub036	20170204001714	3	3
sub037	20170426000005	3	3
sub038	20170518002194	2	2
sub039	20170425002487	1	1
sub040	20170902000876	1	1
sub041	20171002000282	3	3
sub042	20170420000636	0	0
sub043	20170325000428	5	5
sub044	20170528000084	3	3
sub045	20170324001892	1	1
sub046	20170511000016	2	2

sub047	20171019001652	1	1
sub048	20170402000556	3	3
sub049	20171005000770	1	1
sub050	20171105000372	1	1
sub051	20170422000935	0	0
sub052	20170608001310	3	3
sub053	20170511001392	2	2
sub054	20170612002216	3	3
sub055	20170316001977	4	4
sub056	20170120000152	0	0
sub057	20170825001844	3	3
sub058	20170125000984	5	5
sub059	20170912002314	0	0
sub060	20180109000613	4	4
sub061	20180226000725	4	4
sub062	20181221002264	2	2
sub063	20181020001229	2	2
sub064	20180801000501	3	3
sub065	20180131001727	3	3
sub066	20181208000909	6	6
sub067	20181207001317	2	2
sub068	20180412001426	2	2
sub069	20180619001505	6	5
sub070	20180427000292	3	3
sub071	20181103001264	2	3
sub072	20181007000826	3	2
sub073	20180911001645	5	2
sub074	20180719000020	4	2
sub075	20180428001767	4	1
sub076	20180619002401	3	1
sub077	20180503002304	2	4
sub078	20180929000040	2	2
sub079	20180929000037	3	2
sub080	20180130001917	2	3
sub081	20180120000249	2	4
sub082	20180221000793	2	4
sub083	20181004000706	3	2
sub084	20180716000006	2	2
sub085	20181127002511	3	3
sub086	20180108000002	2	4
sub087	20180216000198	2	2
sub088	20180521000314	3	4
sub089	20180314002318	2	3

sub090	20180910002366	3	2
sub091	20181019001130	2	2
sub092	20181116001089	5	4
sub093	20181214000208	1	2
sub094	20180412001795	2	3
sub095	20180316001329	2	2
sub096	20180802001789	3	2
sub097	20181010000767	3	2
sub098	20180612002507	4	5
sub099	20180620002296	2	3
sub100	20180314000010	1	2
sub101	20180311000432	3	
sub102	20180708000024	4	
sub103	20181015001677	2	
sub104	20190105000694	3	
sub105	20190108002459	0	
sub106	20190519000853	1	
sub107	20190526000209	3	
sub108	20190701002502	3	
sub109	20190716000013	2	
sub110	20190717001385	2	
sub111	20190727000556	3	
sub112	20190803000014	3	
sub113	20190901000442	0	
sub114	20190903000373	5	
sub115	20190904002299	3	
sub116	20190906001283	2	
sub117	20190917002094	2	
sub118	20190923002580	2	
sub119	20191003000352	2	
sub120	20191004001025	2	
sub121	20191027000468	2	
sub122	20191028002738	3	
sub123	20191024001280	4	
sub124	20191030001526	2	
sub125	20191111002510	3	
sub126	20191126002590	1	
sub127	20191127002176	3	
sub128	20191208000592	3	
sub129	20191224000008	3	
sub130	20191228000237	1	
sub131	20160413000006	1	1
sub132	20161215001667	1	4

sub133	20200112000228	2	4
sub134	20200101000392	3	2
sub135	20201130003288	3	2
sub136	20201203002778	2	3
sub137	20201217002368	2	2
sub138	20200403000012	1	2
sub139	20200814000015	3	4
sub140	20200412000331	2	2
sub141	20200711001264	2	3
sub142	20200411000014	4	3
sub143	20200214000572	3	3
sub144	20200124000041	4	5
sub145	20200613001086	2	1
sub146	20200531000253	0	1
sub147	20201220000155	1	2
sub148	20200609001016	2	2
sub149	20200409001101	2	2
sub150	20200118000372	3	2
sub151	20201023001238	3	3
sub152	20201109000009	2	3
sub153	20201212001420	4	3
sub154	20201129000299	2	3
sub155	20200301000025	4	2
sub156	20200306000927	2	1
sub157	20201009003102	2	2
sub158	20200410001952	2	2
sub159	20200218000582	3	3
sub160	20200821002584	3	2

七、总结与评价

7.1 模型优点

(1) 对于问题一，利用患者的个人史、疾病史、发病相关信息以及患者首次影像检查记录建立了随机森林特征提取模型以及随机森林预测模型。可以提供高度准确的与猜测，减少过拟合风险、对于高维度数据的处理能力较强。

(2) 对于问题二，利用患者的水肿体积和重复时间点，建立了多项式回归模型，其简单易理解，灵活度高。探索水肿体积随时间进展模型的个体差异，建立了基于 DTW 距离的 K-Means 聚类模型，其能有效捕获时间相关性，对噪声和变形具有鲁棒性。

(3) 对于问题三，利用患者的个人史、疾病史、发病相关信息以及患者首次影像检查记录建立了 Spearman 秩相关检验模型，其鲁棒性高，适用范围广。之后建立了基于 XGBoost 的 90 天 mRS 评分预测模型，其预测性能强，处理速度快，同时支持特征选择。

7.2 模型缺点

(1) 问题一建立的模型可能难以解释，而且对输入数据中的微小变化敏感，导致模型的不稳定性。

(2) 问题二建立的多项式回归模型需要特征工程，无法捕获数据的复杂性。基于 DTW 距离的 K-Means 聚类模型计算复杂度高，超参数的调整难。

(3) 问题三建立的 Spearman 秩相关检验模型可能会丧失一部分特征信息，对样本大小敏感。基于 XGBoost 的 90 天 mRS 评分预测模型调参花费时间长，过拟合的风险较大。

7.3 模型的改进与推广

(1) 问题一的数据需要更好的方法来进行过采样，这样才能提高模型的鲁棒性。特征提取可以综合多个特征提取模型综合得到相关性强的特征。

(2) 问题二单个聚类可能无法得到适用性强的模型，可以使用多个聚类模型综合评价得到的亚组更具普遍性。

(3) 问题三手动提取特征可以改成基于深度学习的无监督网络特征提取模型，如自编码器等等。

八、参考文献

- [1] Schirmer, M. D., A.-K. Giese, P. Fotiadis, M. R. Etherton, L. Cloonan, A. Viswanathan, S. M. Greenberg, O. Wu and N. S. Rost (2019). "Spatial Signature of White Matter Hyperintensities in Stroke Patients." *Frontiers in Neurology* 10.
- [2] 方匡南,吴见彬,朱建平,等.随机森林方法研究综述[J].统计与信息论坛, 2011, 26(3):7.DOI:10.3969/j.issn.1007-3116.2011.03.006.
- [3] 姚登举,杨静,詹晓娟.基于随机森林的特征选择算法[J].吉林大学学报:工学版, 2014(1):5.DOI:10.13229/j.cnki.jdxbgxb201401024.
- [4] Muda L , Begam M , Elamvazuthi I .Voice Recognition Algorithms using Mel Frequency Cepstral Coefficient (MFCC) and Dynamic Time Warping (DTW) Techniques[J].Ttps, 2010, 2.DOI:doi:Muda, Lindasalwa and K.M. Begam and Elamvazuthi, I, (2010).
- [5] 陈海燕,刘晨晖,孙博.时间序列数据挖掘的相似性度量综述[J].控制与决策, 2017, 32(1):11.DOI:10.13195/j.kzyjc.2016.0462.
- [6] 宋辞,裴韬.基于特征的时间序列聚类方法研究进展[J].地理科学进展, 2012, 31(10):11.DOI:CNKI:SUN:DLKJ.0.2012-10-009.
- [7] 张洪侠,李洪军,刘天戟,等.基于 XGBoost 算法的 2 型糖尿病精准预测模型研究[J].中国实验诊断学, 2018, 22(3):5.DOI:10.3969/j.issn.1007-4287.2018.03.008.
- [8] 胡雪梅,李佳丽,蒋慧凤.机器学习方法研究肝癌预测问题[M].[2023-09-26].
- [9] Sun C , Hong S , Song M ,et al.A Review of Deep Learning Methods for Irregularly Sampled Medical Time Series Data[J]. 2020.DOI:10.48550/arXiv.2010.12493.
- [10] Shickel B , Tighe P J , Bihorac A ,et al.Deep EHR: A Survey of Recent Advances in Deep Learning Techniques for Electronic Health Record (EHR) Analysis[J].IEEE journal of biomedical and health informatics, 2017:1589.DOI:10.1109/JBHI.2017.2767063.

九、附录

表 4-答案文件：

问题一：

		问题 1：血肿扩张		
	首次影像检查流水号	是否发生血肿扩张	血肿扩张时间	血肿扩张预测概率
		1 是, 0 否	单位：小时	
sub001	20161212002136	0		0.1220109
sub002	20160406002131	0		0.30747995
sub003	20160413000006	1	9.5225	0.4535491
sub004	20161215001667	0		0.04978754
sub005	20161222000978	1	26.4675	0.52868998
sub006	20161110001074	0		0.10381119
sub007	20161208000139	0		0.0991669
sub008	20161219000091	0		0.1695109
sub009	20161031001987	1	40.056111	0.59193501
sub010	20161012002008	0		0.15266935
sub011	20160209000219	0		0.13307093
sub012	20161031001142	0		0.08060181
sub013	20161124000397	0		0.12742454
sub014	20160513001799	0		0.08825217
sub015	20161013001234	0		0.26643406
sub016	20161130000004	0		0.1695109
sub017	20160510002436	1	14.870278	0.40747995
sub018	20160602001707	0		0.16485909
sub019	20160117000135	0		0.07312254
sub020	20160723000013	0		0.40571429
sub021	20160317001244	0		0.21693406
sub022	20160803001239	0		0.0648158
sub023	20160321000142	0		0.13757398
sub024	20170802000637	0		0.07134901
sub025	20171226002293	0		0.10726848
sub026	20171008000512	0		0.08060181
sub027	20170206000071	0		0.05886622
sub028	20171013002097	0		0.1440625
sub029	20170607000010	0		0.03192951
sub030	20171025000480	0		0.22619143

sub031	20170307002130	0		0.14454505
sub032	20171009000137	0		0.38591941
sub033	20170115000362	1	30.813056	0.96666667
sub034	20170119000729	0		0.0715109
sub035	20171014001244	0		0.02549647
sub036	20170204001714	1	39.501111	0.21971026
sub037	20170426000005	0		0.0935625
sub038	20170518002194	1	15.809167	0.1645805
sub039	20170425002487	1	29.176111	0.7252557
sub040	20170902000876	0		0.10248161
sub041	20171002000282	0		0.09311428
sub042	20170420000636	0		0.14600133
sub043	20170325000428	0		0.14893884
sub044	20170528000084	0		0.10265341
sub045	20170324001892	0		0.0787347
sub046	20170511000016	0		0.24332592
sub047	20171019001652	0		0.16094617
sub048	20170402000556	1	12.860833	0.27376335
sub049	20171005000770	0		0.0986321
sub050	20171105000372	0		0.0826145
sub051	20170422000935	0		0.11862634
sub052	20170608001310	0		0.05192951
sub053	20170511001392	0		0.0595109
sub054	20170612002216	1	16.225	0.54193501
sub055	20170316001977	0		0.16635361
sub056	20170120000152	0		0.1996879
sub057	20170825001844	1	14.615833	0.85
sub058	20170125000984	0		0.09999258
sub059	20170912002314	0		0.16628571
sub060	20180109000613	1	23.726111	0.15363515
sub061	20180226000725	1	6.5416667	0.5577529
sub062	20181221002264	0		0.04992043
sub063	20181020001229	0		0.0966321
sub064	20180801000501	0		0.00860175
sub065	20180131001727	0		0.03392951
sub066	20181208000909	0		0.07134901
sub067	20181207001317	0		0.09173868
sub068	20180412001426	0		0.0715109
sub069	20180619001505	0		0.15630975
sub070	20180427000292	1	9.6533333	0.28056676
sub071	20181103001264	0		0.29339071
sub072	20181007000826	0		0.17131363
sub073	20180911001645	0		0.10772301

sub074	20180719000020	0		0.11028542
sub075	20180428001767	0		0.10313515
sub076	20180619002401	1	15.993611	0.30079262
sub077	20180503002304	1	14.119444	0.43795614
sub078	20180929000040	0		0.39339071
sub079	20180929000037	1	27.847222	0.27605468
sub080	20180130001917	1	20.573333	0.36088147
sub081	20180120000249	1	27.421667	0.5527041
sub082	20180221000793	0		0.0715109
sub083	20181004000706	0		0.1069304
sub084	20180716000006	0		0.22425679
sub085	20181127002511	0		0.0935625
sub086	20180108000002	0		0.08567141
sub087	20180216000198	0		0.31728305
sub088	20180521000314	0		0.05598111
sub089	20180314002318	0		0.16056291
sub090	20180910002366	0		0.14645588
sub091	20181019001130	0		0.08567141
sub092	20181116001089	1	11.924722	0.4481039
sub093	20181214000208	0		0.09369539
sub094	20180412001795	0		0.14251069
sub095	20180316001329	1	7.4316667	0.52183911
sub096	20180802001789	0		0.08091781
sub097	20181010000767	0		0.14044836
sub098	20180612002507	1	42.756667	0.55739422
sub099	20180620002296	1	17.672778	0.53795614
sub100	20180314000010	0		0.05697
sub101	20180311000432	0		0.30248161
sub102	20180708000024	0		0.18339868
sub103	20181015001677	0		0.18066667
sub104	20190105000694	0		0.1695109
sub105	20190108002459	0		0.15829018
sub106	20190519000853	0		0.24284862
sub107	20190526000209	0		0.0915625
sub108	20190701002502	0		0.10326804
sub109	20190716000013	0		0.31890756
sub110	20190717001385	0		0.07324717
sub111	20190727000556	0		0.17736358
sub112	20190803000014	0		0.13757398
sub113	20190901000442	0		0.33915152
sub114	20190903000373	0		0.17719178
sub115	20190904002299	0		0.2442164
sub116	20190906001283	0		0.08325377

sub117	20190917002094	0		0.07567141
sub118	20190923002580	0		0.0595109
sub119	20191003000352	0		0.12450614
sub120	20191004001025	0		0.18098508
sub121	20191027000468	0		0.3071085
sub122	20191028002738	0		0.13961465
sub123	20191024001280	0		0.0201648
sub124	20191030001526	0		0.27452221
sub125	20191111002510	0		0.16141371
sub126	20191126002590	0		0.0735236
sub127	20191127002176	0		0.38081328
sub128	20191208000592	0		0.0826145
sub129	20191224000008	0		0.0795625
sub130	20191228000237	0		0.15630975
sub131	20160413000006	1	15.393611	0.47173092
sub132	20161215001667	0		0.55739422
sub133	20200112000228	0		0.1912619
sub134	20200101000392	0		0.1440625
sub135	20201130003288	0		0.36671889
sub136	20201203002778	1	19.375	0.17425679
sub137	20201217002368	0		0.1826145
sub138	20200403000012	0		0.26919927
sub139	20200814000015	1	13.839444	0.28043002
sub140	20200412000331	0		0.15822301
sub141	20200711001264	0		0.08060181
sub142	20200411000014	1	10.289167	0.0895109
sub143	20200214000572	0		0.24688739
sub144	20200124000041	0		0.20203891
sub145	20200613001086	0		0.04284862
sub146	20200531000253	0		0.15829018
sub147	20201220000155	1	8.8630556	0.0989951
sub148	20200609001016	0		0.0826145
sub149	20200409001101	0		0.26141371
sub150	20200118000372	0		0.06302042
sub151	20201023001238	1	8.2302778	0.43795614
sub152	20201109000009	0		0.2069304
sub153	20201212001420	0		0.25178219
sub154	20201129000299	0		0.17133911
sub155	20200301000025	0		0.24796661
sub156	20200306000927	0		0.16978754
sub157	20201009003102	1	1.3666667	0.24238109
sub158	20200410001952	0		0.0807347
sub159	20200218000582	1	26.533333	0.14489605

sub160	20200821002584	1	17.483333	0.10265341
--------	----------------	---	-----------	------------

问题二：

		问题 2：水肿体积进展曲线		
	首次影像检查流水号	残差（全体）	残差（亚组）	所属亚组
sub001	20161212002136	278542.3815	-1883.032312	2
sub002	20160406002131	6624.266163	232.2616199	1
sub003	20160413000006	58603.41005	-140.5892732	2
sub004	20161215001667	54614.6144	-529.7038263	3
sub005	20161222000978	44022.01546	-3493.826222	3
sub006	20161110001074	224687.7815	-13435.34192	3
sub007	20161208000139	66751.28228	-5118.794161	5
sub008	20161219000091	40825.85659	-1811.129529	5
sub009	20161031001987	24772.37565	-2200.277936	2
sub010	20161012002008	31260.96956	5656.797392	3
sub011	20160209000219	69589.52879	-266.1396616	3
sub012	20161031001142	67754.07016	23797.94002	1
sub013	20161124000397	90862.75774	-2203.473405	3
sub014	20160513001799	15544.04374	-35.75954992	2
sub015	20161013001234	117994.3241	-4980.473715	5
sub016	20161130000004	87191.32034	-145.1995876	2
sub017	20160510002436	19824.08012	103.7480954	5
sub018	20160602001707	70245.74342	1259.12515	2
sub019	20160117000135	49972.21905	5027.261639	3
sub020	20160723000013	63222.44909	28.34001286	3
sub021	20160317001244	38711.46246	213.2932346	5
sub022	20160803001239	55830.23723	-12.47385684	5
sub023	20160321000142	89515.73094	7539.298446	3
sub024	20170802000637	62492.52929	-2536.648896	3
sub025	20171226002293	17215.02515	-311.6647597	2
sub026	20171008000512	42930.40223	1362.884791	2
sub027	20170206000071	58632.15749	-17361.65557	1
sub028	20171013002097	76328.40381	10271.49462	1
sub029	20170607000010	76653.72308	-112.79269	3
sub030	20171025000480	194807.2466	-3391.139075	5
sub031	20170307002130	41073.59325	-592.4257497	2
sub032	20171009000137	70049.12319	-1400.473394	1
sub033	20170115000362	83145.63749	3107.616017	3
sub034	20170119000729	113117.8475	2654.166103	5

sub035	20171014001244	84254.78031	-3246.746011	3
sub036	20170204001714	64045.11331	79.12905363	4
sub037	20170426000005	75137.6679	-9585.531645	1
sub038	20170518002194	34662.8838	-762.4695191	5
sub039	20170425002487	240780.9385	24136.25926	1
sub040	20170902000876	117716.8984	3440.398533	5
sub041	20171002000282	68729.38996	-162.1062357	1
sub042	20170420000636	64993.78523	-1356.920854	3
sub043	20170325000428	20944.86007	-612.1758317	1
sub044	20170528000084	2711826.608	36188.35289	3
sub045	20170324001892	28383.60908	-1766.190749	2
sub046	20170511000016	42542.66692	-4031.114259	1
sub047	20171019001652	54116.21782	945.4371469	1
sub048	20170402000556	75339.24455	1269.996128	2
sub049	20171005000770	100585.0969	-398.8880091	3
sub050	20171105000372	41446.22657	-46.08717579	2
sub051	20170422000935	55222.23129	-616.7392801	3
sub052	20170608001310	117571.6434	24901.67851	3
sub053	20170511001392	31896.27964	-5573.355496	1
sub054	20170612002216	32678.3646	-184.8622398	2
sub055	20170316001977	32916.10372	-536.8513118	2
sub056	20170120000152	153192.185	1341.207334	3
sub057	20170825001844	52104.35205	-0.874126135	2
sub058	20170125000984	22244.59751	-161.7713105	5
sub059	20170912002314	74667.41885	-439.7477773	1
sub060	20180109000613	109304.2222	-2312.491726	3
sub061	20180226000725	212080.2623	4653.300137	2
sub062	20181221002264	31037.46995	-19.78410701	2
sub063	20181020001229	226303.5347	1836.948832	5
sub064	20180801000501	117752.447	-642.8545041	3
sub065	20180131001727	75636.46674	299.6285173	2
sub066	20181208000909	57937.57337	-66.78636443	3
sub067	20181207001317	53916.62789	-63.5556769	2
sub068	20180412001426	364634.9126	17055.64423	3
sub069	20180619001505	83017.4256	-7979.775348	3
sub070	20180427000292	43992.41651	-2.44116445	2
sub071	20181103001264	118799.7339	26645.52813	1
sub072	20181007000826	31024.18149	-43.30063132	5
sub073	20180911001645	57598.63998	-42.58618057	5
sub074	20180719000020	209216.5841	11080.7274	5
sub075	20180428001767	30974.80207	7601.417261	1
sub076	20180619002401	83951.38398	-1388.403964	1
sub077	20180503002304	45329.31307	3239.543534	5

sub078	20180929000040	73075.96154	-39.15645822	4
sub079	20180929000037	101541.9032	1723.88121	3
sub080	20180130001917	47305.43315	-437.0384659	3
sub081	20180120000249	5888499.395	5766.463821	2
sub082	20180221000793	24026.00899	-118.448319	4
sub083	20181004000706	34102.73486	-258.283136	2
sub084	20180716000006	36101.71183	659.3711063	2
sub085	20181127002511	157804.0196	-4500.868823	3
sub086	20180108000002	99790.14197	-174.6250041	5
sub087	20180216000198	102275.1282	1393.521447	1
sub088	20180521000314	14561.61565	580.7309307	4
sub089	20180314002318	56173.53613	-2435.464941	5
sub090	20180910002366	112622.6428	-19237.4754	3
sub091	20181019001130	69874.48002	197.6698939	5
sub092	20181116001089	67986.22021	-484.017776	2
sub093	20181214000208	95985.38961	1602.067713	1
sub094	20180412001795	59430.88321	5422.302109	3
sub095	20180316001329	363916.5825	-254.6324207	5
sub096	20180802001789	107855.7844	-2569.285846	3
sub097	20181010000767	57206.95535	-718.4862457	3
sub098	20180612002507	42916.99458	-8011.199657	1
sub099	20180620002296	19096.51905	-615.6387915	2
sub100	20180314000010	61802.65004	-325.5857449	3

问题三：

		问题 3：预后预测建模	
	首次影像检查流水号	预测 mRS（基于首次影像）	预测 mRS
sub001	20161212002136	4	4
sub002	20160406002131	0	0
sub003	20160413000006	5	5
sub004	20161215001667	4	4
sub005	20161222000978	3	3
sub006	20161110001074	5	5
sub007	20161208000139	2	2
sub008	20161219000091	4	4
sub009	20161031001987	3	3
sub010	20161012002008	3	3
sub011	20160209000219	1	1

sub012	20161031001142	0	0
sub013	20161124000397	1	1
sub014	20160513001799	2	2
sub015	20161013001234	2	2
sub016	20161130000004	5	5
sub017	20160510002436	3	3
sub018	20160602001707	0	0
sub019	20160117000135	1	1
sub020	20160723000013	2	2
sub021	20160317001244	3	3
sub022	20160803001239	2	2
sub023	20160321000142	1	1
sub024	20170802000637	1	1
sub025	20171226002293	2	2
sub026	20171008000512	3	3
sub027	20170206000071	6	6
sub028	20171013002097	4	4
sub029	20170607000010	4	4
sub030	20171025000480	5	5
sub031	20170307002130	4	4
sub032	20171009000137	0	0
sub033	20170115000362	5	5
sub034	20170119000729	4	4
sub035	20171014001244	5	5
sub036	20170204001714	3	3
sub037	20170426000005	3	3
sub038	20170518002194	2	2
sub039	20170425002487	1	1
sub040	20170902000876	1	1
sub041	20171002000282	3	3
sub042	20170420000636	0	0
sub043	20170325000428	5	5
sub044	20170528000084	3	3
sub045	20170324001892	1	1
sub046	20170511000016	2	2
sub047	20171019001652	1	1
sub048	20170402000556	3	3
sub049	20171005000770	1	1
sub050	20171105000372	1	1
sub051	20170422000935	0	0
sub052	20170608001310	3	3
sub053	20170511001392	2	2
sub054	20170612002216	3	3

sub055	20170316001977	4	4
sub056	20170120000152	0	0
sub057	20170825001844	3	3
sub058	20170125000984	5	5
sub059	20170912002314	0	0
sub060	20180109000613	4	4
sub061	20180226000725	4	4
sub062	20181221002264	2	2
sub063	20181020001229	2	2
sub064	20180801000501	3	3
sub065	20180131001727	3	3
sub066	20181208000909	6	6
sub067	20181207001317	2	2
sub068	20180412001426	2	2
sub069	20180619001505	6	5
sub070	20180427000292	3	3
sub071	20181103001264	2	3
sub072	20181007000826	3	2
sub073	20180911001645	5	2
sub074	20180719000020	4	2
sub075	20180428001767	4	1
sub076	20180619002401	3	1
sub077	20180503002304	2	4
sub078	20180929000040	2	2
sub079	20180929000037	3	2
sub080	20180130001917	2	3
sub081	20180120000249	2	4
sub082	20180221000793	2	4
sub083	20181004000706	3	2
sub084	20180716000006	2	2
sub085	20181127002511	3	3
sub086	20180108000002	2	4
sub087	20180216000198	2	2
sub088	20180521000314	3	4
sub089	20180314002318	2	3
sub090	20180910002366	3	2
sub091	20181019001130	2	2
sub092	20181116001089	5	4
sub093	20181214000208	1	2
sub094	20180412001795	2	3
sub095	20180316001329	2	2
sub096	20180802001789	3	2
sub097	20181010000767	3	2

sub098	20180612002507	4	5
sub099	20180620002296	2	3
sub100	20180314000010	1	2
sub101	20180311000432	3	
sub102	20180708000024	4	
sub103	20181015001677	2	
sub104	20190105000694	3	
sub105	20190108002459	0	
sub106	20190519000853	1	
sub107	20190526000209	3	
sub108	20190701002502	3	
sub109	20190716000013	2	
sub110	20190717001385	2	
sub111	20190727000556	3	
sub112	20190803000014	3	
sub113	20190901000442	0	
sub114	20190903000373	5	
sub115	20190904002299	3	
sub116	20190906001283	2	
sub117	20190917002094	2	
sub118	20190923002580	2	
sub119	20191003000352	2	
sub120	20191004001025	2	
sub121	20191027000468	2	
sub122	20191028002738	3	
sub123	20191024001280	4	
sub124	20191030001526	2	
sub125	20191111002510	3	
sub126	20191126002590	1	
sub127	20191127002176	3	
sub128	20191208000592	3	
sub129	20191224000008	3	
sub130	20191228000237	1	
sub131	20160413000006	1	1
sub132	20161215001667	1	4
sub133	20200112000228	2	4
sub134	20200101000392	3	2
sub135	20201130003288	3	2
sub136	20201203002778	2	3
sub137	20201217002368	2	2
sub138	20200403000012	1	2
sub139	20200814000015	3	4
sub140	20200412000331	2	2

sub141	20200711001264	2	3
sub142	20200411000014	4	3
sub143	20200214000572	3	3
sub144	20200124000041	4	5
sub145	20200613001086	2	1
sub146	20200531000253	0	1
sub147	20201220000155	1	2
sub148	20200609001016	2	2
sub149	20200409001101	2	2
sub150	20200118000372	3	2
sub151	20201023001238	3	3
sub152	20201109000009	2	3
sub153	20201212001420	4	3
sub154	20201129000299	2	3
sub155	20200301000025	4	2
sub156	20200306000927	2	1
sub157	20201009003102	2	2
sub158	20200410001952	2	2
sub159	20200218000582	3	3
sub160	20200821002584	3	2

源代码:

问题 1 (a) :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
table1 = pd.read_excel('data\表 1-患者列表及临床信息.xlsx',index_col=0)
table2 = pd.read_excel('data\表 2-患者影像信息血肿及水肿的体积及位置.xlsx',index_col=0)
appendix3 = pd.read_excel('data\附表 1-检索表格-流水号 vs 时间.xlsx',index_col=0)
column_names = ['发病到首次影像检查时间间隔']
df1 = table1.loc[:,column_names]
deltatime0=(appendix3['随访 1 时间点']-appendix3['入院首次检查时间点']).dt.total_seconds() / 3600
deltatime1=(appendix3['随访 2 时间点']-appendix3['入院首次检查时间点']).dt.total_seconds() / 3600
deltatime2=(appendix3['随访 3 时间点']-appendix3['入院首次检查时间点']).dt.total_seconds() / 3600
deltatime3=(appendix3['随访 4 时间点']-appendix3['入院首次检查时间点']).dt.total_seconds() / 3600
```

```

deltatime4=(appendix3['随访 5 时间点']-appendix3['入院首次检查时间点
']).dt.total_seconds() / 3600
deltatime5=(appendix3['随访 6 时间点']-appendix3['入院首次检查时间点
']).dt.total_seconds() / 3600
deltatime6=(appendix3['随访 7 时间点']-appendix3['入院首次检查时间点
']).dt.total_seconds() / 3600
deltatime7=(appendix3['随访 8 时间点']-appendix3['入院首次检查时间点
']).dt.total_seconds() / 3600
deltatime8=(appendix3['随访 9 时间点']-appendix3['入院首次检查时间点
']).dt.total_seconds() / 3600
df3 =
pd.concat([deltatime0,deltatime1,deltatime2,deltatime3,deltatime4,deltatime5,delt
atime6,deltatime7,deltatime8], axis=1)
df3
def hm_enlargement1(x):
    if x is np.nan:
        return 0
    if x > 6000:
        return 1
    return 0
def hm_enlargement2(x):
    if x is np.nan:
        return 0
    if x > 0.33:
        return 1
    return 0
result_df = pd.DataFrame()
time_df = pd.DataFrame()
for i in range(1,9):

    delta_hmvol = hmvol.iloc[:,i]-hmvol.iloc[:,0]
    series1 = delta_hmvol.apply(hm_enlargement1)
    delta_hmvol_percent = delta_hmvol/hmvol.iloc[:,0]
    series2 = delta_hmvol_percent.apply(hm_enlargement2)
    tempdf = series1 | series2
    result_df['y_{i}'].format(i)] = tempdf
    time_df['dt_{i}'].format(i)] = tempdf * (df3.iloc[:,i-1]+df1.iloc[:,0])

#     time_df['{i}'].format(i)] = time_df['{i}'].format(i)].replace(0, np.nan)

time_df = time_df.replace(0.00, np.nan)
time_df[:16]
time_min = time_df.min(axis=1)
result_df_max = result_df.max(axis=1)

```

```

answer = pd.concat([result_df_max,time_min],axis=1)
answer.columns = ['是否发生血肿扩张','血肿扩张时间']
answer[:16]
# answer['血肿扩张时间'] = answer['血肿扩张时间'].replace(0, np.nan)
answer.loc[answer['血肿扩张时间'] > 48, '是否发生血肿扩张'] = 0
answer.loc[answer['血肿扩张时间'] > 48, '血肿扩张时间'] = np.nan
answer.to_excel('问题 1 答案-表 4.xlsx',sheet_name = "ans")
answer[:16]

```

```

import os
output_folder = 'output_images'
if not os.path.exists(output_folder):
    os.makedirs(output_folder)
for i in range(160):
    x_values = df3.iloc[i, :]

    y_values = hmvol.iloc[i, :]

    plt.scatter(x_values, y_values)

    plt.xlabel('time(h)')
    plt.ylabel('HM_volume(10e-3 ml)')
    plt.title('Scatter plot')
    filename = os.path.join(output_folder, f'scatter_plot_{i}.png')
    plt.savefig(filename)
    plt.show()

```

问题 1（b）：

```

import numpy as np

import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier

import pandas as pd

import openpyxl

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import MinMaxScaler

# 假设 X 包含输入特征，y 包含输出标签

excel_data2 = pd.read_excel('问题 1b_input.xlsx')

```



```

# 获取指定列范围的数据
# selected_data = excel_data2.iloc[1:100, 1:85]
selected_data = excel_data2.iloc[0:100, 1:85]

X = selected_data
X_test1 = excel_data2.iloc[0:160, 1:85]
excel_data3 = pd.read_excel('问题 1 答案.xlsx')
# 获取指定列范围的数据
y = excel_data3.iloc[0:100, 1]
y_test = excel_data3.iloc[0:160, 1]

# 假设 X 包含输入特征, y 包含输出标签
num_iterations=10
#缺失数据的行
volume = 130
# 存储每次迭代的特征重要性得分
feature_importance_list = []
for iteration in range(num_iterations):
    # 创建随机森林分类器模型
    rf_classifier = RandomForestClassifier(n_estimators=100)

    # 训练模型
    rf_classifier.fit(X, y)

    # 获取特征重要性得分
    feature_importance = rf_classifier.feature_importances_
    min_score = min(feature_importance)
    max_score = max(feature_importance)
    normalized_importance = [(score - min_score) / (max_score - min_score) for score in

```

```
feature_importance]
```

```
# 存储特征重要性得分
```

```
feature_importance_list.append(normalized_importance)
```

```
# 可视化每个特征在多次迭代中的重要性得分
```

```
# 计算平均特征重要性得分
```

```
average_feature_importance = np.mean(feature_importance_list, axis=0)
```

```
# 选择平均得分最高的 10 个特征
```

```
top_13_indices = np.argsort(average_feature_importance)[-20:]
```

```
top_13_features = X.columns[top_13_indices]
```

```
print(top_13_features)
```

```
top_13_scores = average_feature_importance[top_13_indices]
```

```
Data = X_test1[top_13_features]
```

```
# 检查 X_test1 中是否存在空值
```

```
has_missing_values = Data.isnull().any()
```

```
# 获取包含空值的列的索引
```

```
columns_with_missing_values = has_missing_values[has_missing_values].index
```

```
# 删除包含空值的列
```

```
Data_cleaned = Data.drop(columns=columns_with_missing_values)
```

```
scaler = MinMaxScaler()
```

```
oneData = scaler.fit_transform(Data_cleaned)
```

```
def find_min_squared_difference(X_test1, count):
```

```

min_squared_sum = float('inf') # 初始化为正无穷大的值
min_row_index = -1
min_Id = 0
squared_diff_list = []
cou = []
esm = []
for i in range(len(X_test1)):
    squared_diff = 0
    if i == count:
        continue # 跳过第 130 行
    for j in range(len(X_test1[0])):
        # 计算第 130 行与其他行的平方和
        squared_diff += np.sum((X_test1[count][j] - X_test1[i][j]) ** 2)
    squared_diff_list.append([i,squared_diff])
    print([i,squared_diff])
    cou.append(i)
    esm.append(squared_diff)
min_entry = min(squared_diff_list,key=lambda x:x[1])
min_Id,min_squared_diff= min_entry
return min_squared_diff,min_Id,cou,esm

```

调用函数

```

min_squared_sum, min_row_index,cou,esm= find_min_squared_difference(oneData,130)
print(f'最小平方和： {min_squared_sum}')
print(f'所在行数： {min_row_index}')
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.plot(cou,esm,marker='o',linestyle='-',color='b',label='误差曲线')
plt.title('其他患者与 128 号患者的误差曲线')

```

```
plt.xlabel('患者号码')
plt.ylabel('误差大小')
plt.legend()
plt.show()
```

```
## 创建柱状图
# plt.figure(figsize=(12, 6))
# plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
# plt.bar(top_13_features, top_13_scores)
# plt.xlabel('特征名称', fontweight='bold')
# plt.title('平均得分最高的 10 个特征', fontweight='bold')
#
## 显示柱状图
# plt.xticks(rotation=45)
# plt.tight_layout()
# plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
from imblearn.over_sampling import ADASYN
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
from xgboost import XGBClassifier, XGBRegressor
import openpyxl
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.preprocessing import OneHotEncoder
```

```

# 假设 X 包含输入特征，y 包含输出标签
excel_data2 = pd.read_excel('问题 1b_input.xlsx')
# 获取指定列范围的数据
# selected_data = excel_data2.iloc[1:100, 1:85]
# X = excel_data2.iloc[0:100, 1:85]
x_continue=          excel_data2.iloc[0:100,          1:2].join(excel_data2.iloc[0:100,
10:11]).join(excel_data2.iloc[0:100, 20:85])
x_categorical = excel_data2.iloc[0:100, 2:10].join(excel_data2.iloc[0:100, 11:20])

x_continue_test=          excel_data2.iloc[0:160,          1:2].join(excel_data2.iloc[0:160,
10:11]).join(excel_data2.iloc[0:160, 20:85])
x_categorical_test = excel_data2.iloc[0:160, 2:10].join(excel_data2.iloc[0:160, 11:20])

excel_data3 = pd.read_excel('问题 1 答案.xlsx')
# 获取指定列范围的数据
y = excel_data3.iloc[0:100, 1]
y_test = excel_data3.iloc[0:160, 1]

# 假设 X 包含输入特征，y 包含输出标签

# 创建 StandardScaler 或 MinMaxScaler 对象
scaler = StandardScaler() # 或 scaler = MinMaxScaler()

# 对连续变量进行归一化
XG_train_data_X_continuous = scaler.fit_transform(x_continue)

```

```
XG_test_data_X_continuous = scaler.fit_transform(x_continue_test)
```

```
X_categorical_encoded = pd.get_dummies(x_categorical, columns=x_categorical.columns,  
drop_first=True)
```

```
X_categorical_encoded_test = pd.get_dummies(x_categorical_test,  
columns=x_categorical.columns, drop_first=True)
```

```
X = pd.concat([pd.DataFrame(XG_train_data_X_continuous,  
columns=x_continue.columns), X_categorical_encoded], axis=1)
```

```
X_test = pd.concat([pd.DataFrame(XG_test_data_X_continuous,  
columns=x_continue_test.columns), X_categorical_encoded_test], axis=1)
```

```
adasyn = ADASYN(sampling_strategy='auto')
```

```
X_resampled, y_resampled = adasyn.fit_resample(X, y)
```

```
matching_rows_list = []
```

```
start_value = 0.2
```

```
end_value = 0.8
```

```
step = 0.1
```

```
# 存储每次迭代的特征重要性得分
```

```
feature_importance_list = []
```

```
for count in range(1,80,5):
```

```
    for num_iterations in range(40,200,20):
```

```
        for Tvalue in np.arange(start_value, end_value + step, step):
```

```
            for iteration in range(num_iterations):
```

```

# 创建随机森林分类器模型
rf_classifier = RandomForestClassifier(n_estimators=100)

# 训练模型
rf_classifier.fit(X_resampled, y_resampled)

# 获取特征重要性得分
feature_importance = rf_classifier.feature_importances_
min_score = min(feature_importance)
max_score = max(feature_importance)
normalized_importance = [(score - min_score) / (max_score - min_score)
for score in feature_importance]

# 存储特征重要性得分
feature_importance_list.append(normalized_importance)

# 可视化每个特征在多次迭代中的重要性得分
# 计算平均特征重要性得分
average_feature_importance = np.mean(feature_importance_list, axis=0)
# 选择平均得分最高的 10 个特征
top_13_indices = np.argsort(average_feature_importance)[-count:]
top_13_features = X_resampled.columns[top_13_indices]
top_13_scores = average_feature_importance[top_13_indices]

## 创建柱状图
# plt.figure(figsize=(12, 6))
# plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
# plt.bar(top_13_features, top_13_scores)
# plt.xlabel('特征名称', fontweight='bold')

```

```

# plt.title('平均得分最高的 10 个特征', fontweight='bold')
#
# # 显示柱状图
# plt.xticks(rotation=45)
# plt.tight_layout()
# plt.show()

# 使用选定的特征训练一个新的随机森林模型
X_train_NEW = X_resampled[top_13_features]
X_test_NEW = X_test[top_13_features]
# X_train = X
# X_test = X_test1

# 创建随机森林分类器
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train_NEW, y_resampled)

# 预测概率
y_probabilities = rf_classifier.predict_proba(X_test_NEW)[: , 1]

# 比对正确个数
y_predictions = np.where(y_probabilities > 0.5, 1, 0)
# 假设 X_test1 和 y_predictions 是你的测试数据和预测结果

# 找到值相同的行
matching_rows = np.where(y_predictions == y_test)[0]

matching_rows_list.append([count,num_ iterations,Tvalue,len(matching_rows)])
print([count, num_ iterations, Tvalue, len(matching_rows)])

```



```

# matching_rows 包含了值相同的行的索引
max_matching_rows_entry = max(matching_rows_list, key=lambda x: x[3])
count, num_iterations, Tvalue, matching_rows_len = max_matching_rows_entry

print(f'count: {count}')
print(f'num_iterations: {num_iterations}')
print(f'Tvalue: {Tvalue}')
print(f'matching_rows: {matching_rows_len}')

```

问题 2 (a) :

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
data_2_a = pd.read_csv('data_2_a.csv')
data_2_a = data_2_a.fillna(0)
# print(data_2_a)
# 示例时间序列数据集
new_xy=[]
for index, row in data_2_a.iterrows():
    new_xy.append((row[1],row[3]))
    for j in range(len(row)):
        if(j%2==0 and j>2 and row[j]!=0 and row[j+1]!=0):
            # print (row[j]==np.nan)
            new_xy.append((row[j],row[j+1]))
# print(new_xy)

x_values = [x for x, _ in new_xy]
y_values = [y for _, y in new_xy]
max_x_value = max(x_values)
max_y_value = max(y_values)
x_threshold = max_x_value * 0.5
y_threshold = max_y_value * 0.7
new_xy = [(x, y) for x, y in new_xy if x <= x_threshold and y <= y_threshold]

x_values, y_values = zip(*new_xy)

```

```

plt.figure(figsize=(8, 6))

lues, y_values, marker='o', color='blue', label='Scatter Points')

#
x_values = np.array(x_values).reshape(-1, 1)
poly_degree = 4
poly_features = PolynomialFeatures(degree=poly_degree)
x_poly = poly_features.fit_transform(x_values)
model = LinearRegression()
model.fit(x_poly, y_values)
print("多项式系数: ", model.coef_.tolist())
print("常数项: ", model.intercept_)
x_pred = np.linspace(min(x_values), max(x_values), 100).reshape(-1, 1)
x_pred_poly = poly_features.transform(x_pred)
y_pred = model.predict(x_pred_poly)

plt.plot(x_pred, y_pred, color='red', linewidth=2, label='Polynomial Regression
Line (Degree {})'.format(poly_degree))

plt.title('Scatter Plot with Polynomial Regression')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.show()

def function_e(xx):
    x_to_predict = np.array([[xx]])
    x_to_predict_poly = poly_features.transform(x_to_predict)
    predicted_temperature = model.predict(x_to_predict_poly)
    return predicted_temperature[0]
e=[]
for index, row in data_2_a.iterrows():
    ee=abs(row[3] - function_e(row[1]))
    for j in range(len(row)):
        if(j%2==0 and j>2 and row[j]!=0 and row[j+1]!=0):
            ee=ee+abs(function_e(row[j]) - row[j+1])
    e.append(ee)
print("多项式系数: ", model.coef_.tolist())
print("常数项: ", model.intercept_)
# print(e)
dff=pd.DataFrame(e)

```

```
dff.to_csv('data_2_a_e.csv', index=False, encoding='utf-8')
```

问题二（b）：

```
import numpy as np
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
import pandas as pd
data_2_a = pd.read_csv('data_2_a.csv')
data_2_a = data_2_a.fillna(10)

import matplotlib as mpl
mpl.rcParams["font.family"] = "SimHei"
mpl.rcParams["axes.unicode_minus"]=False
plt.rcParams['font.sans-serif']=['SimHei']

time_series_data = []
for index, row in data_2_a.iterrows():

    new_xy=[(row[1],row[3])]
    for j in range(len(row)):
        if(j%2==0 and j>2 and row[j]!=10 and row[j+1]!=10):
            new_xy.append((row[j],row[j+1]))
    # print(new_xy)
    # new_xy=[(row[1],row[3]/row[3])]
    # for j in range(len(row)):
    #     if(j%2==0 and j>2 and row[j]!=10 and row[j+1]!=10):
    #         new_xy.append((row[j],(row[j+1])/row[3]))
    # print(new_xy)

x_values, y_values = zip(*new_xy)

x_mean = np.mean(x_values)
x_std = np.std(x_values)
y_mean = np.mean(y_values)
y_std = np.std(y_values)
```

```

x_normalized = [(x - x_mean) / x_std for x in x_values]
y_normalized = [(y - y_mean) / y_std for y in y_values]

new_xy = list(zip(x_normalized, y_normalized))

```

```

# # 提取 x 和 y 坐标值
# x_values, y_values = zip(*new_xy)
# # 使用最小-最大归一化将 x 和 y 归一化到[0, 1]范围
# x_min = min(x_values)
# x_max = max(x_values)
# y_min = min(y_values)
# y_max = max(y_values)
# x_normalized = [(x - x_min) / (x_max - x_min) for x in x_values]
# y_normalized = [(y - y_min) / (y_max - y_min) for y in y_values]
# # 归一化后的数据在 x_normalized 和 y_normalized 中
# new_xy = list(zip(x_normalized, y_normalized))
# # 归一化结束%%
# # print(new_xy)

```

```

time_series_data.append(np.array(new_xy, dtype=float))

```

```

desired_length = 8

```

```

interpolated_data = []

```

```

for original_data in time_series_data:
    x = np.array([point[0] for point in original_data])
    y = np.array([point[1] for point in original_data])

    f = interp1d(x, y, kind='linear', fill_value="extrapolate")

    new_x = np.linspace(min(x), max(x), desired_length)

    new_y = f(new_x)

    interpolated_data.append(list(zip(new_x, new_y)))

```

```

for new_xy in interpolated_data:

```

```

x_values, y_values = zip(*new_xy)

x_mean = np.mean(x_values)
x_std = np.std(x_values)
y_mean = np.mean(y_values)
y_std = np.std(y_values)

x_normalized = [(x - x_mean) / x_std for x in x_values]
y_normalized = [(y - y_mean) / y_std for y in y_values]

new_xy = list(zip(x_normalized, y_normalized))

# x_values, y_values = zip(*new_xy)
# x_min = min(x_values)
# x_max = max(x_values)
# y_min = min(y_values)
# y_max = max(y_values)
# x_normalized = [(x - x_min) / (x_max - x_min) for x in x_values]
# y_normalized = [(y - y_min) / (y_max - y_min) for y in y_values]
# # 归一化后的数据在 x_normalized 和 y_normalized 中
# new_xy = list(zip(x_normalized, y_normalized))
# # print(new_xy)

distance_matrix = np.zeros((len(interpolated_data), len(interpolated_data)))
for i in range(len(interpolated_data)):
    for j in range(i + 1, len(interpolated_data)):
        distance, _ = fastdtw(interpolated_data[i], interpolated_data[j],)
        distance_matrix[i][j] = distance
        distance_matrix[j][i] = distance
n_clusters = 5
kmeans = KMeans(n_clusters=n_clusters, random_state=0).fit(distance_matrix)

# plt.figure(figsize=(10, 5))
# cluster_labels = kmeans.labels_
# for i in range(len(time_series_data)):
#     print(f"时间序列{i + 1} 属于簇 {cluster_labels[i]}")
# for cluster_num in range(n_clusters):
#     cluster_indices = np.where(kmeans.labels_ == cluster_num)[0]
#     print(f"簇 {cluster_num} 中的时间序列: {cluster_indices}")
#     # 创建一个子图
#     plt.subplot(1, n_clusters, cluster_num + 1)

```

```

#     for idx in cluster_indices:
#         plt.plot(time_series_data[idx][:, 0], time_series_data[idx][:, 1],
label=f'Time Series {idx + 1}')
#     plt.title(f'Cluster {cluster_num}')
#     plt.xlabel('X-axis')
#     plt.ylabel('Y-axis')
#     # plt.legend()
# plt.tight_layout()
# plt.show()
# # print(cluster_labels)
# # dff=pd.DataFrame(cluster_labels)
# # dff.to_csv('data_2_c00.csv', index=False, encoding='utf-8')

```

```

n_clusters = len(np.unique(kmeans.labels_))
for cluster_num in range(n_clusters):
    cluster_indices = np.where(kmeans.labels_ == cluster_num)[0]
    print(f"簇 {cluster_num} 中的时间序列: {cluster_indices}")
    plt.figure(figsize=(10, 5))
    for idx in cluster_indices:
        plt.plot(time_series_data[idx][:, 0], time_series_data[idx][:, 1],
label=f'Time Series {idx + 1}')
        plt.title(f'标准化后得到的亚组{cluster_num+1}')
        plt.xlabel('标准化后的 X')
        plt.ylabel('标准化后的 Y')
        plt.legend()
        plt.show()

```

拟合:

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression

```

```

from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso

import matplotlib as mpl
mpl.rcParams["font.family"] = "SimHei"
mpl.rcParams["axes.unicode_minus"]=False
plt.rcParams['font.sans-serif']=['SimHei']

data_2_b = pd.read_csv('data_2_b_拟合.csv',encoding='ISO-8859-1')
data_2_b = data_2_b.fillna(10)
# new_xy=[[ ],[[ ],[[ ],[[ ],[[ ]]]
# for
def function_f_x(x,model,poly_features):
    x_to_predict = np.array([[x]])

    x_to_predict_poly = poly_features.transform(x_to_predict)

    y_predicted = model.predict(x_to_predict_poly)
    print("多项式系数: ",model.coef_.tolist())
    print("常数项: ",model.intercept_)
    print(f" x=f({x}),y=: ", y_predicted[0])
    return y_predicted[0]

def bzh(row,index):
    x_values, y_values = zip(*row)

    x_mean = np.mean(x_values)
    # print(x_mean)
    # if(x_mean==data_2_b.iloc[index,21]):
    #     print(111)
    #     print(x_mean)
    x_std = np.std(x_values)
    y_mean = np.mean(y_values)
    y_std = np.std(y_values)
    x_normalized = [(x - x_mean) / x_std for x in x_values]
    y_normalized = [(y - y_mean) / y_std for y in y_values]
    row = list(zip(x_normalized, y_normalized))
    return row

time_series_data = []
for index, row in data_2_b.iterrows():

```

```

new_xy=[(row[1],row[3])]
for j in range(len(row)):
    if(j%2==0 and j>2 and j<20 and row[j]!=10 and row[j+1]!=10):
        new_xy.append((row[j],row[j+1]))
time_series_data.append(new_xy)
data_after_bzh=[]

for row,index in zip(time_series_data,range(len(time_series_data))):
    row=bzh(row,index)
    data_after_bzh.append(row)

xy_5=[[[]],[[]],[[]],[[]]]
for row,index in zip(data_after_bzh,range(len(data_after_bzh))):
    for i in row:
        xy_5[data_2_b.iloc[index,20]].append(i)

yazu=4
x_values, y_values = zip(*xy_5[yazu])
plt.figure(figsize=(8, 6))
plt.scatter(x_values, y_values, marker='o', color='blue', label=f'亚组{yazu+1}标准化后的散点')
x_values = np.array(x_values).reshape(-1, 1)
poly_degree = 3
poly_features = PolynomialFeatures(degree=poly_degree)
x_poly = poly_features.fit_transform(x_values)

# alpha = 0.043
# # model = Ridge(alpha=alpha)
# model = Lasso(alpha=alpha) # 使用 Lasso 模型
# model.fit(x_poly, y_values)

```



```

# x_pred = np.linspace(min(x_values), max(x_values), 100).reshape(-1, 1)
# x_pred_poly = poly_features.transform(x_pred)
# y_pred = model.predict(x_pred_poly)
# print("多项式系数: ",model.coef_.tolist())
# print("常数项: ",model.intercept_)
# plt.plot(x_pred, y_pred, color='red', linewidth=2, label=f'亚组{yazu+1}标准化后的拟合曲线 Y=f(X)')
# plt.title(f'标准化后亚组{yazu+1}的拟合曲线')
# plt.xlabel('标准化后的 X')
# plt.ylabel('标准化后的 Y')
# plt.legend()
# plt.show()
# # x=function_f_x(5,model,poly_features)

```

```

model = LinearRegression()
model.fit(x_poly, y_values)
x_pred = np.linspace(min(x_values), max(x_values), 100).reshape(-1, 1)
x_pred_poly = poly_features.transform(x_pred)
y_pred = model.predict(x_pred_poly)
print("多项式系数: ",model.coef_.tolist())
print("常数项: ",model.intercept_)
plt.plot(x_pred, y_pred, color='red', linewidth=2, label=f'亚组{yazu+1}标准化后的拟合曲线 Y=f(X)')
plt.title(f'标准化后亚组{yazu+1}的拟合曲线')
plt.xlabel('标准化后的 X')
plt.ylabel('标准化后的 Y')
plt.legend()
plt.show()
# x=function_f_x(5,model,poly_features)

```

问题二（c）：

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
table1 = pd.read_excel('data\表 1-患者列表及临床信息.xlsx',index_col=0)
table2 = pd.read_csv('data\data_2_c.csv',encoding='gb2312',index_col=0)
# table2 = pd.read_excel('data\data_2_c.xlsx',sheet_name = 'data_2_c')

```

```

# table3 = pd.read_excel('data\表 3-患者影像信息血肿及水肿的形状及灰度分
布.xlsx', index_col=1)
table2 = table2[0:100]
t1 = table1.iloc[:100, 15:24]
remedydf = t1.join(table2['预估'])
remedydf['预估'] = remedydf['预估'].astype(int)
# remedydf[:15]
columns_to_plot = remedydf.columns[:-1]

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签

for column in columns_to_plot:
    # plt.figure(figsize=(8, 4)) # 可选, 设置图形大小
    sns.countplot(x=remedydf.columns[-1], hue =column, data=remedydf)
    plt.xlabel('发展模式')
    plt.ylabel(column)
    plt.title(f'{column} vs. 发展模式')
    filename = f'histplot_{column}.png'
    plt.savefig(filename)
    plt.show()

remedydf['前期发展'] = remedydf['预估'].replace({0: 1, 1: 1, 2: 3, 3: 2, 4: 1})
remedydf['中期发展'] = remedydf['预估'].replace({0: 3, 1: 2, 2: 2, 3: 1, 4: 3})
remedydf['后期发展'] = remedydf['预估'].replace({0: 0, 1: 3, 2: 1, 3: 3, 4: 2})
clusterdf = remedydf.drop(columns=['预估'])
clusterdf[:16]

from scipy.stats import chi2_contingency

df = clusterdf
for column in df.columns[:-3]:
    contingency_table = pd.crosstab(remedydf[column], remedydf['后期发展'])

    chi2, p, dof, expected = chi2_contingency(contingency_table)

    print(f"——{column}——")
    print(f"卡方值 (Chi-squared): {chi2}")
    print(f"p 值 (p-value): {p}")
    print(f"自由度 (Degrees of Freedom): {dof}")
    print("期望频数 (Expected Frequencies):")
    print(expected)
    print('_____')
cdf = clusterdf.iloc[:, :7]

```

```

corr_matrix = cdf.corr(method='pearson')

plt.figure(figsize=(12,9))
sns.heatmap(corr_matrix,annot=True)
plt.title('各种治疗方法的相关性', fontsize=18)
plt.show()

correlation = clusterdf.corr()['中期发展']

# 打印相关性结果
print(correlation)
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols

# 创建示例 DataFrame

df = clusterdf
df = df.rename(columns={'镇静、镇痛治疗': '镇静镇痛治疗'})
# 创建一个适用于 Cochran-Armitage 趋势检验的模型
for col in df.columns[:-3]:
    model = ols(f'{col}~ 前期发展', data=df).fit()

    # 进行 Cochran-Armitage 趋势检验
    trend_test = sm.stats.linear_rainbow(model)

    # 打印趋势检验结果
    print(f"_____ {col} _____")
    print("Cochran-Armitage 趋势检验结果: ")
    print(trend_test)

print(len(df))
for col in df.columns[:-3]:
    model = ols(f'{col}~ 中期发展', data=df).fit()

    trend_test = sm.stats.linear_rainbow(model)

    print(f"_____ {col} _____")
    print("Cochran-Armitage 趋势检验结果: ")
    print(trend_test)
for col in df.columns[:-3]:
    model = ols(f'{col}~ 后期发展', data=df).fit()

```

```

# 进行 Cochran-Armitage 趋势检验
trend_test = sm.stats.linear_rainbow(model)

# 打印趋势检验结果
print(f"_____ {col} _____")
print("Cochran-Armitage 趋势检验结果: ")
print(trend_test)

```

问题二（d）：

```

import scipy.stats as stats
import pandas as pd
import numpy as np

data_2_d = pd.read_csv('data_2_d 水肿.csv')
# data_2_d = pd.read_csv('data_2_d 血肿.csv')
data_2_d = data_2_d.fillna(10)
print(data_2_d)
anova_group_0_1 = [[],[]]
method=20 #20-26
# for index, row in data_2_d.iterrows():
#     new_y=[row[3]]
#     for j in range(len(row)):
#         if(j%2==0 and j>2 and j<20 and row[j]!=10 and row[j+1]!=10):
#             new_y.append(row[j+1])
#     # print(f"{data_2_d.columns[method]}:",data_2_d.iloc[index,method])
#     print("最大值: ",max(new_y))
#     print("平均值: ",np.mean(new_y))
#     # print(new_y)
#     anova_group_0_1[data_2_d.iloc[index,method]].append(np.mean(new_y))
#     # anova_group_0_1[data_2_d.iloc[index,method]].append(max(new_y))
#     # anova_group_0_1[0].append()

# print(anova_group_0_1[1])
# print(anova_group_0_1[0])
# # print(data_2_d.columns[method])

for i in range(7):
    anova_group_0_1 = [[],[]]
    # method=26 #20-26
    for index, row in data_2_d.iterrows():
        new_y=[row[3]]
        for j in range(len(row)):
            if(j%2==0 and j>2 and j<20 and row[j]!=10 and row[j+1]!=10):

```

```

        new_y.append(row[j+1])
    # print(f"{data_2_d.columns[method]}:",data_2_d.iloc[index,method])
    # print("最大值: ",max(new_y))
    # print("均值: ",np.mean(new_y))
    # print("方差: ",np.var(new_y))
    # print("斜度: ",stats.skew(new_y))
    # print(new_y)
    anova_group_0_1[data_2_d.iloc[index,method+i]].append(np.max(new_y))
    # anova_group_0_1[data_2_d.iloc[index,method+i]].append(np.mean(new_y))
    # anova_group_0_1[data_2_d.iloc[index,method+i]].append(np.var(new_y))
    #
anova_group_0_1[data_2_d.iloc[index,method+i]].append(stats.skew(new_y))
    # anova_group_0_1[0].append()

    # print(anova_group_0_1[1])
    # print(anova_group_0_1[0])
    # print(data_2_d.columns[method])
    # f_statistic, p_value = stats.f_oneway(anova_group_0_1[0],
anova_group_0_1[1])
    f_statistic, p_value = stats.mannwhitneyu(anova_group_0_1[0],
anova_group_0_1[1], alternative='two-sided')
    # f_statistic, p_value = stats.spearmanr(anova_group_0_1[0],
anova_group_0_1[1])

    # print(f"{data_2_d.columns[method+i]}的统计量:")
    # print("F 统计量:", f_statistic)
    # print("P 值:", p_value)

alpha = 0.05
print()
if p_value < alpha:
    # print("存在显著影响")
    # print(f"{data_2_d.columns[method+i]}的统计量:")
    # print("F 统计量:", f_statistic)
    # print("P 值:", p_value)
    print(f"\033[91m{data_2_d.columns[method+i]}的统计量:\033[0m")
    print("\033[91mP 值:", "{:.4f}".format(p_value),"\033[0m")
    print("\033[91mF 统计量:", "{:.4f}".format(f_statistic),"\033[0m")
    print("\033[91m存在显著影响\033[0m")

else:
    print(f"{data_2_d.columns[method+i]}的统计量:")

```

```

    print("P 值:", "{:.4f}".format(p_value))
    print("F 统计量:", "{:.4f}".format(f_statistic))
    print("无显著影响")
    print(f"{data_2_d.columns[method+i]}0 的数量:", len(anova_group_0_1[0]), "\t1 的
数量:", len(anova_group_0_1[1]))
    for i in [0,1]:
        statistic, p = stats.shapiro(anova_group_0_1[i])
        alpha = 0.05
        if p > alpha:
            print(f"样本{i}满足正态分布")
        else:
            print(f"样本{i}不满足正态分布")

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
excel_file_path = 'data\分析水肿血肿的关系.csv'
df = pd.read_csv(excel_file_path, encoding='gb2312')

column_names = df.columns[[0,1]]
print(df.head(5))
for column_name in column_names:
    plt.figure(figsize=(8, 6))
    sns.histplot(df[column_name], kde=True, element='bars')
    plt.title(f'Distribution of {column_name}')
    plt.xlabel(column_name)
    plt.ylabel('Frequency')
    plt.show()
sns.jointplot(data=df, x='水肿体积', y='血肿体积', kind='reg')
# plt.xlabel(column_name)
# plt.ylabel('Frequency')
# plt.show()

import numpy as np
import statsmodels.api as sm
import statsmodels.robust as robust
x_data = df['水肿体积']
y_data = df['血肿体积']

# 使用鲁棒回归方法（Huber 回归）拟合曲线

```

```

model = robust.robust_linear_model.HuberRegressor()
model.fit(x_data.values.reshape(-1, 1), y_data)

# 获取拟合参数
intercept = model.intercept_
slope = model.coef_[0]

# 生成拟合曲线上的点
x_fit = x_data
y_fit = intercept + slope * x_data

# 绘制原始数据点和拟合曲线
import matplotlib.pyplot as plt

plt.scatter(x_data, y_data, label='原始数据')
plt.plot(x_fit, y_fit, label='拟合曲线', color='red')
plt.xlabel('X 轴')
plt.ylabel('Y 轴')
plt.legend()
plt.title('鲁棒回归拟合曲线')

# 显示图形
plt.show()

```

问题三（a）：

```

import scipy.stats as stats

from scipy.interpolate import griddata

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import StandardScaler, MinMaxScaler

from sklearn.svm import SVR

from xgboost import XGBClassifier, XGBRegressor

from sklearn.utils import resample

import pandas as pd

import numpy as np

```

```

import matplotlib.pyplot as plt

excel_data2 = pd.read_excel('问题 1b_input.xlsx')

# selected_data = excel_data2.iloc[1:100, 1:85]

# X = excel_data2.iloc[0:100, 1:85]

x_continue= excel_data2.iloc[0:70, 1:3].join(excel_data2.iloc[0:70,
10:11]).join(excel_data2.iloc[0:70, 20:75])

x_categorical = excel_data2.iloc[0:70, 3:10].join(excel_data2.iloc[0:70,
11:20])


x_continue_test= excel_data2.iloc[70:100, 1:3].join(excel_data2.iloc[70:100,
10:11]).join(excel_data2.iloc[70:100, 20:75])

x_categorical_test = excel_data2.iloc[70:100,
3:10].join(excel_data2.iloc[70:100, 11:20])


x_continue_test160= excel_data2.iloc[0:160, 1:3].join(excel_data2.iloc[0:160,
10:11]).join(excel_data2.iloc[0:160, 20:75])

x_categorical_test160 = excel_data2.iloc[0:160,
3:10].join(excel_data2.iloc[0:160, 11:20])


x_continue_test100= excel_data2.iloc[0:100, 1:3].join(excel_data2.iloc[0:100,
10:11]).join(excel_data2.iloc[0:100, 20:75])

x_categorical_test100 = excel_data2.iloc[0:100,
3:10].join(excel_data2.iloc[0:100, 11:20])


excel_data3 = pd.read_excel('表 1-患者列表及临床信息.xlsx')

y = excel_data3.iloc[0:70, 1]

```



```

y_test = excel_data3.iloc[70:100, 1]

y_test100 = excel_data3.iloc[0:100, 1]

y_test160 = excel_data3.iloc[0:160, 1]

scaler = MinMaxScaler()

XG_train_data_X_continuous = pd.DataFrame(scaler.fit_transform(x_continue),
columns=x_continue.columns)

XG_test_data_X_continuous = pd.DataFrame(
    scaler.fit_transform(x_continue_test),
    columns=x_continue_test.columns,
    index=x_continue_test.index
)

XG_test_data_X_continuous100 = pd.DataFrame(
    scaler.fit_transform(x_continue_test100),
    columns=x_continue_test100.columns,
    index=x_continue_test100.index
)

XG_test_data_X_continuous160 = pd.DataFrame(
    scaler.fit_transform(x_continue_test160),
    columns=x_continue_test160.columns,
    index=x_continue_test160.index
)

X_test160 = pd.concat([pd.DataFrame(XG_test_data_X_continuous160,
columns=XG_test_data_X_continuous160.columns), x_categorical_test160], axis=1)

X = X_test160.iloc[0:70, : ]

```

```

X_test = X_test160.iloc[70:100, : ]

X_test100 = X_test160.iloc[0:100, : ]

X_resampled, y_resampled = resample(X, y, n_samples=len(X))

X_resampled100, y_resampled100 = resample(X_test100, y_test100,
n_samples=len(X_test100))

spearman_corr_matrix, p_value = stats.spearmanr(X_resampled,y_resampled )

correlations = []

p_values = []

x_clo = []

for i, column in enumerate(X_resampled.columns):

    rho, p = spearman_corr_matrix[i, -1], p_value[i,-1]

    if(p>0.05):

        correlations.append(rho)

        p_values.append(p)

        x_clo.append(column)


start = 0.01

end = 0.2

step = 0.01

mse_list = []

results = pd.DataFrame({'自变量': x_clo, 'Spearman 相关系数': correlations, 'p
值': p_values})

print(len(x_clo))

print(x_clo)

# for n_estimators in range(10,200,10):

#     for learning_rate in np.arange(start, end + step, step):

```

```

#         XGdataX = X_resampled[x_clo]

#         XGdataY = y_resampled

#         model =
XGBRegressor(n_estimators=n_estimators,learning_rate=learning_rate)

#         model.fit(XGdataX, XGdataY)

#         y_pred = model.predict(X_test[x_clo])

#         y_pred_rounded = [round(value) for value in y_pred]

#         mse = mean_squared_error(y_pred_rounded, y_test)

#         mse_list.append([n_estimators,learning_rate,mse])

#         print([n_estimators,learning_rate,mse])

#

#

#

# max_matching_rows_entry = min(mse_list, key=lambda x: x[2])

# l_n_estimators,l_learning_rate,l_mse = max_matching_rows_entry

# print(max_matching_rows_entry)

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False


# x = [row[0] for row in mse_list]

# y = [row[1] for row in mse_list]

# z = [row[2] for row in mse_list]

# xi = np.linspace(min(x), max(x))

# yi = np.linspace(min(y), max(y))

# xi, yi = np.meshgrid(xi, yi)

#

```

```

# zi = griddata((x, y), z, (xi, yi), method='cubic')

#

# fig = plt.figure()

# ax = fig.add_subplot(111,projection='3d')

# surf = ax.plot_surface(xi, yi, zi, cmap='BuPu', linewidth=0,
antialiased=False)

# fig.colorbar(surf)

# ax.set_title('均方误差随弱分类器数量与学习率的变化趋势')

# ax.set_xlabel('n_estimators')

# ax.set_ylabel('learning_rate')

#

#

# plt.show()


#使用模型

XGdataX = X[x_clo]

XGdataY = y

# model = svm_regressor = SVR(kernel='rbf', C=10, epsilon=0.8)

model = XGBRegressor(n_estimators=25,learning_rate=0.17)

model.fit(XGdataX, XGdataY)

y_pred = model.predict(X_test160[x_clo])

y_pred_rounded = [round(value) for value in y_pred]

# mse_start = mean_squared_error(y_pred, y_test160)

# rmse = np.sqrt(mse_start)

# r_squared = r2_score(y_test160, y_pred)

```

```

# r_squared_std = rmse / np.std(y_test160)

# print("均方误差: ",mse_start)

# print("均方误差标准差: ",rmse)

# print("R-square 误差: ",r_squared)

# print("R-square 标准差: ",r_squared_std)

print(y_pred_rounded)

common_rows_count = 0

common_rows_count_top2 = 0

common_rows_count_top3 = 0

list1=y_test100.tolist()

for i in range(0,160,1):

    if(list1[i] == y_pred_rounded[i]):

        common_rows_count += 1

    if(abs(list1[i]-y_pred_rounded[i]) <= 1):

        common_rows_count_top2 += 1

    if (abs(list1[i]- y_pred_rounded[i]) <= 2):

        common_rows_count_top3 += 1

#

#

print(f"相同的行数为: {common_rows_count}")

print(f"top2 正确率: {common_rows_count_top2}")

print(f"top3 正确率: {common_rows_count_top3}")

print(f"准确率为: {common_rows_count/100}")

# # 交叉验证

```

```

# model_data = []

# for n_estimators in range(10,100,5):

#     for learning_rate in np.arange(start, end + step, step):

#         model = XGBRegressor(objective='reg:squarederror',
n_estimators=n_estimators,learning_rate=learning_rate)

#

#         mse_scores = cross_val_score(model, X_resampled100[x_clo],
y_resampled100, cv=5, scoring='neg_mean_squared_error')

#         r2_scores = cross_val_score(model,X_resampled100[x_clo],
y_resampled100,cv=5, scoring='r2')

#

#         mean_mse = -mse_scores.mean()

#         std_mse = mse_scores.std()

#         mean_r2 = r2_scores.mean()

#         std_r2 = r2_scores.std()

#

#

#         print([n_estimators,learning_rate,mean_mse,std_mse,mean_r2,std_r2])

#

model_data.append([n_estimators,learning_rate,mean_mse,std_mse,mean_r2,std_r2])

#

# min_matching_rows_entry = min(model_data, key=lambda x: x[2])

# print(min_matching_rows_entry)

# SVM 交叉验证

# Cstart = 0.1

# Cend =10

```

```

# Cstep = 0.1

# estart = 0.01

# eend = 0.2

# estep = 0.01

# model_data = []

# for C in np.arange(Cstart, Cend + Cstep, Cstep):

#     for epsilon in np.arange(estart, eend + estep, estep):

#         svm_regressor = SVR(kernel='rbf',C=C, epsilon=epsilon)

#         scores = cross_val_score(svm_regressor, X, y, cv=5,
scoring='neg_mean_squared_error')

#         mean_mse = -scores.mean()

#         std_mse = scores.std()

#         print([C,epsilon,mean_mse,std_mse])

#         model_data.append([C,epsilon,mean_mse,std_mse])

# min_matching_rows_entry = min(model_data, key=lambda x: x[2])

# print(min_matching_rows_entry)

#

#

# #

# for i in range(2,6,1):

#

#     x = [row[0] for row in model_data]

#     y = [row[1] for row in model_data]

#     z = [row[i] for row in model_data]

#     xi = np.linspace(min(x), max(x))

#     yi = np.linspace(min(y), max(y))

```

```

#     xi, yi = np.meshgrid(xi, yi)

#     zi = griddata((x, y), z, (xi, yi), method='cubic')

#

#     fig = plt.figure()

#     ax = fig.add_subplot(111,projection='3d')

#     surf = ax.plot_surface(xi, yi, zi, cmap='BuPu', linewidth=0,
antialiased=False)

#     fig.colorbar(surf)

#     if(i == 2):

#         ax.set_title('均方误差随弱分类器数量与学习率的变化趋势')

#     if(i == 3):

#         ax.set_title('均方误差标准差随弱分类器数量与学习率的变化趋势')

#     if(i == 4):

#         ax.set_title('R-squared 均值随弱分类器数量与学习率的变化趋势')

#     if (i == 5):

#         ax.set_title('R-squared 标准差随弱分类器数量与学习率的变化趋势')

#     ax.set_xlabel('n_estimators')

#     ax.set_ylabel('learning_rate')

#

#

#     plt.show()

```

问题三（b）：

```

import scipy.stats as stats

from scipy.interpolate import griddata

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.model_selection import cross_val_score

```



```

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.svm import SVR
from xgboost import XGBClassifier, XGBRegressor
from sklearn.utils import resample

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

excel_data = pd.read_excel('问题 1b_input.xlsx', usecols='B:V')

csv_data = pd.read_csv('T3_b.csv')

merged_data = pd.concat([excel_data, csv_data], axis=1)

excel_data2 = pd.read_excel('问题 1b_input.xlsx')
# selected_data = excel_data2.iloc[1:100, 1:85]
# X = excel_data2.iloc[0:100, 1:85]
x_continue=          excel_data2.iloc[0:70,          1:3].join(excel_data2.iloc[0:70,
10:11]).join(excel_data2.iloc[0:70, 20:75])
x_categorical = excel_data2.iloc[0:70, 3:10].join(excel_data2.iloc[0:70, 11:20])

x_continue_test=          excel_data2.iloc[70:100,          1:3].join(excel_data2.iloc[70:100,
10:11]).join(excel_data2.iloc[70:100, 20:75])
x_categorical_test = excel_data2.iloc[70:100, 3:10].join(excel_data2.iloc[70:100, 11:20])

x_continue_test160=          excel_data2.iloc[0:160,          1:3].join(excel_data2.iloc[0:160,
10:11]).join(excel_data2.iloc[0:160, 20:75])

```

```

x_categorical_test160 = excel_data2.iloc[0:160, 3:10].join(excel_data2.iloc[0:160, 11:20])

x_continue_test100= excel_data2.iloc[0:100, 1:3].join(excel_data2.iloc[0:100,
10:11]).join(excel_data2.iloc[0:100, 20:75])
x_categorical_test100 = excel_data2.iloc[0:100, 3:10].join(excel_data2.iloc[0:100, 11:20])


excel_data3 = pd.read_excel('表 1-患者列表及临床信息.xlsx')
# 获取指定列范围的数据
y = excel_data3.iloc[0:70, 1]
y_test = excel_data3.iloc[70:100, 1]
y_test100 = excel_data3.iloc[0:100, 1]
y_test30 = excel_data3.iloc[130:160, 1]
y_test160 = excel_data3.iloc[0:160, 1]

scaler = MinMaxScaler()

XG_train_data_X_continuous = pd.DataFrame(scaler.fit_transform(x_continue),
columns=x_continue.columns)
XG_test_data_X_continuous = pd.DataFrame(
    scaler.fit_transform(x_continue_test),
    columns=x_continue_test.columns,
    index=x_continue_test.index
)
XG_test_data_X_continuous100 = pd.DataFrame(
    scaler.fit_transform(x_continue_test100),
    columns=x_continue_test100.columns,
    index=x_continue_test100.index

```

```

)
XG_test_data_X_continuous160 = pd.DataFrame(
    scaler.fit_transform(x_continue_test160),
    columns=x_continue_test160.columns,
    index=x_continue_test160.index
)

merged_data_scaler = pd.DataFrame(
    scaler.fit_transform(merged_data),
    columns=merged_data.columns,
    index=merged_data.index
)

X_test160 = pd.concat([pd.DataFrame(XG_test_data_X_continuous160,
columns=XG_test_data_X_continuous160.columns), x_categorical_test160], axis=1)

merged_data_scaler_train1 = merged_data_scaler.iloc[0:70,
2:7 ].join(merged_data_scaler.iloc[0:70, 9:19 ]).join(merged_data_scaler.iloc[0:70, 21: ])

merged_data_scaler_test2 = merged_data_scaler.iloc[70:100,
2:7 ].join(merged_data_scaler.iloc[70:100, 9:19 ]).join(merged_data_scaler.iloc[70:100,21: ])

merged_data_scaler_test1001 = merged_data_scaler.iloc[0:100,
2:7 ].join(merged_data_scaler.iloc[0:100, 9:19 ]).join(merged_data_scaler.iloc[0:100,21: ])

merged_data_scaler_train = merged_data_scaler.iloc[0:70, : ]
merged_data_scaler_test = merged_data_scaler.iloc[70:100, : ]
merged_data_scaler_test100 = merged_data_scaler.iloc[0:100, : ]
merged_data_scaler_test30 = merged_data_scaler.iloc[130:160, : ]

```

```

X = X_test160.iloc[0:70, :]
X_test = X_test160.iloc[70:100, :]
X_test100 = X_test160.iloc[0:100, :]
X_test30 = X_test160.iloc[130:160, :]
X_resampled, y_resampled = resample(X, y, n_samples=len(X))
X_resampled100, y_resampled100 = resample(X_test100, y_test100,
n_samples=len(X_test100))

merged_data_scaler_train_resampled, y_train_resampled =
resample(merged_data_scaler_train, y, n_samples=len(merged_data_scaler_train))
# 计算多元 Spearman 秩相关系数矩阵
spearman_corr_matrix, p_value = stats.spearmanr(merged_data_scaler_test100, y_test100)

correlations = []
p_values = []
x_clo = []
for i, column in enumerate(merged_data_scaler_test100.columns):
    rho, p = spearman_corr_matrix[i, -1], p_value[i, -1]
    if(p>0.05):
        correlations.append(rho)
        p_values.append(p)
        x_clo.append(column)

#
# start = 0.01
# end = 0.2
# step = 0.01
# mse_list = []
# results = pd.DataFrame({'自变量': x_clo, 'Spearman 相关系数': correlations, 'p 值':
p_values})

```

```

# print(len(x_clo))
# print(x_clo)
# for n_estimators in range(10,200,10):
#     for learning_rate in np.arange(start, end + step, step):
#         XGdataX = X_resampled[x_clo]
#         XGdataY = y_resampled
#
#                                     model =
XGBRegressor(n_estimators=n_estimators,learning_rate=learning_rate)
#         model.fit(XGdataX, XGdataY)
#         y_pred = model.predict(X_test[x_clo])
#
#         y_pred_rounded = [round(value) for value in y_pred]
#         mse = mean_squared_error(y_pred_rounded, y_test)
#         mse_list.append([n_estimators,learning_rate,mse])
#         print([n_estimators,learning_rate,mse])
#
#
#
#
# max_matching_rows_entry = min(mse_list, key=lambda x: x[2])
# l_n_estimators,l_learning_rate,l_mse = max_matching_rows_entry
# print(max_matching_rows_entry)
plt.rcParams['font.sans-serif'] = ['SimSun']
plt.rcParams['axes.unicode_minus'] = False

# x = [row[0] for row in mse_list]
# y = [row[1] for row in mse_list]
# z = [row[2] for row in mse_list]
## 创建 xi 和 yi 的网格坐标

```

```

# xi = np.linspace(min(x), max(x))
# yi = np.linspace(min(y), max(y))
# xi, yi = np.meshgrid(xi, yi)
#
# zi = griddata((x, y), z, (xi, yi), method='cubic')
#
# fig = plt.figure()
# ax = fig.add_subplot(111, projection='3d')
# surf = ax.plot_surface(xi, yi, zi, cmap='BuPu', linewidth=0, antialiased=False)
# fig.colorbar(surf)
# ax.set_title('均方误差随弱分类器数量与学习率的变化趋势')
# ax.set_xlabel('n_estimators')
# ax.set_ylabel('learning_rate')
#
#
# plt.show()

```

```

XGdataX = merged_data_scaler_train[x_clo]
XGdataY = y
# model = svm_regressor = SVR(kernel='rbf', C=10, epsilon=0.8)
model = XGBRegressor(n_estimators=15, learning_rate=0.23)
model.fit(XGdataX, XGdataY)
y_pred = model.predict(merged_data_scaler_test30[x_clo])
y_pred_rounded = [round(value) for value in y_pred]
# mse_start = mean_squared_error(y_pred, y_test100)
# rmse = np.sqrt(mse_start)
# r_squared = r2_score(y_test100, y_pred)
# r_squared_std = rmse / np.std(y_test100)

```

```

# print("均方误差: ",mse_start)
# print("均方误差标准差: ",rmse)
# print("R-square 误差: ",r_squared)
# print("R-square 标准差: ",r_squared_std)

print(y_pred_rounded)
# common_rows_count = 0
# common_rows_count_top2 = 0
# common_rows_count_top3 = 0
# list1=y_test100.tolist()
# # 使用集合（set）来找到相同的值
# for i in range(0,100,1):
#     if(list1[i] == y_pred_rounded[i]):
#         common_rows_count += 1
#     if(abs(list1[i]-y_pred_rounded[i]) <= 1):
#         common_rows_count_top2 += 1
#     if (abs(list1[i]- y_pred_rounded[i]) <= 2):
#         common_rows_count_top3 += 1
#
# print(f"相同的行数为: {common_rows_count}")
# print(f"top2 正确率: {common_rows_count_top2}")
# print(f"top3 正确率: {common_rows_count_top3}")
# print(f"准确率为: {common_rows_count/100}")

# start = 0.01
# end = 0.5
# step = 0.01
# model_data = []

```

```

# for n_estimators in range(10,100,5):
#     for learning_rate in np.arange(start, end + step, step):
#         model = XGBRegressor(objective='reg:squarederror',
n_estimators=n_estimators,learning_rate=learning_rate)
#         mse_scores = cross_val_score(model,merged_data_scaler_test100, y_test100,
cv=5, scoring='neg_mean_squared_error')
#         r2_scores = cross_val_score(model,merged_data_scaler_test100,
y_test100,cv=5, scoring='r2')
#         mean_mse = -mse_scores.mean()
#         std_mse = mse_scores.std()
#         mean_r2 = r2_scores.mean()
#         std_r2 = r2_scores.std()
#
#
#         print([n_estimators,learning_rate,mean_mse,std_mse,mean_r2,std_r2])
#
model_data.append([n_estimators,learning_rate,mean_mse,std_mse,mean_r2,std_r2])
#
# min_matching_rows_entry = min(model_data, key=lambda x: x[2])
# print(min_matching_rows_entry)
# Cstart = 0.1
# Cend =10
# Cstep = 0.1
# estart = 0.01
# eend = 0.2
# estep = 0.01
# model_data = []
# for C in np.arange(Cstart, Cend + Cstep, Cstep):
#     for epsilon in np.arange(estart, eend + estep, estep):

```



```

#         svm_regressor = SVR(kernel='rbf',C=C, epsilon=epsilon)
#         scores = cross_val_score(svm_regressor, X, y, cv=5,
scoring='neg_mean_squared_error')
#         mean_mse = -scores.mean()
#         std_mse = scores.std()
#         print([C,epsilon,mean_mse,std_mse])
#         model_data.append([C,epsilon,mean_mse,std_mse])
# min_matching_rows_entry = min(model_data, key=lambda x: x[2])
# print(min_matching_rows_entry)
# for i in range(2,6,1):
#     x = [row[0] for row in model_data]
#     y = [row[1] for row in model_data]
#     z = [row[i] for row in model_data]
#     # 创建 xi 和 yi 的网格坐标
#     xi = np.linspace(min(x), max(x))
#     yi = np.linspace(min(y), max(y))
#     xi, yi = np.meshgrid(xi, yi)
#     zi = griddata((x, y), z, (xi, yi), method='cubic')
#
#     fig = plt.figure()
#     ax = fig.add_subplot(111,projection='3d')
#     surf = ax.plot_surface(xi, yi, zi, cmap='BuPu', linewidth=0, antialiased=False)
#     fig.colorbar(surf)
#     if(i == 2):
#         ax.set_title('均方误差随弱分类器数量与学习率的变化趋势')
#     if(i == 3):
#         ax.set_title('均方误差标准差随弱分类器数量与学习率的变化趋势')
#     if(i == 4):
#         ax.set_title('R-squared 均值随弱分类器数量与学习率的变化趋势')

```

```

#     if (i == 5):
#         ax.set_title('R-squared 标准差随弱分类器数量与学习率的变化趋势')
#         ax.set_xlabel('n_estimators')
#         ax.set_ylabel('learning_rate')
#         plt.show()

```

问题 3 (c) :

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
table1 = pd.read_excel('data\表 1-患者列表及临床信息.xlsx', index_col=0)
table2 = pd.read_csv('data\data_2_c.csv', encoding='gb2312', index_col=0)
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
table2 = table2[0:100]

t1 = table1.iloc[:, 0:]
t1.drop(columns=['性别'], inplace=True)
t1['高压'] = t1['血压'].str.split('/').str[0].astype(int)
t1['低压'] = t1['血压'].str.split('/').str[1].astype(int)
t1.drop(columns=['血压'], inplace=True)
t1.drop(columns=['数据集划分'], inplace=True)
t1.drop(columns=['入院首次影像检查流水号'], inplace=True)
t1.drop(columns=['发病到首次影像检查时间间隔'], inplace=True)

# cdf = clusterdf.iloc[:, :7]
corr_matrix = t1.corr(method='pearson')

plt.figure(figsize=(16, 12))
sns.heatmap(corr_matrix, annot=True)
plt.title('各因素与 90 天 mRS 相关性', fontsize=18)
plt.show()

excel_file_path = 'data\分析水肿血肿的关系.csv'
df = pd.read_csv(excel_file_path, encoding='gb2312', nrows=100)

```

```

column_names = df.columns[[0,1]]

tt = t1.iloc[:,0]
new_df = tt.reset_index(drop=True)
df['90 天 mRS']=new_df

corr_matrix = df.corr(method='pearson')

plt.figure(figsize=(12,9))
sns.heatmap(corr_matrix,annot=True)
plt.title('各因素相关性', fontsize=18)
plt.show()

l1df = pd.read_csv('data/T3_b.csv',encoding='gb2312',nrows=100)

l1df['90 天 mRS']=new_df
corr_matrix = l1df.corr(method='k')
plt.figure(figsize=(32,24))
sns.heatmap(corr_matrix,annot=True)
plt.title('影像特征与 90 天 mRS 相关性', fontsize=18)
plt.show()

```